DRUNIX

Building an Operating System from Scratch

Booting, memory, drivers, filesystems, processes, and a Unix shell

Drew Tennenbaum

First Edition • 2026

Copyright © 2026 Drew Tennenbaum. All rights reserved.

This book documents the Drunix operating system: a bare-metal x86 operating system built from scratch in C and x86 assembly, running under GRUB2 on 32-bit protected-mode hardware. It is written for readers who are technical but have not previously built an operating system.

Typeset with XeLaTeX and pandoc.

Contents

Part I — From Firmware to Kernel **1**
Chapter 1 — Booting with GRUB2 . . . 3
Chapter 2 — The GDT and Protected Mode . . . 8
Chapter 3 — The Kernel Entry and VGA Output . . . 12
Chapter 4 — Interrupts and Exceptions . . . 17
Chapter 5 — The IRQ Dispatch Registry . . . 25
Chapter 6 — SSE2 and CPU Extensions . . . 31

Part II — Managing Memory **35**
Chapter 7 — Detecting Memory . . . 36
Chapter 8 — Memory Management . . . 40

Part III — Hardware Interfaces **57**
Chapter 9 — Kernel Boot Log . . . 58
Chapter 10 — Keyboard Input . . . 62
Chapter 11 — ATA Disk Driver . . . 67
Chapter 12 — Block and Character Device Registries . . . 71

Part IV — Files and Storage **73**
Chapter 13 — An Inode-Based Filesystem (DUFS v3) . . . 74
Chapter 14 — The Virtual Filesystem Switch . . . 85

Part V — Running User Programs **91**
Chapter 15 — ELF Loader and Process Management . . . 92
Chapter 16 — System Calls . . . 106
Chapter 17 — File I/O and Open File Descriptors . . . 110

Part VI — The User Environment **113**
Chapter 18 — The TTY Line Discipline . . . 115
Chapter 19 — Signals . . . 121
Chapter 20 — User-Space Runtime Library . . . 129
Chapter 21 — A Minimal C Standard Library . . . 133
Chapter 22 — The Interactive Shell . . . 138

Part VII — Extending the Kernel **143**
Chapter 23 — Runtime Loadable Modules . . . 144

Chapter 24 — Core Dumps 148

Part VIII — Fault-Driven Memory **155**

Chapter 25 — Page Faults and Demand Paging 156

Chapter 26 — Copy-on-Write Fork 163

Chapter 27 — Debugging 169

Part I — From Firmware to Kernel

When the machine powers on, it is not running your code. The CPU starts in real mode — a 16-bit addressing mode inherited from the 1970s — and begins executing firmware stored in ROM (read-only memory on the motherboard). That firmware is responsible for initializing the hardware and finding something to boot. What it finds is GRUB, and what GRUB finds is the kernel binary. By the time GRUB hands control over, the CPU is still in a constrained state: 32-bit protected mode is enabled, but there is no interrupt table, no configured segment descriptors beyond what GRUB provided, no way to handle exceptions, and no access to any of the extended CPU features that modern code depends on. The machine is alive but barely governable.

This part is about taking full control of that machine. The problem is not any single missing piece — it is that several pieces must all be in place before any of them can be used safely. The CPU needs a properly configured Global Descriptor Table — a structure that defines the memory regions kernel and user code are allowed to use — before it can enforce privilege levels. It needs an interrupt descriptor table — a lookup table that tells the CPU what code to run when an exception or hardware event fires — before any hardware event can be handled without crashing. It needs the programmable interrupt controller remapped before hardware interrupt requests arrive on the right numbered vectors. And it needs the floating-point and vector instruction extensions enabled before any code that touches those registers can run. None of these steps produces a visible result on their own; together they bring the kernel to a state where everything that follows is possible.

Chapter 1 follows the hand-off from GRUB: what the Multiboot protocol provides, where the kernel binary lands in memory, and what the linker script must guarantee before the first C instruction executes. Chapter 2 builds the Global Descriptor Table and makes the flat 32-bit memory model explicit, establishing the segment selectors that all kernel and user code will use from this point forward. Chapter 3 takes the first output: the kernel writes to the VGA framebuffer — the block of memory the graphics card reads to place pixels on screen — to confirm it is running, establishes a scrollback shadow buffer, and sets up cursor control through the hardware's I/O port interface. Chapter 4 installs the interrupt descriptor table and remaps the 8259 Programmable Interrupt Controller — the chip that routes hardware signals from devices into CPU interrupts — so that device interrupts arrive on vectors that do not collide with CPU exceptions. Chapter 5 adds a dispatch registry on top of the raw interrupt descriptor table so that individual drivers can register handlers without knowing the interrupt vector layout, and wires up the Programmable Interval Timer to produce a periodic tick. Chapter 6 enables the SSE2 floating-point extensions — a set of CPU instructions that operate on wide registers used for math-heavy code — by setting the control register bits that gate their use, then captures a clean floating-point state template that every future process will inherit.

By the end of Part I the machine is fully under kernel control. Exceptions and hardware interrupts are handled. The address space is flat and well-defined. The CPU's extended instruction set is available. The path is clear to build everything else on top.

Chapter 1 — Booting with GRUB2

The Moment the Machine Wakes Up

When you press the power button, the processor starts executing instructions from a fixed address in **firmware** — a permanently installed program stored on a chip on the motherboard. On older machines this firmware is called the **BIOS** (Basic Input/Output System); on newer ones it is called **UEFI** (Unified Extensible Firmware Interface). Both do the same thing at this stage: they scan the attached disks looking for something bootable and hand control to the first program they find.

At this instant the CPU is running in **16-bit real mode**. Real mode is a backwards-compatibility mode in which the processor behaves like an Intel 8086 from 1978: 16-bit registers, 1 MB of addressable memory, and no memory protection of any kind. A bootloader has to live with these restrictions until it can switch the CPU into a modern mode.

Writing a bootloader that handles all of this reliably — across USB sticks, real hard disks, and emulators — is a small project on its own. It involves calling BIOS services to read sectors, parsing disk formats, setting up descriptor tables, and flipping the CPU into 32-bit mode. Rather than reinvent that work, we delegate it to **GRUB2**, the GNU GRand Unified Bootloader. GRUB is the same program that loads every major Linux distribution, so it is battle-tested on every PC you are likely to encounter.

By the time GRUB hands control to our kernel, it has already done four things on our behalf:

1. Loaded the kernel image from the bootable disc into RAM.
2. Switched the CPU from 16-bit real mode into 32-bit **protected mode** — the modern mode where the full 4 GB address space is visible and memory protection is available.
3. Asked the BIOS for a map of usable RAM (the details of that map are covered in Chapter 7).
4. Jumped into our kernel at a known entry point with a documented calling convention.

That last point is what makes GRUB useful: there is a published standard, called **Multiboot**, that describes exactly how GRUB hands control over. If our kernel follows the Multiboot rules, GRUB will load it; if it does not, GRUB refuses and prints an error.

How GRUB Recognises Our Kernel

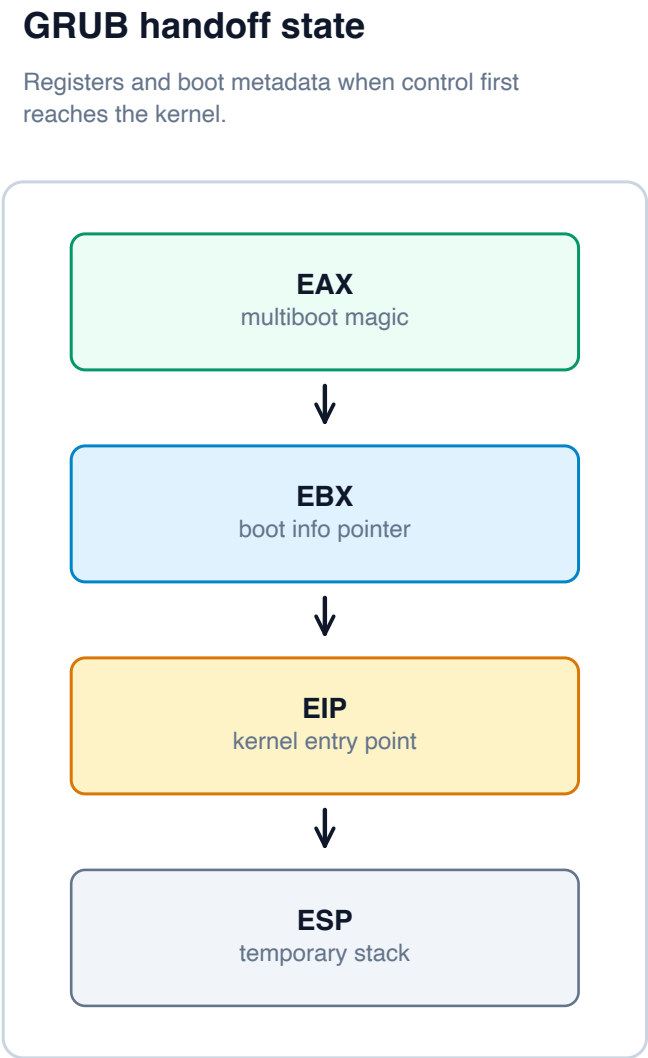
The Multiboot1 specification has two sides. The kernel must publish a small header — a fixed pattern of bytes within the first 8 KB of the executable file — that tells GRUB "I am Multiboot-compliant". GRUB, in return, promises to jump to the kernel with a specific CPU state and with a pointer to a structure describing the machine.

The header is embedded near the start of the kernel binary. It is twelve bytes long: a magic number GRUB scans for (`0x1BADB002`), a flags field (we use `0`, the minimum), and a checksum that makes the three values add up to zero modulo 2^{32}. The linker is instructed to place the header at the very front of the binary so it always lands within GRUB's 8 KB scan window.

When GRUB finds the header, validates the checksum, loads the kernel, and jumps into it, two registers carry information across the handoff boundary:

- `EAX` (the accumulator register) contains the value `0x2BADB002` — a different magic number that tells the kernel it was loaded by a Multiboot-compliant bootloader and that the other register can be trusted.
- `EBX` (the base register) contains a 32-bit physical address pointing to a **Multiboot info structure** — a block of memory GRUB filled in with the RAM map, the boot command line, and any modules loaded alongside the kernel.

The CPU state at the moment GRUB transfers control to `_start` looks like this:



Chapter 1, Figure 1

`EIP` is the instruction pointer — the register the CPU always advances to point at the next instruction to execute. GRUB sets it to the address of `_start` by jumping there. `ESP` is the stack pointer; its value is undefined on entry, which is the first problem the assembly stub has to fix.

The Assembly Entry Point

Our entry point is the symbol `_start`, marked by the linker as the executable's starting address. To find it, GRUB reads the **ELF** (Executable and Linkable Format, the standard binary container on Linux/x86) header, locates `_start`, and jumps there.

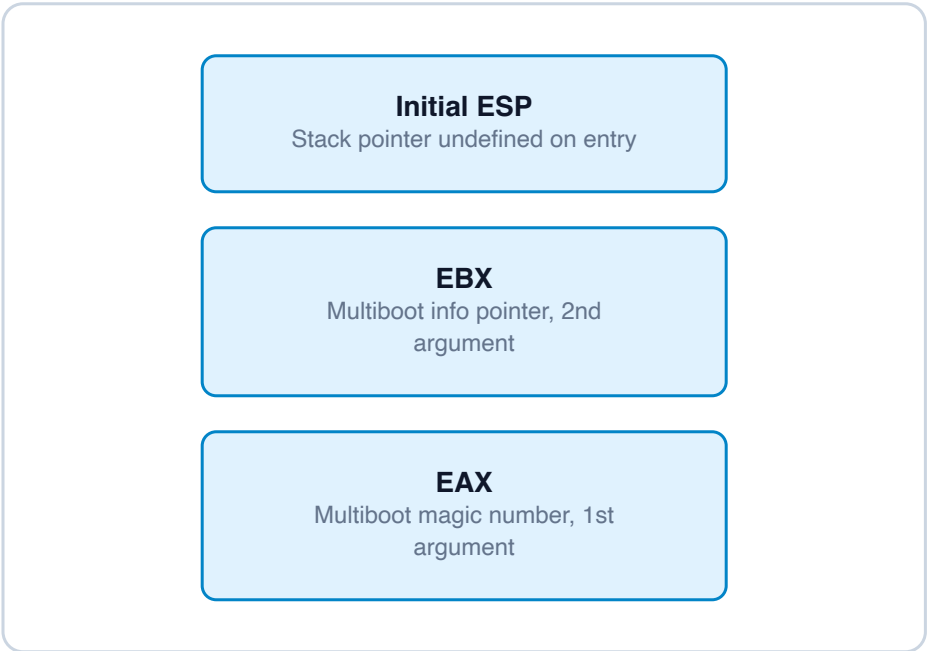
`_start` does three things in order before any C code runs.

First, it sets up a stack. GRUB leaves `ESP` in an undefined state, so the very first instruction points `ESP` at a block of memory the kernel owns. That block is sixteen kilobytes of space reserved in the kernel's **BSS** segment — the region of an executable that holds uninitialised globals. BSS costs nothing in the file on disk (the loader just zeroes the range at startup), but it is real allocated RAM once the image is loaded.

Second, `_start` preserves the Multiboot values. The moment any C function is called, the compiler is free to overwrite `EAX` and `EBX` with other values, so those registers must be saved to the stack before the first call. The values are pushed in the order required by the standard x86 **cdecl calling convention**, so that `start_kernel` receives them as ordinary C function arguments.

Entry stack after _start pushes Multiboot values

Stack grows downward; EAX lands at the lowest address and becomes the first argument.



Chapter 1, Figure 2

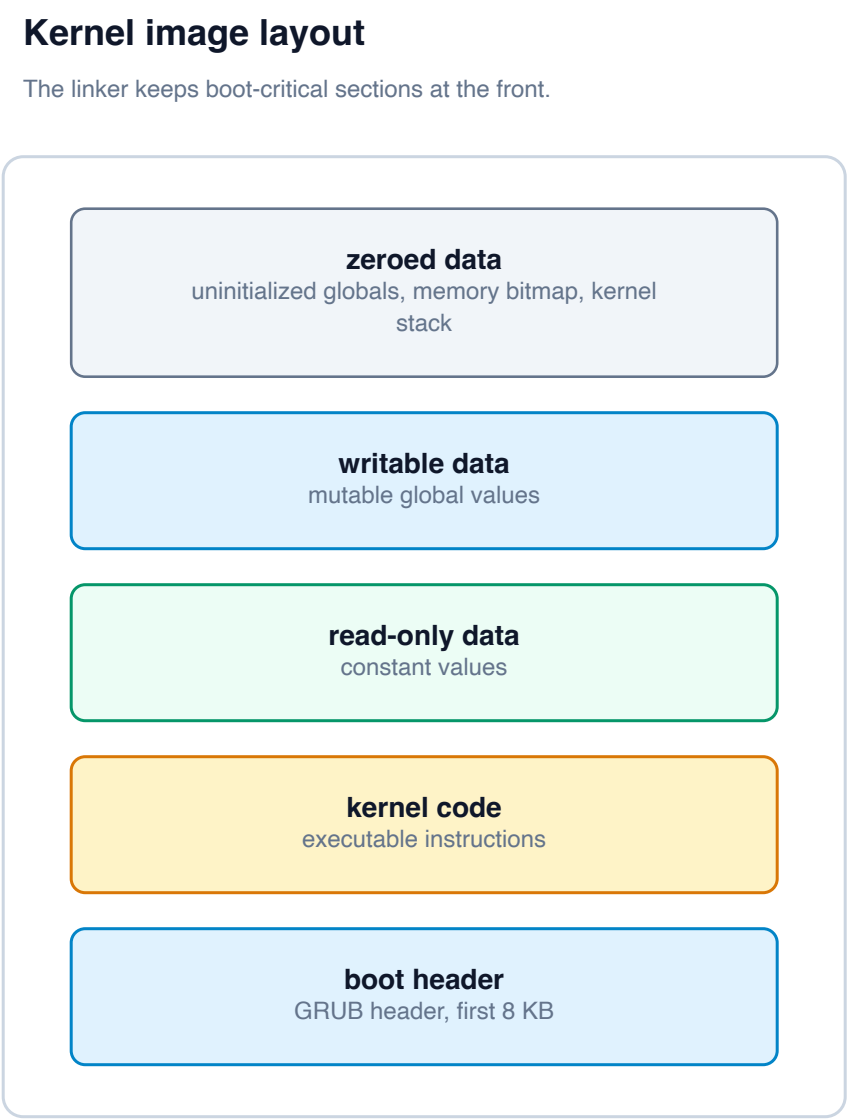
Third, `_start` calls `start_kernel`. From C's perspective this looks like any other function call; the assembly trampoline has taken a raw machine state and shaped it into a valid C call frame.

If `start_kernel` ever returns — it should never exit — `_start` executes an infinite self-loop to prevent the CPU from falling off the end of the code and interpreting random bytes as instructions.

Where the Kernel Lives in Memory

We load our kernel at **1 megabyte** (`0x100000`), fixed at link time. This is the conventional load address for a Multiboot kernel. The reason is that everything below one megabyte on an x86 PC is a patchwork of legacy regions — the BIOS interrupt vector table, the VGA video buffer, BIOS ROM shadows — and none of it is safe to use as general RAM. Starting at one megabyte puts the kernel in the "extended memory" region, which is guaranteed to be real, usable RAM.

The linker script lays the sections out in a fixed order:



Chapter 1, Figure 3

The linker is told to keep the Multiboot header at the very beginning of the binary, so no matter how large the kernel grows the header always sits within GRUB's 8 KB scan window.

The linker exposes symbols that mark the start and end boundaries of the kernel image. The physical memory manager in Chapter 8 reads these symbols at boot time to learn exactly which physical pages the kernel occupies, so it can mark them as reserved and never hand them out as general allocations.

Building the Bootable Image

The build turns our kernel source into a bootable ISO image — a disc format that **QEMU** (Quick Emulator, the open-source hardware emulator we use to run our kernel on a virtual machine) treats exactly as a real PC treats a CD. We compile and link the kernel into an ELF binary positioned at `0x100000`. A GRUB configuration file holds a single menu entry telling GRUB to load that binary using the Multiboot protocol — that is all GRUB needs from us. The tool `grub-mkrescue` then assembles the staging directory, a GRUB stage-1 loader, and the necessary boot records into one self-contained bootable ISO.

We launch QEMU with that ISO as a virtual optical drive and a separate image as a primary **ATA** (Advanced Technology Attachment — the interface protocol connecting hard drives to the system bus) hard drive. The virtual firmware finds GRUB on the disc, GRUB loads our kernel, and the hard-drive image sits untouched until the ATA driver in Chapter 11 begins reading sectors from it.

The first time this works — QEMU opens, GRUB's menu appears, and a moment later our own kernel is executing on what looks to it exactly like real hardware — is a genuine milestone. Every instruction the CPU runs from that point on is code we wrote.

Where the Machine Is by the End of Chapter 1

At this moment, just before `start_kernel` runs:

- GRUB has finished. It will never run again.
- The CPU is in 32-bit protected mode, but with only the minimal descriptor tables GRUB set up for its own use. Interrupts are disabled.
- The kernel binary is loaded at physical address `0x100000`, with every section placed according to the linker script.
- A 16 KB stack is set up in the kernel's BSS segment, and `ESP` points at its top.
- The Multiboot magic number and the info-structure pointer are sitting on the stack, ready to be received by `start_kernel` as ordinary C function arguments.

From here the kernel takes over completely. Chapter 2 covers the next step: building a proper **GDT** (Global Descriptor Table — the structure that defines memory segments and privilege levels for the CPU) and taking full ownership of the processor's mode.

Chapter 2 — The GDT and Protected Mode

What the CPU Needs Before It Can Behave Like a Modern Processor

Chapter 1 ended with GRUB having placed the CPU in protected mode — the 32-bit operating mode where the full 4 GB address space is visible. That sounds like the hard work is done, but protected mode is not a single state. It runs on top of a mandatory data structure, and if that structure is missing or malformed the CPU will fault the moment it tries to execute anything.

The structure is called the **Global Descriptor Table**, abbreviated **GDT**. GRUB installs a minimal one before jumping to our kernel — just enough for it to function while loading the kernel image — but that table belongs to GRUB's memory and is not guaranteed to remain valid. So the very first thing we do in `start_kernel` is build a permanent GDT in kernel memory and load it. This chapter explains what the GDT is, what goes in it, and how we take ownership of the CPU mode.

Why Protected Mode Requires a Descriptor Table

Before protected mode, the CPU operates in **real mode**, a backwards-compatibility mode modelled on the Intel 8086. Two fundamental defects make real mode unsuitable for any serious operating system.

First, registers are only 16 bits wide. A single register can hold a value between 0 and 65,535, so pointing at anything further away requires combining two registers using awkward segment arithmetic. The effective address is `(segment register × 16) + offset`, limiting the total addressable space to one megabyte.

Second, there is no memory protection whatsoever. Any instruction can read or write any byte of memory — including the BIOS data area, the kernel code, or a neighbour process's private data. There is no mechanism to isolate a buggy or malicious program.

Protected mode fixes both problems: registers are 32 bits wide (enabling a 4 GB address space), and the hardware can enforce access rules. But "the hardware can enforce rules" requires telling the hardware what those rules are. That is exactly what the GDT does — it is the table we hand to the CPU to describe how memory is partitioned and who is allowed to touch what.

What a GDT Entry Actually Is

A GDT is an array in RAM. Each eight-byte entry is called a **segment descriptor** and describes one region of memory: where it starts, how large it is, and what operations are permitted inside it (read, write, execute) and at what privilege level.

The eight-byte layout has an unfortunate history. The original Intel 80286 used a six-byte format with a 24-bit base address and a 16-bit limit. When the 80386 extended the processor to 32 bits, Intel added the missing bits to the two bytes at the end rather than redesigning the format. The result is that the base address and limit are each split across two non-contiguous regions of the descriptor, and any OS targeting x86 — including ours — must pack and unpack them through this awkward layout:

Field	Bytes	Meaning
Limit 0-15	`0–1`	Low 16 bits of segment limit
Base 0-15	`2–3`	Low 16 bits of base address
Base 16-23	`4`	Base address bits 16-23
Access	`5`	Presence, privilege, and type flags
Flags and limit high	`6`	Granularity flags and limit bits 16-19
Base 24-31	`7`	Base address bits 24-31

When code loads a **segment selector** into one of the CPU's segment registers — `CS` for the current code segment, `DS`/`ES`/`FS`/`GS` for data, and `SS` for the stack — the CPU uses that selector as an index into the GDT and reads the corresponding descriptor to determine what the register means. Every memory access is then validated against the permissions in that descriptor.

A selector is a 16-bit value with three sub-fields:

Bits	Field	Meaning
`0–1`	RPL	Requested privilege level
`2`	TI	Global or local table
`3–15`	Index	Which descriptor slot to use

The **RPL** (Requested Privilege Level) field encodes the privilege level at which the memory access is being requested. The **TI** (Table Indicator) bit selects between the global GDT and a per-process **LDT** (Local Descriptor Table); we use only the GDT in this project.

We tell the CPU where the GDT lives in memory through a special register called **GDTR**. We load it with the `lgdt` instruction, which takes a pointer to a six-byte structure giving the table's base address and limit:

Field	Bytes	Meaning
Limit	`0–1`	Table size minus one
Base address	`2–5`	Physical address of first entry

The CPU copies these two fields into GDTR when `lgdt` executes.

The Three Descriptors in the Boot GDT

Our initial GDT has exactly three entries. Each descriptor carries a **DPL** (Descriptor Privilege Level) — a two-bit value that controls which CPU ring level may load the selector. Ring 0 is the most privileged level and is where kernel code runs; ring 3 is where user programs run.

The first entry is required by the CPU specification to be a **null descriptor** — eight zero bytes. Loading a null selector into a segment register is permitted, but using that register to address memory causes the CPU to raise a fault. This acts as a safety net: if a bug leaves a segment register uninitialised, the next memory access crashes loudly and visibly rather than silently reading garbage.

The second entry is the **code segment descriptor**. It covers the entire 4 GB address space starting at address zero and is marked executable and readable at ring 0. The flat, full-address-space coverage is an intentional design choice called a **flat memory model**: segmentation is neutralised entirely, and every address the kernel produces is passed through to the hardware unchanged. All memory protection in this project is instead handled by paging (Chapter 8).

The third entry is the **data segment descriptor**. It too covers the full 4 GB range, but is marked as a writable data segment rather than an executable code segment, also at ring 0. We load this selector into `DS`, `SS`, `ES`, `FS`, and `GS`.

Offset	Selector	DPL	Type	Base	Limit	Purpose
`0x00`	`0x00`	—	Null	—	—	Required trap for uninitialised selectors
`0x08`	`0x08`	0	Code	`0x00000000`	4 GB	Kernel code segment
`0x10`	`0x10`	0	Data	`0x00000000`	4 GB	Kernel data and stack segment

The byte offset of each descriptor within the table *is* its selector value: the null descriptor is at offset 0, the code descriptor at offset 8 (`0x08`), and the data descriptor at offset 16 (`0x10`). These two constants — `0x08` and `0x10` — appear throughout the kernel wherever segment registers are set.

This three-entry table is a temporary scaffold. Chapter 15 describes how we expand it to six entries by adding ring-3 user code and data segments and a **TSS** (Task State Segment, a hardware structure needed for privilege-level transitions). The kernel and data selectors deliberately remain at `0x08` and `0x10` after that expansion, so the interrupt handlers installed in Chapter 4, which hard-code `0x10` when reloading `DS` after an interrupt, continue working without modification.

Decoding the Permission Bits

Each descriptor carries an **access byte** that the CPU checks on every memory reference. Its eight bits encode:

Bits	Field	Meaning
`0`	Accessed	Set by the CPU on use
`1`	Readable	Read or write permitted
`2`	Conforming	Privilege-transition rule
`3`	Executable	CPU may fetch instructions
`4`	S	Code or data segment
`5–6`	DPL	Privilege level
`7`	Present	Entry is valid

The code segment's access byte is `10011010`:

- **Present = 1** — the descriptor is valid.
- **DPL = 00** — ring 0, kernel privilege only.

- **S = 1** — this is a code or data segment (as opposed to a system descriptor like a TSS or gate).
- **Executable = 1** — the CPU may fetch instructions from this segment.
- **Conforming = 0** — the segment does not allow lower-privilege code to call into it without a privilege transition.
- **Readable = 1** — the segment may also be read as data, which is necessary when the compiler places constants adjacent to code in the `.rodata` section.

The data segment's access byte is `10010010` — identical except the executable bit is cleared and the readable bit is set to writable.

How the Kernel Loads Its Own GDT

The sequence from the moment `start_kernel` begins running through the moment the new GDT is live unfolds in three steps.

First, the kernel fills in the three descriptor entries. Each descriptor's base and limit are packed into the non-contiguous field layout described earlier, so the flat values have to be split across the correct byte positions inside each eight-byte `gdt_entry_t`.

Next, the kernel stores the base address and limit of the completed table into the six-byte format `lgdt` expects.

Finally, the kernel executes `lgdt` to load the new table's address into GDTR. Loading GDTR alone is not enough: the CPU's segment register cache (sometimes called "shadow registers") still holds the descriptor values from GRUB's old GDT. To force the CPU to re-read all the descriptors from the new table, the startup code performs a **far jump** — a jump instruction that explicitly includes a segment selector — using selector `0x08`. A far jump unconditionally reloads `CS` and flushes the instruction pipeline, guaranteeing that all subsequent instruction fetches consult the new GDT. After the jump, the kernel loads `0x10` into every data segment register before returning.

The far jump is the critical step. Without it, `CS` would still reference whatever GRUB's GDT said, and our new GDT's code descriptor would never take effect.

Where the Machine Is by the End of Chapter 2

At this point in `start_kernel`, before any other subsystem is initialised:

- The CPU is in 32-bit protected mode, and our kernel's own GDT is loaded and active.
- The GDT contains a null descriptor, a ring-0 code descriptor, and a ring-0 data descriptor — all covering the full 4 GB address range.
- `CS` holds `0x08`; `DS`, `SS`, `ES`, `FS`, and `GS` all hold `0x10`.
- Interrupts are still disabled — the **IDT** (Interrupt Descriptor Table, the structure that maps interrupt numbers to handler addresses) has not been installed yet.
- All memory protection is currently handled by segmentation alone, which is effectively a no-op because both descriptors span the entire address space. Paging, which provides real isolation, is not enabled until Chapter 8.

We now own the descriptor table. The CPU is running under our rules, not GRUB's. Chapter 3 covers the kernel entry code in more detail and follows `start_kernel` as it begins initialising each subsystem in turn.

Chapter 3 — The Kernel Entry and VGA Output

Where the CPU Lands When C Begins

Chapter 2 ended with our own **GDT** (Global Descriptor Table) in place and the CPU running in 32-bit protected mode under our rules rather than GRUB's. The next instruction after that setup is a plain C function call: the assembly stub hands control to `start_kernel`, and from that point on the machine is mostly following code the C compiler generated for us.

`start_kernel` receives the same two values Chapter 1 left on the stack: the Multiboot magic number and a pointer to the **Multiboot info structure**. A Multiboot info structure is a block of boot-time metadata GRUB prepared for us, including flags and the firmware's memory map. `start_kernel` is not the kernel's permanent event loop, but it is the one straight-line bootstrap path that brings the machine from "GRUB just jumped here" to "the first user process is running".

The Startup Sequence

The body of `start_kernel` is a strict sequence. The order is not cosmetic. Each step either depends on state created by the previous one or is deliberately delayed until the machine is safe enough to survive it.

The first step makes the CPU's floating-point and SIMD state usable early enough that later process code can save and restore it correctly. **SSE** stands for Streaming SIMD Extensions, and **SIMD** means Single Instruction, Multiple Data: one instruction operating on several values at once. We are not turning the whole kernel into vectorized code here. We are simply claiming ownership of that part of the CPU before any later subsystem depends on it.

Next comes memory setup. The kernel copies the Multiboot flags out of the boot structure before physical-page tracking begins, because the page-tracking metadata may overwrite the place where GRUB left that structure behind. The **PMM** (Physical Memory Manager, the part of the kernel that tracks which 4 KB physical pages are free) then walks the firmware memory map and marks usable and reserved page frames.

Immediately after that, the kernel turns on paging. Paging means the CPU stops treating addresses as direct physical locations and instead sends them through the **MMU** (Memory Management Unit, the hardware that translates virtual addresses into physical ones). We are still early in the boot, but from this point forward the CPU is running with virtual memory active.

Once paging is live, the kernel activates its heap allocator so that any later initialization code can request and release variable-sized memory blocks on demand. Right after that, whatever text is already visible in the VGA buffer is captured into an in-memory mirror so the console driver can take over cleanly. That detail matters because the console driver is about to become the narrator for every later bootstrap step.

The next milestone is rebuilding the descriptor tables under full kernel control. The small boot-time GDT is replaced with the kernel's permanent one: null, kernel code, kernel data, user code, user data, and a **TSS** (Task State Segment, a hardware structure the CPU uses when switching from user privilege back into kernel privilege). The kernel code and data selectors stay at `0x08` and `0x10`, because later interrupt-entry code expects those exact selector values.

After the GDT comes the first real interrupt table under kernel control: the **IDT** (Interrupt

Descriptor Table, the table that tells the CPU which handler belongs to each exception or interrupt vector). At this moment the interrupt flag is still clear, so ordinary hardware interrupts cannot arrive yet. But exceptions, breakpoints, and the basic trap path now have somewhere valid to go, which makes the machine much less fragile.

Only then does the kernel start wiring up devices. It clears the **IRQ** (Interrupt ReQuest) dispatch table, gives the **PIT** (Programmable Interval Timer, the hardware timer that ticks at a fixed rate) ownership of IRQ 0, reads the real-time clock once to seed the wall clock, and prepares the **TTY** (TeleTYpewriter, the terminal input layer that turns raw key events into line-oriented input) so keyboard interrupts can become line-oriented input.

Storage comes up next. The **ATA** (AT Attachment) disk driver is initialised and published into the block-device registry so higher layers can find the disk by name. Then the DUFS filesystem implementation is published into the filesystem registry so the virtual filesystem layer can mount it later.

Only after those pieces are ready does the kernel perform the dangerous part: it remaps the PIC, programs the timer hardware, and executes `sti`, the instruction that finally lets maskable hardware interrupts reach the CPU. This is the line where the kernel stops being a carefully choreographed single-threaded bootstrap and becomes a machine that can be interrupted at almost any instruction boundary.

With interrupts live, the kernel mounts three namespaces through the **VFS** (Virtual File System, the layer that gives the rest of the kernel one uniform file API no matter which underlying filesystem answers a path lookup): DUFS at `/`, a synthetic device tree at `/dev`, and a synthetic process-information tree at `/proc`.

The last stretch launches the first user program. The scheduler table is cleared, the shell executable is located on disk, loaded as an **ELF** (Executable and Linkable Format, the standard executable file format on x86/Linux) image, marked runnable, and selected as the first process to run. One final privilege drop switches into ring 3 and does not return. From that instant on, the shell is in user mode, the timer can preempt it, and the kernel is reacting to events instead of marching through startup code.

Why the Screen Driver Matters So Early

All the way through that sequence, the kernel keeps printing status lines. At this stage there is no terminal emulator, no window system, and no pixel-addressed framebuffer driver. The only output device we can rely on is the **VGA** (Video Graphics Array) text buffer.

The VGA text buffer is a region of physical memory at address `0xB8000`. The display hardware continuously reads from that memory and interprets it as an 80-column by 25-row grid of character cells. If we change bytes in that region, the next refresh shows different characters on the monitor. There is no higher-level protocol involved yet. The “driver” is just memory writes plus a few controller-register updates.

Each screen cell occupies two consecutive bytes. The first byte is the character code, stored as **ASCII** (American Standard Code for Information Interchange, the classic one-byte character encoding for English letters, digits, and punctuation). The second byte is an attribute byte: its low four bits encode the foreground colour and its high four bits encode the background colour. The kernel’s default attribute is `0x0F`, which means bright white text on a black background.

The buffer is laid out row by row as one flat 4,000-byte region:

Chapter 3, Figure 1

Each attribute byte is split into two colour nibbles:

Bits	Field	Example
`0–3`	Foreground colour	Bright white
`4–7`	Background colour	Black

Given a row `r` and column `c`, the byte offset into the buffer is `2 * (r * 80 + c)`. Writing a character means placing the ASCII code at that offset and the attribute byte at the next byte. A tiny helper does exactly this pair of writes for every visible character.

The Blinking Cursor

The underline cursor that marks the next write position is not stored in the text buffer itself. It lives in VGA controller registers. To reach those registers, the kernel uses **I/O ports**: a second x86 address space that is separate from ordinary memory and can only be accessed with the `IN` and `OUT` instructions.

The VGA controller exposes two ports at `0x3D4` and `0x3D5`. Port `0x3D4` is an index register: writing a value there selects which internal VGA register to access next. Port `0x3D5` is the data register. The cursor position occupies two internal registers: register `0x0E` holds the high byte and `0x0F` holds the low byte of the cursor's linear character index (not byte offset). To move the cursor, the kernel writes the index and then the byte to each port in turn.

Because C has no built-in syntax for `IN` or `OUT`, the kernel wraps those instructions in tiny inline-assembly helpers. From this point on, that wrapper becomes part of everyday kernel life: the VGA controller, keyboard controller, timer, real-time clock, and ATA controller all talk through I/O ports.

Printing Strings

`print_string` walks a null-terminated string one byte at a time. For each byte it either interprets a control character or draws one character cell:

- A newline (`\n`) advances the cursor to the start of the next row.
- A carriage return (`\r`) moves the cursor to column zero of the current row.
- A backspace (`\b`) moves the cursor back one cell, writes a space to erase the character there, and leaves the cursor ready for the next keystroke. This is the mechanism the shell uses to display character deletion.
- A tab (`\t`) expands into spaces up to the next multiple of eight columns.
- Any other byte is written to the current cell and the cursor advances.

The function also understands a small subset of ANSI colour escape sequences, so later kernel log lines can switch colours without changing the basic text-mode design. After each visible character, it checks whether the cursor has fallen off the bottom of the 25-row screen. If it has, the screen is scrolled upward by one line before output continues.

A second entry point, `print_bytes`, uses the same output path but stops after a caller-supplied byte count instead of waiting for a terminating zero byte. That matters later when the kernel needs to print arbitrary byte streams that may contain embedded `0x00` bytes.

Scrolling

Scrolling happens when the cursor would move past the final row. The kernel copies rows 1 through 24 upward into rows 0 through 23, then clears row 24 to spaces in the default colour. It performs that copy twice: once in the hardware VGA buffer at `0xB8000`, and once in the kernel's shadow copy of the screen.

The Scrollback Buffer

Before the driver discards the top visible row, it saves that row into a 500-row ring buffer called `scrollback`. A ring buffer is a fixed-size array treated as a loop: when the write position reaches the end, it wraps back to the beginning and starts overwriting the oldest entries.

Three pieces of state work together to make scrollback possible:

Structure	Stored state	Purpose
Scrollback ring	Rows that already scrolled off the top of the live screen	Provides history rows when the user looks backward
Shadow VGA	Copy of the 25 live screen rows	Preserves the current visible text independent of hardware redraws

Structure	Stored state	Purpose
View offset (`sb_view`)	How many rows back from live view are being shown	Chooses whether redraw uses only live rows or a history-and-live mix
Saved cursor (`shadow_cursor`)	Logical write position in the live screen image	Restores the cursor when the display returns to live view

The scrollback ring holds the rows that have already disappeared off the top of the live screen. The **shadow VGA** is a 4,000-byte in-memory mirror of the current 25 visible rows. Every visible write updates both the hardware buffer and this mirror, so the kernel always has an authoritative copy of the live screen contents even when the hardware cursor is hidden or the user is looking backward through history.

The integer `sb_view` says how far back the user is looking. When `sb_view` is zero, the display shows the live screen exactly as recorded in `shadow_vga`. When `sb_view` is `N`, the repaint routine takes `N` rows from scrollback history and places them above the lower `25 - N` rows from the live shadow screen. Scrolling backward and forward simply adjusts `sb_view` and redraws the hardware buffer from that combined view.

A separate variable, `shadow_cursor`, tracks the logical write position even when the hardware cursor is hidden. When `sb_view` is non-zero, the driver pushes the hardware cursor off-screen so it does not appear in the middle of old history. The moment new output arrives, the driver snaps the view back to live mode, restores the visible cursor, and continues writing at `shadow_cursor`.

Where the Machine Is by the End of Chapter 3

By the end of this chapter, the CPU has crossed the line from "bootstrapping the kernel" to "running a real system":

- The one-time bootstrap path has finished.
- The kernel owns the GDT and IDT, paging is active, and the heap is available.
- The timer, keyboard, TTY layer, ATA driver, and filesystem registrations are in place.
- Hardware interrupts are enabled, so the CPU can now be preempted by timer ticks and input events.
- The VGA text driver can draw characters, manage colours, move the cursor, scroll, and show history from its 500-row scrollback ring.
- DUFS is mounted at `/`, with `devfs` at `/dev` and `procfs` at `/proc`.
- The shell has been loaded from disk and launched in ring 3 as the first user-space process.

The important thing to hold in your head going into Chapter 4 is that the machine is no longer a fragile boot stub. We have a working kernel console, a valid trap path, and a live user process. From here on, when something goes wrong, the CPU has somewhere to go and the kernel has somewhere to tell us about it.

Chapter 4 — Interrupts and Exceptions

When Straight-Line Execution Stops

Chapter 3 ended with the kernel still executing in a straight line. The **GDT** (Global Descriptor Table) is live, the early **IDT** (Interrupt Descriptor Table) is loaded, and the machine is one `sti` instruction away from becoming truly asynchronous. The important change in this chapter is not just that we fill in another table. It is that, from this point onward, the CPU may stop the current instruction stream between two instructions and enter the kernel for a completely different reason.

That is what an **interrupt** is. An interrupt is a signal that makes the CPU save enough state to resume later, switch to a handler chosen by an interrupt vector number, and run that handler before returning to the interrupted code.

On x86 there are two broad kinds of interrupt.

The first is a **CPU exception**. An exception is raised by the processor itself when the current instruction cannot complete normally: divide by zero, invalid opcode, page fault, and so on. Exception vectors 0 through 31 are defined by the architecture. Many of them also have short mnemonics such as `#DE` (Divide Error), `#UD` (Invalid Opcode), `#GP` (General Protection fault), and `#PF` (Page Fault). Some exceptions push an extra 32-bit **error code** onto the stack before entering the handler. That error code gives the kernel more detail about why the fault happened.

The second is a **hardware IRQ** (Interrupt ReQuest). A hardware IRQ is raised by a physical device that wants service: the timer fires, the keyboard receives a scancode, or the disk controller finishes a transfer. On the classic PC-compatible x86 machine these signals first pass through the **PIC** (Programmable Interrupt Controller, specifically the 8259A pair), which forwards one pending IRQ to the CPU as an interrupt vector.

Both sources use the same lookup table. The IDT has 256 entries, and each entry tells the CPU which handler address to jump to, which code segment to enter, and what privilege rules apply to that entry.

Once the kernel has finished setting the table up, the vector space it actively uses looks like this:

Vector	Source	Meaning
0–31	CPU	Architectural exceptions (`#DE`, `#UD`, `#GP`, `#PF`, …)
32–39	Master PIC	Remapped IRQs 0–7 (timer, keyboard, …)
40–47	Slave PIC	Remapped IRQs 8–15 (RTC, ATA, …)
48–127	—	Unused
128	Software `int 0x80`	System-call entry gate
129–255	—	Unused

The Shape of an IDT Entry

Each IDT entry is eight bytes long. Intel calls this entry a **gate** because it is a controlled way for execution to cross into a handler. In C the layout is:

```c
typedef struct {
    uint16_t offset_low;   // handler address, bits 0–15
    uint16_t selector;     // code segment selector (0x08 = kernel CS)
    uint8_t  zero;         // must be zero
    uint8_t  type_attr;    // present bit, privilege level, gate type
    uint16_t offset_high;  // handler address, bits 16–31
} __attribute__((packed)) idt_entry_t;
```

The handler address is split into a low half and a high half. The `selector` field names the code segment the CPU should load when entering the handler. In this kernel that selector is always `0x08`, the kernel code segment established by the GDT. The `type_attr` byte says whether the entry is present, which privilege ring may trigger it with a software interrupt, and what kind of gate it is.

The full eight-byte entry looks like this:

Bytes	Field	Meaning
`0–1`	Offset low	Low half of the handler address
`2–3`	Selector	Kernel code segment selector
`4`	Zero	Required zero byte
`5`	Type and attributes	Present bit, privilege level, and gate kind
`6–7`	Offset high	High half of the handler address

The `type_attr` byte itself is packed as follows:

Bits	Field	Meaning
`0–3`	Gate kind	Interrupt gate, trap gate, or task gate
`4`	System bit	Kept clear for IDT gates
`5–6`	DPL	Highest caller privilege allowed for software entry
`7`	Present	Entry is valid

Most entries use `0x8E`: present, **DPL** (Descriptor Privilege Level) 0, 32-bit interrupt gate. An **interrupt gate** has one especially useful property: on entry, the CPU clears the interrupt enable flag in `EFLAGS`, so another hardware IRQ cannot interrupt the current handler halfway through.

Vector `0x80`, the software interrupt the kernel uses for system calls, is the exception. It uses `0xEF`: present, DPL 3, 32-bit **trap gate**. A **trap gate** does not clear the interrupt enable flag, which means timer ticks and keyboard IRQs can still arrive while the kernel is servicing a system call from user space.

There is one more special case: vector 8, `#DF` (Double Fault), is installed as a **task gate** instead of a normal interrupt gate. A task gate points at a dedicated **TSS** (Task State Segment, a CPU-defined structure that can hold a complete hardware task context, including a stack pointer). The kernel uses that task gate to enter the double-fault path on a known-good emergency stack even if the ordinary ring-0 stack is already damaged.

Why the PIC Must Be Remapped

Before the kernel can accept hardware IRQs, it has to correct an awkward piece of PC history. The original IBM PC wired IRQs 0 through 7 onto CPU vectors 8 through 15. That arrangement was acceptable in 16-bit real mode, but in protected mode those vector numbers already belong to CPU exceptions.

Left alone, the first timer tick would arrive on vector 8, which is the same vector the CPU uses for `#DF`:

IRQ line	Vector	Collision
IRQ 0	`8`	Collides with double fault
IRQ 1	`9`	Collides with a reserved exception slot
IRQ 2	`10`	Collides with invalid TSS
IRQ 3	`11`	Collides with segment not present
IRQ 4	`12`	Collides with stack-segment fault
IRQ 5	`13`	Collides with general protection fault
IRQ 6	`14`	Collides with page fault
IRQ 7	`15`	Collides with a reserved exception slot

That overlap is disastrous because the kernel could no longer tell whether vector 8 meant "the timer fired" or "the CPU hit a catastrophic exception path". The fix is to **remap** the PIC so the hardware IRQs live somewhere else in the IDT.

The function `pic_remap` sends the PIC four setup bytes called **ICW1** through **ICW4** (Initialisation Command Words). Together they move IRQs 0 through 7 to vectors 32 through 39, and IRQs 8 through 15 to vectors 40 through 47:

IRQ line	Typical source	Vector
IRQ 0	Timer	`32`
IRQ 1	Keyboard	`33`
IRQ 2	Cascade link to slave PIC	`34`
IRQ 3	Serial port	`35`
IRQ 4	Serial port	`36`
IRQ 5	Legacy peripheral	`37`
IRQ 6	Floppy controller	`38`
IRQ 7	Parallel port	`39`
IRQ 8	Real-time clock	`40`
IRQ 9	Legacy peripheral	`41`
IRQ 10	Legacy peripheral	`42`
IRQ 11	Legacy peripheral	`43`
IRQ 12	PS/2 mouse	`44`
IRQ 13	Math coprocessor	`45`
IRQ 14	Primary ATA channel	`46`
IRQ 15	Secondary ATA channel	`47`

After remapping, the kernel also writes an **interrupt mask** to each PIC. An interrupt mask is a byte in which each set bit disables one IRQ input line. The kernel begins with only IRQ 0 (timer) and IRQ 1 (keyboard) unmasked, because those are the only device handlers registered at this stage. Leaving an unhandled IRQ line enabled would let the CPU jump into a live vector whose higher-level software path does not exist yet.

The PIC is programmed entirely through **I/O ports** — the separate x86 port-address space introduced in Chapter 3. Ports `0x20` and `0x21` control the master PIC, and ports `0xA0` and `0xA1` control the slave PIC.

Loading the Table Before Enabling Interrupts

Once the kernel has filled in the IDT entries, it loads the table with `lidt`. The `lidt` instruction writes the base address and size of the IDT into the **IDTR** (Interrupt Descriptor Table Register), which is the CPU's dedicated register for "where the interrupt table lives".

This step happens before the kernel enables ordinary hardware interrupts. That split matters. After `lidt`, the CPU already has a valid destination for exceptions and debugger traps. Only later, after the PIC is remapped and the early device handlers are registered, does the kernel execute `sti` (Set Interrupt Flag) to re-enable maskable hardware interrupts. In other words, the exception path comes alive first; asynchronous device delivery comes alive second.

Why the GNU Debugger Needs the IDT Early

This early `lidt` is not just for normal faults. It also matters for the **GNU Debugger** (**GDB**), which often resumes the target and relies on a trap or temporary breakpoint to stop again a moment later. Those stops still enter the kernel through the CPU's ordinary exception mechanism.

If the kernel has not loaded its own IDT yet, a trap taken while stepping through early boot has nowhere valid to go. Instead of returning neatly to the debugger, the CPU can fall into firmware state or low memory. Loading the IDT early gives those traps a valid landing point even though the machine is still not accepting ordinary device IRQs yet.

Why the IDT Points at Assembly Stubs

An IDT entry stores a raw machine address. The CPU jumps there directly when an interrupt arrives. A normal C function is not prepared for that.

On interrupt entry the CPU pushes an **interrupt return frame** onto the current stack: the saved instruction pointer `EIP`, code segment `CS`, and flags register `EFLAGS`. If the interrupt crosses a privilege boundary from ring 3 to ring 0, the CPU also pushes the user stack pointer `ESP` and stack segment `SS`. Some exceptions push an error code as well.

None of that matches the C calling convention. A C function expects to start with a normal return address at the top of the stack and with any arguments laid out according to the compiler's ABI rules. So the IDT does not point straight at C functions. It points at tiny **ISR** (Interrupt Service Routine) assembly stubs.

Those stubs do the minimum amount of machine-specific cleanup needed to turn "raw CPU interrupt entry" into "something shared C code can understand". For exceptions that do not receive a CPU-pushed error code, the stub first pushes a dummy zero so the final stack shape

stays uniform. Then it pushes the vector number and jumps to a shared trampoline. Exceptions that already have a CPU-pushed error code skip the dummy push and only add the vector number.

Hardware IRQ stubs follow the same pattern. Each one pushes a dummy zero for the error-code slot, pushes its remapped vector number, and jumps to the common IRQ trampoline.

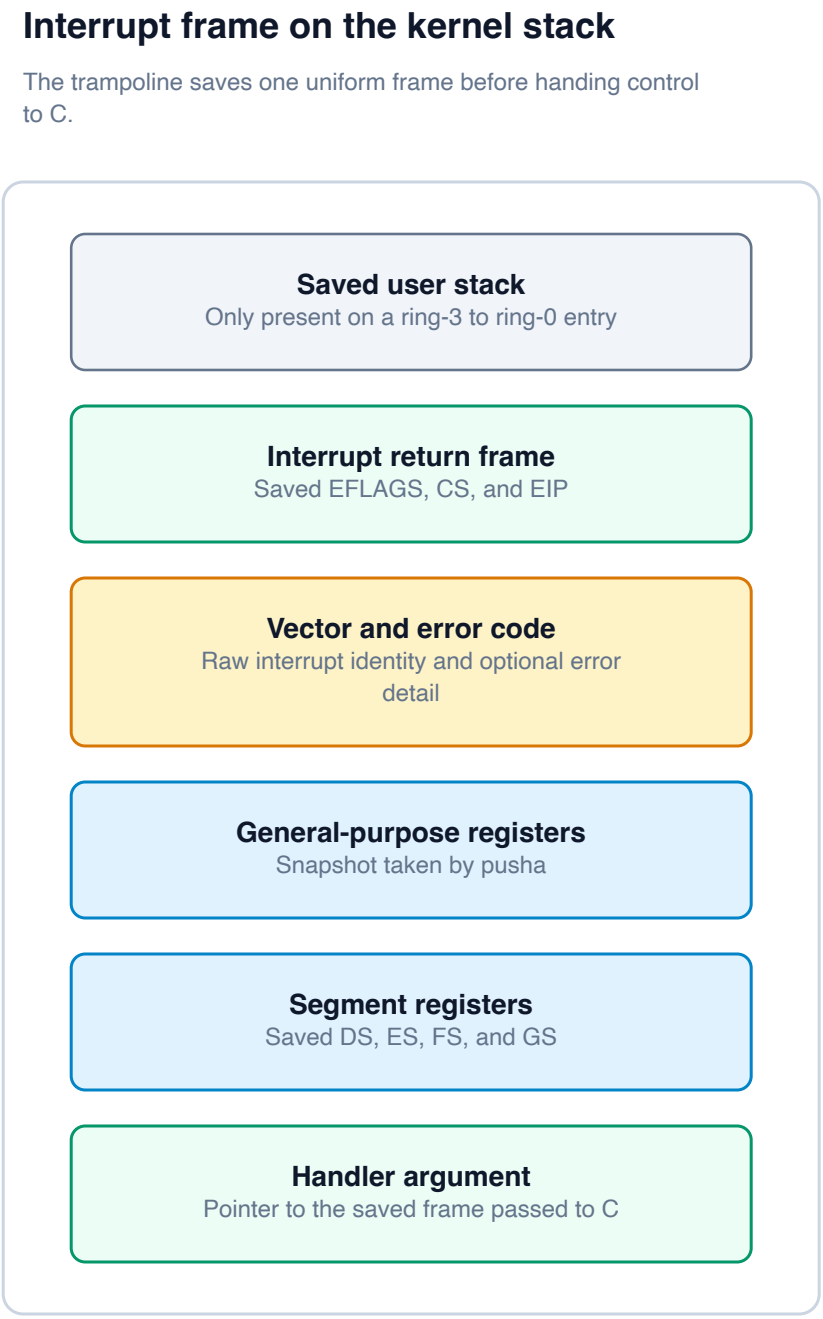
The Common Trampoline

The shared trampolines — `isr_common` for exceptions and `irq_common` for hardware IRQs — save the rest of the machine state before calling into C.

For exception entry, the sequence is:

1. `pusha` saves the eight general-purpose registers.
2. `DS`, `ES`, `FS`, and `GS` are pushed so the current data-segment state is preserved.
3. The kernel data selector `0x10` is loaded into all four segment registers.
4. The current stack pointer is pushed as a pointer to the saved frame.
5. `isr_handler` runs in C using that saved-frame pointer.
6. A signal-delivery check runs before returning to user mode.
7. The segment registers and general-purpose registers are restored in reverse order.
8. The trampoline discards the vector number and error code slots and finishes with `iret` (Interrupt Return), which restores the CPU's saved execution state.

By the time the C exception handler receives control, the stack has been normalised into this layout:



Chapter 4, Figure 1

The IRQ trampoline saves the same register state, but its C handoff is slightly different. Instead of passing a frame pointer to `irq_dispatch`, it passes the vector number and error-code slot as ordinary arguments, then optionally runs the scheduler and signal-delivery checks before restoring the saved state and executing `iret`.

The Saved Frame in C

The exception path treats the saved stack image as an `isr_frame_t` — a struct whose field order matches exactly what the assembly pushed:

```
typedef struct __attribute__((packed)) {
    uint32_t gs, fs, es, ds;                              /* segment pushes */
    uint32_t edi, esi, ebp, esp_saved, ebx, edx, ecx, eax; /* pusha */
    uint32_t vector, error_code;                          /* stub pushes */
    uint32_t eip, cs, eflags;                             /* CPU iret frame */
    uint32_t user_esp, user_ss;                           /* ring-3→0 only */
} isr_frame_t;
```

Having the entire machine snapshot in one typed struct makes the handler far easier to reason about. The kernel can inspect `frame->vector` to decide which exception fired, print `frame->eip` to show where execution stopped, and read `frame->error_code` when the architecture provides one.

Some exceptions need extra CPU state outside the frame itself. A page fault, for example, also reports the faulting linear address through control register `CR2`, so the handler reads that register as part of its diagnostic path.

The frame is not only readable. It is also writable. Later kernel paths use the saved registers as a place to communicate return values back to interrupted code, and the scheduler can resume a process by restoring a different saved frame and letting `iret` return into that process's state instead.

Dispatching Hardware IRQs

Hardware IRQ delivery uses the same low-level save-and-restore machinery, but the higher-level C path is deliberately small. `irq_dispatch` subtracts 32 from the remapped vector number to recover a zero-based IRQ line number, looks up that line in a 16-slot table of function pointers, and calls the registered handler if one exists.

After the device-specific handler returns, `irq_dispatch` sends an **EOI** (End Of Interrupt) command back to the PIC. Without that EOI, the PIC assumes the interrupt is still in service and will not forward more interrupts of the same or lower priority.

For IRQs 0 through 7, sending EOI means writing `0x20` to the master PIC command port at `0x20`. For IRQs 8 through 15, the kernel must send `0x20` to the slave PIC first and then to the master PIC. Centralising that logic in `irq_dispatch` keeps individual device drivers from having to know anything about PIC bookkeeping.

Where the Machine Is by the End of Chapter 4

By the end of this chapter, the kernel has crossed an important line: execution is no longer purely synchronous.

- CPU exceptions have a valid entry path through the IDT, with assembly stubs that turn raw interrupt entry into a uniform C-visible frame.
- Hardware IRQs no longer collide with exception vectors because the PIC has been remapped from vectors 8–15 to vectors 32–47.

- The timer and keyboard IRQ lines are the only unmasked device inputs, so the kernel is interruptible but still tightly controlled.
- Both exception handling and hardware IRQ dispatch now end by restoring the saved machine state with `iret`, so interrupted code resumes exactly where it left off.

From this point on, the CPU can enter the kernel between ordinary instructions because something happened elsewhere in the machine. The next step is to put a cleaner software dispatch layer on top of those raw IRQ vectors so device drivers can register handlers without caring about the remapped vector numbers.

Chapter 5 — The IRQ Dispatch Registry

One Handler Can't Scale

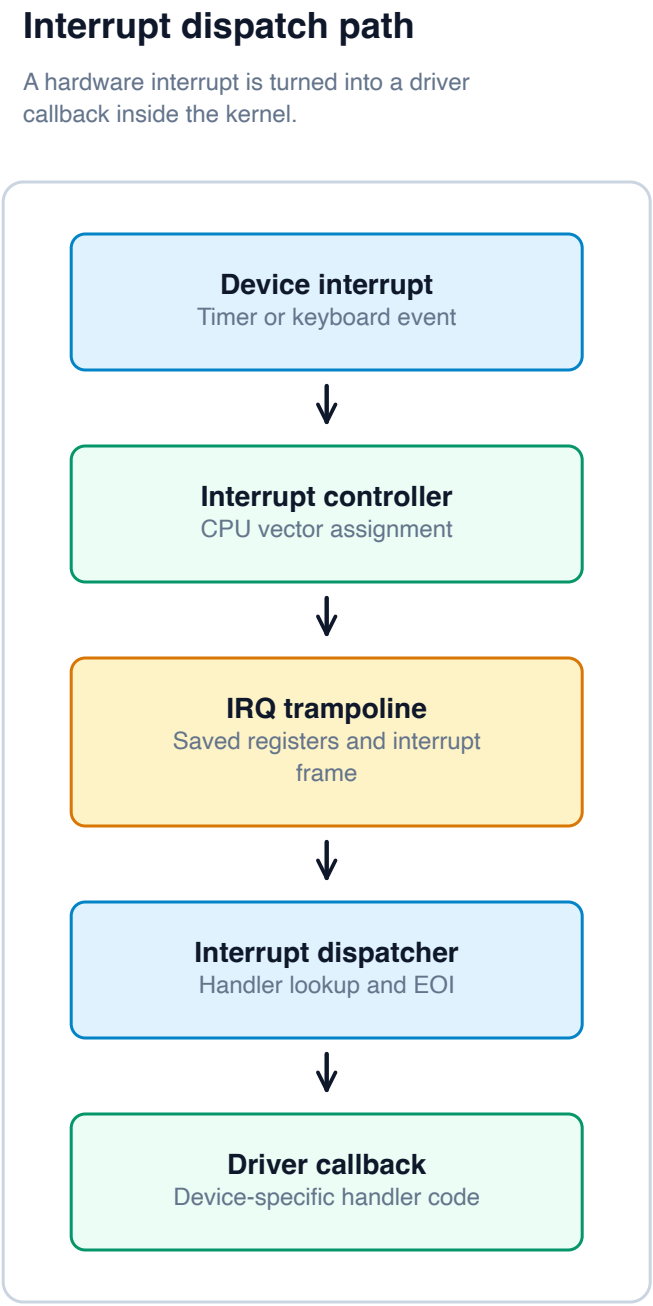
Chapter 4 ended with the IDT live and the CPU able to handle interrupts and exceptions. Every hardware IRQ funnelled into a single C function inside the keyboard driver. That function contained a chain of `if/else if` checks: if the vector number is 32 call the scheduler tick; if it is 33 call the keyboard handler. Adding a third device meant extending the chain by hand. The keyboard code had become an unrelated dependency for every future driver.

A more important problem is that the IRQ numbers — 32 for the PIT timer and 33 for the keyboard — were baked into the logic of a file nominally about the keyboard. Any developer adding a serial port driver would need to know the exact vector numbers used by two completely different subsystems before they could touch anything.

The IRQ dispatch registry solves both problems at once. Instead of one global handler, we keep a table of 16 function pointers, indexed by IRQ line number (zero-based, so IRQ 0 is the PIT timer and IRQ 1 is the PS/2 keyboard). Each driver claims its own IRQ line during startup. The common IRQ path converts the raw vector number to an IRQ line index, looks up the registered function pointer, and calls it.

The Full IRQ Path

The path from a hardware event to a registered C handler now looks like this:



Chapter 5, Figure 1

The Registry Structure

The dispatch table is a static array of 16 `irq_handler_fn` function pointers, all initialised to NULL during early startup. The type is:

```
typedef void (*irq_handler_fn)(void);
```

Each entry corresponds to one of the 16 IRQ lines the 8259A **PIC** (Programmable Interrupt Controller) can deliver. IRQ lines 0–7 are handled by the master PIC and arrive at CPU vectors 32–39. Lines 8–15 are handled by the slave PIC and arrive at vectors 40–47. The table uses the zero-based line index because that is stable across different PIC remapping configurations.

Registering a Handler

A driver registers its handler during init, before interrupts are enabled. The registration stores one function pointer in the slot for that IRQ line. If a second driver claims the same line, the new handler replaces the old one — there is no chain.

Slot	Handler	Purpose
`0`	Timer handler	Handles timer ticks
`1`	Keyboard handler	Handles keyboard input
`2..14`	None	No handler registered yet
`15`	None	No handler registered yet

The Dispatch Function

The common IRQ dispatch step receives the raw IDT vector number (32–47) and a dummy error code (always zero for hardware IRQs, pushed by the stub). It then:

1. Computes `irq_num = vector - 32`.
2. If `irq_num < 16` and `irq_table[irq_num]` is non-NULL, calls the registered function.
3. Sends an **EOI** (End of Interrupt) to the PIC. For IRQ lines 8–15 the EOI goes to both the slave PIC (port `0xA0`) and the master PIC (port `0x20`). For lines 0–7 only the master receives it.

The EOI step is centralised here so individual handlers do not need to perform it. A driver that forgets to send EOI would leave the PIC stuck and prevent all future interrupts of the same priority. By placing it in the dispatch layer, we guarantee EOI always goes out, regardless of what the handler does.

The PIT Timer Handler

The **PIT** (Programmable Interval Timer, the Intel 8253/8254 chip) generates the periodic IRQ 0 that drives the scheduler's timeslice counter. Before this chapter, the scheduler tick was called directly from the keyboard driver's IRQ handler, an odd placement. Now the timer driver owns IRQ 0. During kernel startup, before interrupts are enabled, it claims that line with a handler that does two things in sequence: advances the wall clock by one tick, then nudges the scheduler so it can decide whether to preempt the running process.

The PIT's firing frequency is fixed during interrupt bring-up. The PIT's internal oscillator runs at 1,193,182 Hz. Writing a divisor of 11 932 to the channel-0 reload register — command byte `0x36` to port `0x43`, then the low and high bytes of 11 932 to port `0x40` — configures the chip to fire in mode 2 (rate generator), producing approximately 100 interrupts per second. One IRQ 0 therefore arrives roughly every 10 milliseconds, which is the scheduler's timeslice granularity.

The Wall Clock

Every real Unix system needs a notion of what time it is right now — a value that survives power cycles and can be read cheaply at any point during execution. On x86 the answer is the **CMOS RTC** (Complementary Metal-Oxide Semiconductor Real-Time Clock), a battery-backed chip that keeps ticking even when the machine is off. We read it exactly once at boot and then maintain a software counter from that seed.

Reading the CMOS RTC

The RTC is exposed through two I/O ports: address port `0x70` and data port `0x71`. Writing a register number to `0x70` selects the register; reading `0x71` returns its value. The relevant registers are:

Register	Address	Contents
Seconds	`0x00`	current second (0–59)
Minutes	`0x02`	current minute (0–59)
Hours	`0x04`	current hour (0–23 or 12-hour + PM bit)
Day of month	`0x07`	day (1–31)
Month	`0x08`	month (1–12)
Year	`0x09`	year modulo 100
Status A	`0x0A`	bit 7 = update-in-progress flag
Status B	`0x0B`	bit 2 = binary mode; bit 1 = 24-hour mode

The RTC updates its registers approximately once per second, and during the update window any individual register read may return a partially incremented value. We avoid this by reading the entire set twice and comparing. If both reads agree, the values are stable. If they disagree, or if the status A update-in-progress flag is set at either sample, the loop retries. After up to 100 000 tries the read is abandoned and the clock is left at zero, which means the system has no wall time — a degraded but safe state.

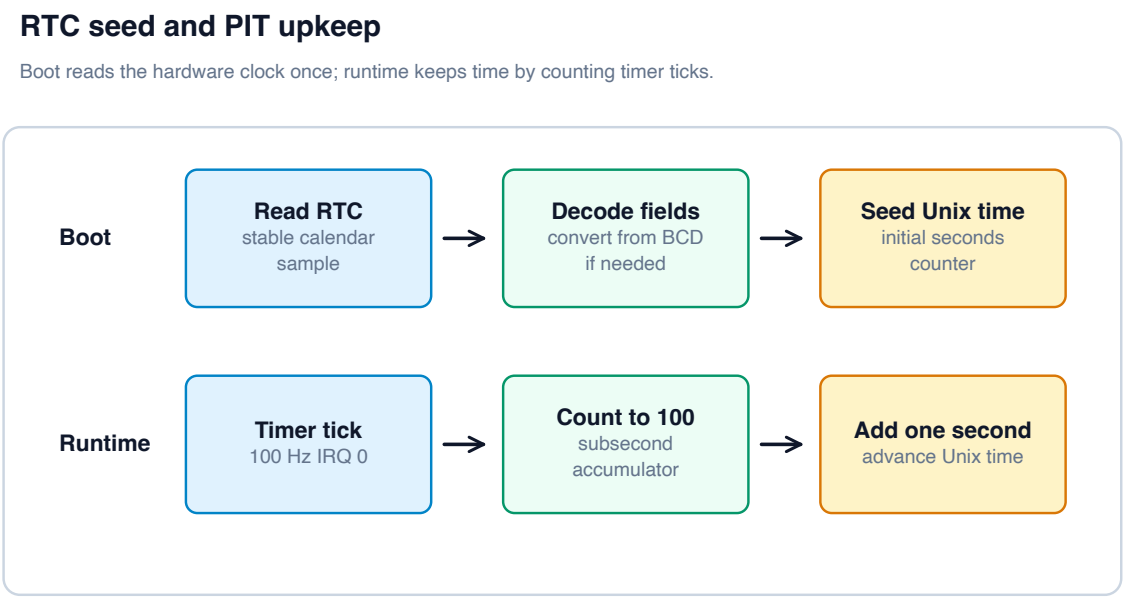
After a stable sample is in hand, the kernel converts the raw register values to binary. By default the CMOS stores values in **BCD** (Binary-Coded Decimal), where each decimal digit occupies one nibble of the byte — for example, the decimal value 42 is stored as `0x42`. Unless Status B bit 2 is set (indicating binary mode), each field is decoded from BCD into an ordinary integer. The 12/24-hour format is also normalised: if Status B bit 1 is clear, the RTC is in 12-hour mode and the PM indicator is encoded in the top bit of the hours byte.

Once all six fields are in calendar form, they are converted to a **Unix timestamp** — the number of seconds elapsed since 1970-01-01 00:00:00 UTC. The conversion counts the number of days from 1970 to the current year (accounting for leap years with the Gregorian rule: divisible by 4 but not 100, or divisible by 400), adds the days elapsed in the current year up to the current month and day, and then multiplies total days by 86 400 before adding the hours, minutes, and seconds. The result is stored in a static `g_unix_time` variable.

Advancing the Clock on PIT Ticks

Reading the RTC every second would be possible but wasteful. Instead, the PIT fires at 100 Hz and the timer path increments a sub-second tick counter on every interrupt. When that counter reaches 100, it resets to zero and increments the Unix-time counter by one. This way the wall clock advances at exactly one second per 100 PIT ticks — the same rate as the scheduler.

We expose the current Unix-time counter to user space through `SYS_CLOCK_GETTIME`. Because the CMOS is only read once, the clock drifts over long runtimes; for a system with no **NTP** (Network Time Protocol — the internet standard for synchronising clocks across machines) and a battery-backed RTC that is set accurately at boot, the drift is small enough to be irrelevant.



Chapter 5, Figure 2

The New Boot Sequence Excerpt

After this chapter, the interrupt-related portion of kernel startup proceeds in this order:

1. The IRQ dispatch table is cleared so no slot contains stale garbage that could be accidentally invoked.
2. The timer driver registers its handler for IRQ 0.
3. The ATA disk driver probes the drive.
4. The keyboard driver registers its handler for IRQ 1.
5. The IDT is filled, the PIC is remapped, the timer is programmed to 100 Hz, and interrupts are finally enabled.

Every registration completes before interrupts are turned on, which ensures the table is fully consistent before any IRQ can arrive.

Where the Machine Is by the End of Chapter 5

We now have a proper interrupt dispatch mechanism. Any driver can register for any IRQ line without knowing what other drivers have registered. The PIT and keyboard handlers are installed through the same registry as any future driver, and the centralised EOI means no driver can accidentally leave the PIC in a locked state by forgetting the End of Interrupt write.

The system also has a wall clock seeded from the CMOS RTC at boot and maintained by the PIT interrupt at 100 Hz. User programs can read the current Unix time through `SYS_CLOCK_GETTIME` without ever touching a hardware port directly.

Chapter 6 — SSE2 and CPU Extensions

An Unconfigured CPU Cannot Run Modern Code

Chapter 5 left the interrupt dispatch registry in place and the wall clock ticking. Before we create any processes, there is a piece of hardware setup that must happen first: enabling the floating-point and vector extensions the CPU keeps locked off by default.

When the CPU comes out of reset it supports an old, conservative subset of the x86 instruction set. Any modern extension — the floating-point unit, the vector instructions, the 128-bit and 256-bit registers — is turned off and must be explicitly enabled by the operating system. The CPU does this to protect the OS from legacy software that does not know how to save and restore the extra registers across context switches.

The particular extension this chapter enables is **SSE2** (Streaming SIMD Extensions 2), a family of instructions introduced by Intel in 2000. **SIMD** (Single Instruction, Multiple Data) is the class of instruction that operates on several values simultaneously — the canonical example being "add four 32-bit floats at the same time". SSE2 provides this capability through eight 128-bit registers named `XMM0` through `XMM7`, each wide enough to hold four 32-bit floats, two 64-bit doubles, or various combinations of integer types. Modern compilers can use SSE2 aggressively even for code that looks like ordinary C, but we deliberately disable compiler-generated SSE for kernel C so that only explicit low-level FPU/SSE ownership code touches those registers.

If we do not enable SSE2 before the first explicit `fxsave` or `fxrstor` executes, the CPU raises an **Invalid Opcode exception** (`#UD`, short for "undefined instruction"). Our handler from Chapter 4 would catch the fault, print an error, and halt. This is exactly what happens if you try to manage per-process FPU state without the setup described here.

We do this work before any process is created because the hardware must be configured before a user program runs. A program that encounters floating-point operations before `OSFXSR` is set will fault on its first such instruction — there is no safe way to fix that mid-flight.

The Four Control Bits That Gate SSE

Four bits across two **control registers** (`CR0` and `CR4`) determine whether SSE instructions are legal:

Register	Bit	Name	After reset	Meaning when set
`CR0`	1	MP (Monitor Coprocessor)	0	Tells the CPU to check the task-switched flag before FPU use
`CR0`	2	EM (Emulate)	1	Every FPU or SSE instruction raises `#UD` — as if there were no FPU at all
`CR4`	9	OSFXSR	0	The OS promises to use `FXSAVE` and `FXRSTOR` to save SSE state on context switches

Register	Bit	Name	After reset	Meaning when set
`CR4`	10	OSXMMEXCPT	0	Routes SSE floating-point exceptions to a specific vector instead of `#UD`

When GRUB hands control over, `EM` is set and `OSFXSR` is clear. Until both change, every SSE instruction is illegal. We therefore have to:

1. Clear `EM` to stop emulating the FPU.
2. Set `MP` so the CPU properly tracks the task-switched flag.
3. Set `OSFXSR` to tell the CPU we know how to save and restore SSE state.
4. Set `OSXMMEXCPT` so that SSE exceptions become their own vector rather than being reported as invalid opcodes.

All four changes are made in a tiny assembly startup routine, because reading and writing control registers cannot be expressed in C. This is the very first floating-point-related work the kernel does, so it can capture a clean FPU template before any process is launched.

Capturing a Clean FPU Snapshot

Once the control bits are in place, we initialise the floating-point unit with the `fninit` instruction (which resets the x87 FPU — the legacy floating-point unit present since the 80387 — to a well-defined state: all exceptions masked, double-extended precision, stack pointer zeroed) and then use `fxsave` (FP state eXtended save) to dump the complete FPU and SSE register state into a 512-byte buffer. That buffer becomes the **template** every new process is initialised from.

The template contains these well-known values:

- `FCW = 0x037F` — the FPU control word, with all x87 floating-point exceptions masked.
- `FSW = 0` — the status word, with no exceptions pending.
- `MXCSR = 0x1F80` — the SSE control register, with all SSE exceptions masked and rounding set to nearest.
- All eight `XMM` registers zeroed.

Any process created from this template starts life with a clean, predictable floating-point environment.

Giving Every Process Its Own FPU State

A process's in-memory descriptor needs a place to store its own 512 bytes of SSE state, because different processes will have different floating-point values in their `XMM` registers at any given moment. The `process_t` struct is extended with a 512-byte `fpu_state` field:

```
typedef struct __attribute__((aligned(16))) {
    uint32_t pd_phys;
    uint32_t entry;
    uint32_t user_stack;
    uint32_t kstack_top;
    uint8_t  fpu_state[512];   /* FXSAVE image */
} process_t;
```

The `FXSAVE` instruction requires the destination address to be **16-byte aligned**, or the CPU raises a general protection fault. The struct is marked as aligned to sixteen bytes, and the four preceding `uint32_t` fields total sixteen bytes, so `fpu_state` naturally falls on a 16-byte boundary within the struct.

When a new process is created, we copy the clean template into that process's `fpu_state` field with the kernel's integer-only memcpy routine.

Just before we hand control to the new process for the first time, we issue an `fxrstor` instruction, which reads 512 bytes from the process's `fpu_state` field and loads the FPU and SSE registers. From that point on, any floating-point computation the user program performs is backed by hardware registers that carry its own values.

Context-Switch Integration

Our scheduler wires `fxsave` and `fxrstor` into the context-switch path. When the timer tick decides to preempt the currently running process, the outgoing process's FPU state is written to its `fpu_state` field with `fxsave`, and the incoming process's state is loaded with `fxrstor` before control returns to its user code. Because all of the plumbing is already in place from this chapter, the scheduler's floating-point handoff reduces to "save the old image, restore the new image" at each switch.

Keeping Kernel C Out of the FPU

Enabling SSE at the hardware level does not mean ordinary kernel C should use it. **GCC** (GNU C Compiler, the compiler this project uses) is told not to emit SSE, MMX, or x87 floating-point instructions for kernel C via `-mno-sse -mno-sse2 -mno-mmx -msoft-float`. That keeps the kernel from silently clobbering the `XMM` registers that belong to the currently running user process between explicit `fxsave` and `fxrstor` points.

This policy is stricter than simply enabling SSE2 globally. It prevents bugs where a harmless-looking struct copy in an interrupt path becomes compiler-generated `movdqu`/`movups` instructions and mutates process FPU state before the scheduler has saved it. Until we have a separate FPU ownership protocol for our own vector work, SSE usage stays explicit and local to the low-level save/restore code.

Where the Machine Is by the End of Chapter 6

By the end of this chapter:

- The CPU is willing to execute explicit SSE2 save/restore instructions without faulting.
- A 512-byte template of a clean FPU state sits in kernel memory, ready to be copied into any new process.
- The `process_t` struct carries a private FPU state buffer at a 16-byte-aligned offset, and the scheduler will save and restore it on every context switch.
- Kernel C is kept off SSE/MMX/x87 so process FPU state is changed only at explicit save/restore boundaries.

The kernel is now ready to preserve user FPU/SSE state without letting unrelated kernel C code touch it accidentally. Chapter 7 covers how the kernel discovers what memory is available before

it can manage that memory.

Part II — Managing Memory

At the end of Part I the kernel is running, but it has no idea what memory it has. The GRUB bootloader placed the kernel binary somewhere above the 1 MB mark and provided a memory map, but the kernel has not read that map yet. Every address the kernel uses — the kernel stack, the screen output buffer, the temporary variables in C — sits in memory that has never been formally claimed. If the kernel were to allocate a buffer for a new data structure, it would have to pick an address by hand and hope nothing else was already there. That is not a foundation; it is a time bomb.

Memory management is not one problem but three nested problems that must be solved in order. First, the kernel needs to know which physical addresses contain usable RAM and which are reserved for hardware, firmware tables, or memory-mapped devices — the BIOS (Basic Input/Output System, the firmware that ran before GRUB) provides an E820 memory map that lists these regions, but it must be read and interpreted before anything overwrites it. Second, the kernel needs a way to allocate and free individual 4 KB pages of physical memory so that it can hand them out to data structures, page tables, and eventually user processes — a bitmap spanning all of usable RAM serves this role compactly. Third, the kernel needs a virtual address space: x86 paging is a hardware mechanism that translates the addresses C code generates into the physical addresses the memory bus sees, and setting it up correctly is what gives each future process its own isolated view of memory.

Chapter 7 reads the Multiboot memory map and builds the initial picture of physical RAM: which regions are usable, where the kernel image sits, and what must be avoided. Chapter 8 builds the entire memory stack on top of that picture — the physical memory manager's bitmap, the two-level x86 page directory and page table structure, the identity map of the first 128 MB that lets kernel virtual addresses equal physical addresses, and the `kmalloc`/`kfree` heap allocator that the rest of the kernel uses for dynamic allocation.

By the end of Part II the kernel has complete and correct control over RAM. It knows what physical pages exist, can allocate and free them, and has a virtual address space in place. Every subsystem built from this point forward can ask for memory and receive it safely.

Chapter 7 — Detecting Memory

Why We Cannot Assume Anything About RAM

Chapter 6 left us with SSE enabled and a clean FPU template in hand. Before we can build an allocator, we face a more fundamental problem: the kernel needs to know two things about physical memory before it can hand any of it out — how much there is, and which regions are safe to use. Neither answer can be guessed. The physical address space of a PC is not a clean contiguous block of RAM — it is a patchwork of usable memory, hardware-mapped regions (the VGA buffer, firmware ROM), and legacy holes left over from decades of backward compatibility. Writing to the wrong address can corrupt firmware data, silently fail, or, worst of all, appear to work and then break something unrelated minutes later.

The authoritative source of this information on x86 is a BIOS interface called **E820**, named after the `INT 0x15` function number (`AX = 0xE820`) that retrieves it. E820 returns a list of memory regions, each tagged with a base address, a length, and a type code that identifies whether the region is usable RAM, reserved for hardware, or some other special category.

E820 can only be called from **real mode** — the 16-bit compatibility mode the CPU starts in. Once the CPU has been switched to protected mode (which GRUB does before it hands control to the kernel), the BIOS is gone and E820 can no longer be called. This creates a timing problem: we need E820 data, but we run in protected mode and cannot ask for it.

How GRUB Solves the Timing Problem

GRUB handles this by calling E820 itself, while it is still in real mode, and embedding the result in the **Multiboot info structure** it passes to the kernel. When GRUB jumps to the kernel's entry point, the `EBX` register holds a pointer to this structure, and one of its fields is a pointer to the memory map that E820 produced.

We receive the Multiboot info pointer as the second argument to `start_kernel` (see Chapter 1) and pass it to the physical memory manager's initialisation function, `pmm_init`. The memory map inside the structure is the foundation the physical memory manager builds its free-page bitmap on top of.

The Shape of a Multiboot Memory Map

The Multiboot info structure contains two relevant fields for memory detection: `mmap_length` (the byte length of the memory map) and `mmap_addr` (the physical address of the first entry). A flag bit in the `flags` field indicates whether the memory map is actually present — on the rare BIOS that does not support E820, GRUB falls back to a simpler interface and does not produce an mmap.

Each entry in the memory map is a variable-length C struct:

```
typedef struct {
    uint32_t size;   /* size of this entry, NOT counting this field */
    uint64_t addr;   /* base physical address */
    uint64_t len;    /* length in bytes */
    uint32_t type;   /* 1 = usable RAM, 2 = reserved, ... */
} __attribute__((packed)) multiboot_mmap_entry_t;
```

The `size` field is unusual: it gives the length of the rest of the entry without counting itself. Iterating the map therefore means advancing by `size + 4` bytes each time, not just `size`. This quirk is inherited from the **ACPI** memory-map format that E820 ultimately derives from. ACPI — the **Advanced Configuration and Power Interface** — is an industry standard that defines how platform firmware describes the machine to the operating system and how the OS controls power management, device configuration, and thermal state. The memory-map format used by E820 is one of those firmware-provided tables; GRUB reads the map through the `INT 0x15` interface and copies the entries into the Multiboot info structure unchanged, so we end up seeing ACPI-defined data even though we never talk to ACPI directly.

The `type` field uses the same codes ACPI defines for E820 entries:

Type	Meaning
1	Usable RAM
2	Reserved — do not use
3	ACPI reclaimable
4	ACPI non-volatile storage
5	Bad memory

For our purposes, only type 1 is treated as free memory. Everything else is left marked as used, which is the safe choice: even ACPI reclaimable memory holds valid data until we have finished reading the ACPI tables.

How the Physical Memory Manager Uses the Map

The physical memory manager starts with a conservative default: every page in the bitmap is marked as used. This guarantees that if the memory map is missing or corrupted, we will not accidentally hand out a page that is actually hardware or firmware.

From that safe state, the physical memory manager walks the memory map and, for every entry with type 1, clears the bits for the pages covering that region, making them available for allocation. After all the type-1 entries have been processed, it re-marks several specific regions as reserved again: the kernel image itself (so the allocator does not hand out pages holding executable code), the PMM bitmap (which lives in the kernel's BSS), the VGA/ROM hole, and any other region the kernel uses internally.

The walk advances from one entry to the next by adding `entry->size + 4` to the entry pointer. The `+ 4` is the part that catches everyone the first time, so it is worth spelling out exactly what `size` is counting and what it is not.

The `size` field does not describe the whole entry. It describes everything that comes *after* itself — the `addr`, `len`, and `type` fields, plus any optional trailer the firmware appends. The four bytes that hold `size` are deliberately excluded from the value `size` reports. So if an entry's `size` field reads 20, that entry actually occupies 24 bytes of memory in total: the four-byte `size` header, followed by the 20 bytes of payload that the header is announcing.

Picture one entry laid out in memory:

Offset	Bytes	Meaning
`+0`	`4`	`size` header
`+4`	`size` bytes	Address, length, type, and any extension fields
`+4 + size`	next entry	First byte of the following record

Walking the map means stepping over the entire entry, header included. From the start of the current entry, the next entry begins `4 + size` bytes later: four bytes to skip past the `size` header itself, plus `size` more bytes to skip past everything the header is counting. Adding only `entry->size` would land four bytes short — inside the current entry's payload — and the loop would happily reinterpret a fragment of the current `addr` field as the next entry's `size`, producing nonsense and almost certainly hanging the kernel on the very next iteration.

Why design the format this way at all? Because it makes the entries self-describing in a way that lets the format grow without breaking older parsers. The memory-map format is deliberately extensible, and some firmware returns entries longer than the 24 bytes the C struct declares, appending ACPI-defined fields such as the 3.0 "extended attributes" word. By placing a length-of-the-rest field at the very front of every entry, ACPI guarantees that any conforming OS — even one written before those extensions existed — can skip cleanly to the next entry by trusting whatever value `size` carries at runtime, instead of hard-coding the layout of the fields it knows about.

This is an easy place to introduce a bug. Writing `sizeof(entry)` looks reasonable but gives four — the size of a pointer on a 32-bit machine — so the walk crawls forward in pointer-sized steps and interprets fragments of the previous entry as the next one. The slightly better-looking `sizeof(*entry)` gives 24, which happens to match QEMU's output exactly and will appear to work on an emulator, but breaks on any real BIOS that returns larger entries. Only `entry->size + 4` is correct in both cases.

If the Multiboot flag for "mmap is present" is clear — meaning GRUB could not produce a memory map — `pmm_init` falls back to assuming that the region from 1 MB to 128 MB is usable RAM. This is the same assumption QEMU's default memory configuration satisfies, so the fallback is enough to boot on any emulator.

A Realistic Memory Map

On a QEMU machine configured with 128 MB of RAM, the memory map typically looks like this:

Base	Length	Type	What it is
`0x00000000`	`0x0009FC00`	1 (usable)	Conventional RAM below the 640 KB line
`0x0009FC00`	`0x00000400`	2 (reserved)	Extended BIOS Data Area
`0x000F0000`	`0x00010000`	2 (reserved)	Legacy ROM (Read-Only Memory) region
`0x00100000`	`0x07EF0000`	1 (usable)	Extended RAM from 1 MB to 128 MB

Base	Length	Type	What it is
`0xFFFC0000`	`0x00040000`	2 (reserved)	BIOS ROM shadow at the top of the 4 GB space

The gap between `0x9FC00` and `0x100000` — the roughly 384 KB region containing the BIOS data area, the VGA text buffer, and the ROM shadow — is a legacy hole in the PC address map that cannot be used as general RAM no matter how much physical memory the machine has. We rely on the memory map to know where these holes are, rather than hard-coding assumptions about the PC layout.

Where the Machine Is by the End of Chapter 7

After `pmm_init` returns, we have a reliable, hardware-provided map of which pages of physical memory are usable. That map is the foundation the rest of the memory management stack in Chapter 8 is built on: the page allocator, the paging system, and the heap all depend on knowing which physical pages are real RAM and which are reserved for something else.

Chapter 8 — Memory Management

Three Layers on Top of Raw RAM

Chapter 7 left us with a reliable map of which 4 KB chunks of physical memory are real RAM and which are holes or hardware-mapped regions. Knowing is not enough — we need to *do* something with that information. Three separate problems remain, and this chapter builds three layered solutions, each on top of the one beneath it.

The first problem is **tracking which physical pages are free and which are in use**. The solution is the **Physical Memory Manager** (PMM), which maintains a bitmap with one bit per page.

The second problem is **controlling how virtual addresses map to physical addresses**. The solution is **paging** — a hardware feature that translates every memory access through a two-level lookup table before it reaches the bus.

The third problem is **allocating arbitrary-sized chunks of memory for the kernel's own short-lived data structures**. The solution is the **kernel heap**, a simple allocator that provides `kmalloc` and `kfree` out of a reserved block of identity-mapped memory.

Each of these three subsystems is covered below. All three must be initialised in a specific order — any other order fails because each layer depends on state the previous layer established.

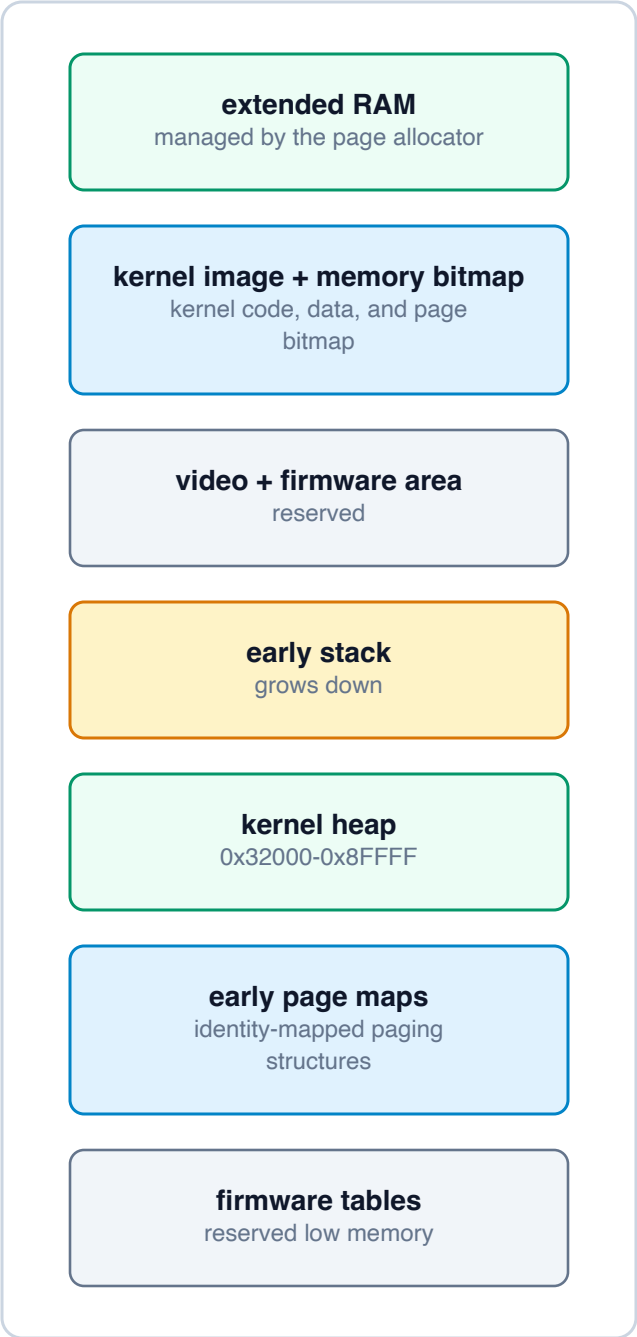
The Physical Memory Layout

Before we could write the allocators, we had to decide where each piece of early-boot data lives in physical memory. These addresses must be chosen before paging is enabled, because once paging is on, every write goes through the page tables, and the page tables themselves have to already be in place.

The physical address space the kernel inherits from the BIOS looks like a stack of regions, growing upward from address zero:

Physical memory map

Low memory is carved up before the page allocator manages extended RAM.



Chapter 8, Figure 1

The same layout in tabular form, which is easier to scan when you need an exact address:

Start	End	Size	Region	Notes
`0x00000000`	`0x000004FF`	1.25 KB	BIOS IVT + BDA	Interrupt vector table & BIOS data area — reserved
`0x00000500`	`0x00010FFF`	~66 KB	Unused low RAM	Free, but not used by the kernel
`0x00011000`	`0x00011FFF`	4 KB	Page directory	The 1 024-entry PD loaded into CR3
`0x00012000`	`0x00031FFF`	128 KB	Page tables	32 page tables, identity-mapping 0–128 MB
`0x00032000`	`0x0008FFFF`	~376 KB	Kernel heap	`kmalloc` / `kfree` range
`0x00090000`	—	—	Early stack top	Grows downward from here
`0x000A0000`	`0x000BFFFF`	128 KB	VGA / ROM hole	`0xB8000` is the VGA text buffer
`0x000C0000`	`0x000FFFFF`	256 KB	BIOS ROM shadow	Reserved
`0x00100000`	kernel end	varies	Kernel image + BSS	Code, data, and the PMM bitmap (in `.bss`)
kernel end	`0x07FFFFFF`	rest	Extended RAM pool	Handed out page by page by the PMM

Everything below 1 MB is reserved or hardware-mapped. The kernel image itself starts at 1 MB, and the PMM bitmap lives inside the kernel's **BSS** (the section of an executable holding uninitialised data — it occupies no space in the file but is allocated and zeroed when the program is loaded), so it ends up just above the kernel's code and data.

Layer 1: The Physical Memory Manager

The **PMM** tracks which physical 4 KB pages are free. It uses a **bitmap**: a flat array where each bit represents one page. A `1` means "in use", a `0` means "free".

For 128 MB of RAM with 4 KB pages, the arithmetic is tidy:

- 128 MB ÷ 4 KB = 32 768 pages.
- 32 768 bits = 4 096 bytes = exactly 4 KB of bitmap.

Visually, the bitmap is a flat array where each bit owns a single page of physical RAM. Byte 0, bit 0 describes the page at physical address `0x00000000`; byte 0, bit 1 describes the page at `0x00001000`; and so on:

Bitmap bit	Page address	Meaning
`byte0 bit0`	`0x0000`	Page already reserved
`byte0 bit1`	`0x1000`	Page can be allocated
`byte0 bit2`	`0x2000`	Page already reserved
`byte1 bit0`	`0x8000`	Page can be allocated

A `1` means "in use", a `0` means "free". Finding a free page is the same as finding the first byte that is not `0xFF`, then the first zero bit inside that byte — the physical address follows from `(byte_index × 8 + bit_index) × 4096`.

The bitmap is a static 4 096-byte array, which places it in the kernel's BSS section and therefore at `0x100000+` alongside the rest of the kernel image. A second static array now sits beside it: one byte of reference count per physical page. The bitmap still answers "is this page allocatable right now?", but the refcount answers "how many live mappings still point at it?" — important once processes can share physical frames. This is the same broad placement strategy Linux uses with `mem_map` — page-tracking metadata sits just above the kernel image, safely above the low-memory region where the bootloader leaves its own data.

Initialisation Order

The physical memory manager initializes with a conservative strategy: first mark every page as reserved, then walk the firmware memory map and open up every region the hardware reports as usable, then re-protect the regions the kernel itself occupies — the kernel image, the bitmap, the page tables, the heap, the stack, and the VGA/ROM hole.

The "start safe, then free, then re-reserve" sequence is deliberate. If the memory map is missing, everything stays marked as used and the allocator refuses to hand out anything until the fallback region (1 MB–128 MB) is explicitly freed. This prevents the kernel from ever accidentally handing out a page that is already occupied by hardware, firmware, or itself.

Allocation

Physical-page allocation therefore follows a simple search. The kernel scans the bitmap byte by byte for the first byte that is not `0xFF`, then bit by bit inside that byte for the first zero. It marks that page in use, sets the page's reference count to 1, and returns the corresponding physical address. If the entire bitmap is full, allocation returns zero to signal out-of-memory.

Releasing a page does not always make it free immediately. The kernel drops one reference to the frame and only clears the bitmap bit when the count reaches zero. Reserved pages such as the kernel image and the identity-mapped paging structures are pinned at refcount 255 during initialisation, which keeps normal teardown from ever recycling them accidentally.

Layer 2: Paging

Why Paging Exists

With just a physical memory manager, we can allocate pages, but the CPU still has unrestricted access to every physical address. Paging adds a hardware-enforced translation layer between

virtual and physical addresses. Every memory access the CPU makes — every instruction fetch, every data load, every stack push — passes through a lookup table that translates the virtual address the program used into the physical address that the RAM chips actually see.

This enables three critical capabilities:

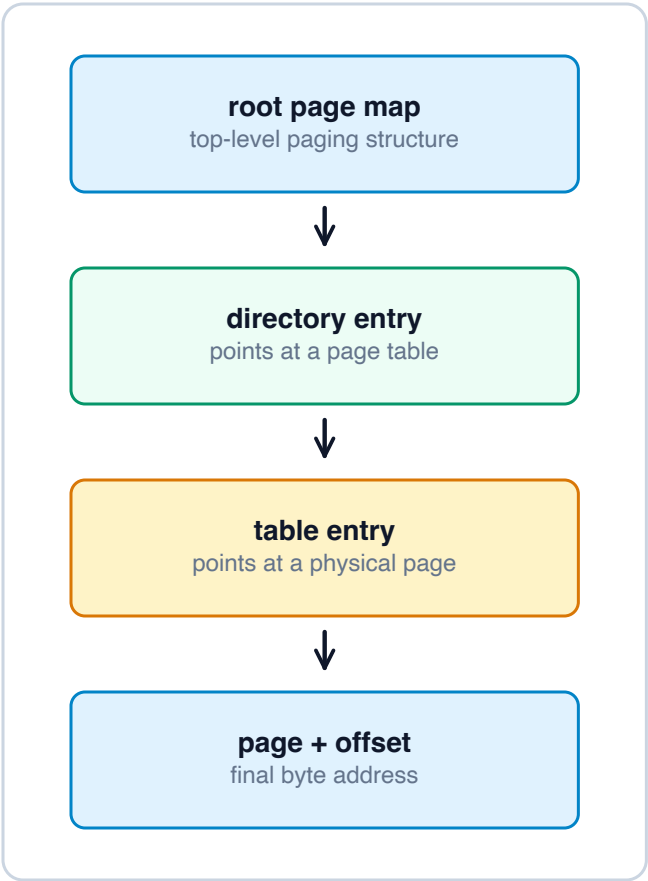
- **Memory isolation between processes.** Each process has its own lookup table (its own page directory), so process A's pointer to `0x400000` resolves to a different physical page than process B's pointer to `0x400000`. Neither process can reach the other's memory.
- **Read-only regions.** The lookup-table entries include permission bits. The kernel's code can be mapped read-only so that a stray write cannot modify it.
- **On-demand mapping.** A page can be marked "not present", and the CPU will fault when it is touched. The fault handler can allocate a physical page, update the table, and retry the instruction.

The x86 Paging Structure

On 32-bit x86, the paging lookup is a two-level table walk. A 32-bit virtual address is sliced into three fields, each of which indexes a different level of the walk:

Bits	Field	Meaning
0-11	Offset	Byte offset within the final 4 KB page
12-21	Table index	Selects one entry inside the page table
22-31	Directory index	Selects one entry inside the page directory

Following those indices through memory is the **page walk**: CR3 points at the page directory, the top ten bits pick a page directory entry (PDE), that PDE points at a page table, the middle ten bits pick a page table entry (PTE), that PTE points at a 4 KB frame, and the bottom twelve bits pick the byte inside the frame:



Chapter 8, Figure 2

The page directory has 1 024 entries (called **PDEs**, Page Directory Entries). Each PDE that is marked "present" points at a page table. Each page table has 1 024 entries (**PTEs**, Page Table Entries). Each PTE maps one 4 KB page. One page directory therefore covers the full 4 GB address space: $1\,024 \times 1\,024 \times 4\,096 = 4$ GB.

Every PDE and PTE is a 32-bit word with the same general shape — the top twenty bits are the physical frame address (because frames are 4 KB-aligned, the low twelve bits are always zero and are reused as flags), and the low twelve bits are permission and status flags:

Bits	Field	Meaning
0	P	1 = present; if 0 the CPU raises a page fault
1	R/W	1 = writable, 0 = read-only

Bits	Field	Meaning
2	U/S	1 = user accessible, 0 = supervisor only
3	PWT	Write-through
4	PCD	Cache disable
5	Accessed	Set by CPU on any access
6	Dirty	PTE only: set by CPU on write
7	PS / PAT	PDE: PS (Page Size) enables 4 MB pages. PTE: **PAT** (Page Attribute Table) index bit
8	Global	PTE only: TLB entry survives CR3 reloads
12-31	Frame address	Top 20 bits of the physical address of the next table (PDE) or final page (PTE)

The CPU is told where the currently active page directory lives by writing its physical address into **CR3**, a control register that holds the current root of the paging tree. Paging itself is enabled by setting bit 31 of **CR0**.

Identity Mapping the First 128 MB

The initial page tables set up in `paging_init` use **identity mapping**: virtual address `X` maps to physical address `X` for every address from zero through 128 MB. This is important because it means the kernel can keep running at the same addresses after paging is enabled — every pointer in every variable continues to refer to the same physical page it did before the switch.

The function fills in the page directory one entry at a time. For the first thirty-two PDEs (covering `0x00000000` through `0x08000000`, a 128 MB region), it allocates a page table, fills every one of that table's 1 024 PTEs with `virtual == physical` mappings, and points the PDE at the table. All remaining PDEs are left as zero — any access to a virtual address above 128 MB will page-fault.

After every page table is filled in, the kernel loads the page directory's physical address into `CR3` and sets bit 31 of `CR0`. On the next instruction, the CPU is translating every memory access. Because everything is identity-mapped, nothing visibly changes.

Private Page Tables for User Processes

When a user program is loaded at virtual address `0x400000` (which falls in PDE index 1), it lands inside a region that every new process initially inherits from the kernel. The page tables covering the 0–128 MB identity mapping are **shared** among all processes: every process's page directory points to the same physical page tables for PDEs 0–31.

If the kernel simply wrote a new user PTE into one of those shared page tables, every later process would overwrite the previous process's mappings. When the earlier process resumed and `CR3` was reloaded (flushing the **TLB**, the CPU's cache of recent translations), it would execute the new process's physical pages instead of its own — a guaranteed crash.

The fix is to detect when a user mapping lands in a 4 MB region still backed by a shared kernel page table. In that case, the kernel allocates a brand-new page table, copies the existing kernel entries into it so ring-0 code in this address space can still reach what it needs, installs that private table in the page directory, and only then writes the user PTE. After that, the process has its own page table for the region containing its user segments, and the shared kernel mapping is untouched.

Cloning a Page Directory

Creating a fresh address space for a new process is straightforward: allocate a blank page directory, copy the kernel PDEs, and leave the user PDEs empty. Cloning an existing address space for a fork operation is harder, because the child needs independent page tables but we want to avoid copying user frames eagerly.

The clone therefore duplicates the paging structures in two layers. For every present PDE that contains user mappings, it allocates a fresh child page table and copies the parent's entries into it. For each present user PTE inside that table, rather than allocating a new frame immediately, it marks the page as copy-on-write: the write bit is cleared, a software-defined COW flag is set, and the physical frame's reference count is incremented so the same physical memory is shared between parent and child. A write fault later forces the real per-process copy. The important structural point here is that paging is no longer only a static map built at process-creation time — the same page-table entries are part of a live fault-and-retry protocol.

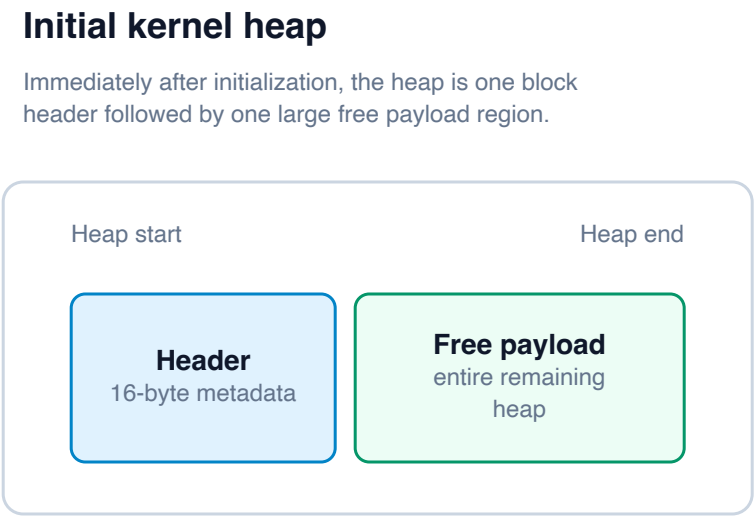
Layer 3: The Kernel Heap

Why a Heap is Needed

The PMM can hand out whole pages, which is fine for page directories, page tables, and large buffers. But many kernel data structures are smaller than a page — a keyboard ring buffer slot, an eight-byte scheduler list node, a fifty-byte path string. Allocating a full 4 KB page for each would waste almost all of the memory. We therefore have our own **heap**: a reserved block of identity-mapped memory inside which `kmalloc` hands out arbitrary byte-sized ranges.

The heap occupies the physical range `0x32000`–`0x8FFFF`, about 376 KB, inside the first megabyte of RAM. It is fully identity-mapped by paging, so we can reach any byte in it with a normal C pointer.

Inside that region, the heap is structured as a linked list of variable-sized blocks. Immediately after `kheap_init`, there is exactly one block covering the whole heap:



Chapter 8, Figure 3

The Block Header

Every allocation inside the heap is preceded by a 16-byte **header** that stores metadata immediately next to the user's data:

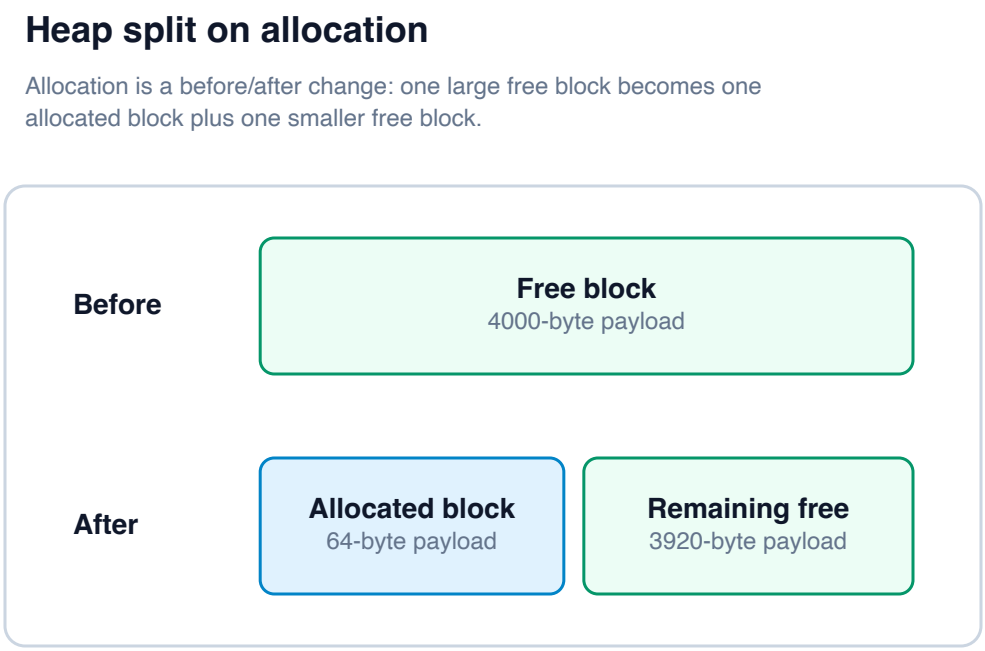
```
typedef struct heap_block {
    uint32_t            magic;   // 0xDEADBEEF — integrity canary
    uint32_t            size;    // bytes of user-accessible payload
    uint32_t            free;    // 1 = free, 0 = allocated
    struct heap_block  *next;    // pointer to next block in list
} heap_block_t;
```

The heap is managed as an **embedded free list**: every block in the heap, whether free or allocated, is a 16-byte header followed by payload bytes, and the `next` field chains every block together in order. At initialisation time the kernel installs a single free block that spans the entire heap region.

Allocation: First-Fit With Splitting

`kmalloc` uses a first-fit policy. It walks the block list from the beginning, looking for the first block that is both marked free and at least as large as the requested size. When it finds one, it checks whether the block is significantly larger than needed — larger by at least the minimum split threshold. If so, it **splits** the block: it carves a new free block out of the extra space and reduces the original block's size to exactly what was asked for. Then it marks the original block as allocated and returns a pointer to the byte just past the header.

Splitting avoids waste: if the caller wants 64 bytes and the first free block is 4 000 bytes, the allocator carves off a 64-byte allocated block and leaves a 3 920-byte free block behind rather than consuming all 4 000 bytes. Visually:

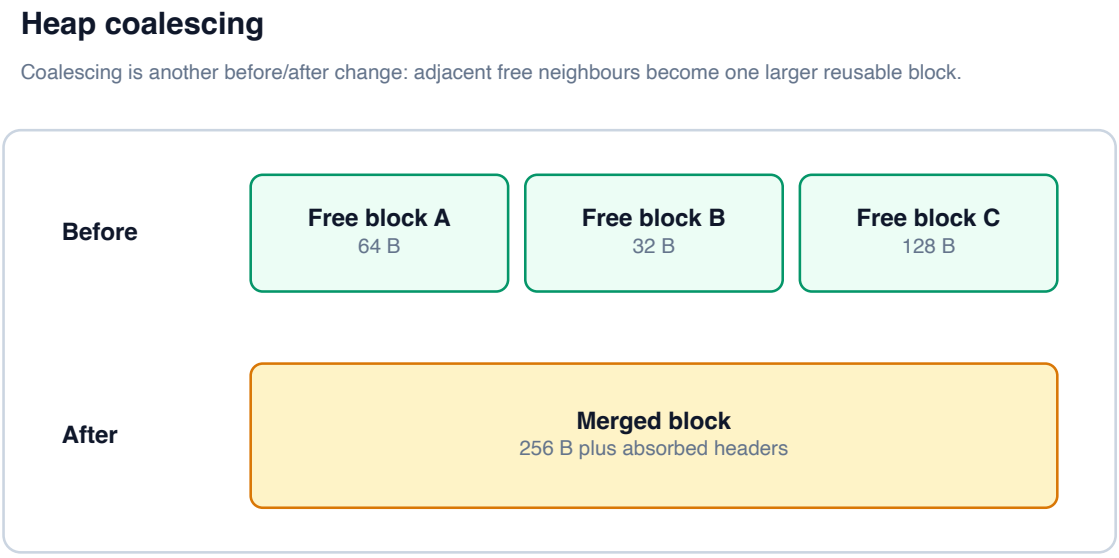


Chapter 8, Figure 4

Freeing: Mark and Coalesce

`kfree` takes a user pointer, backs up 16 bytes to find the header, verifies the magic value, and marks the block free. Then it makes a single forward pass over the list merging any pair of adjacent free blocks into one — a process called **coalescing**. Each merge combines two free blocks into a single larger free block, extending the first and unlinking the second.

Coalescing is what prevents **heap fragmentation**. Without it, repeated allocate/free cycles would leave the heap full of tiny unusable gaps even when there is plenty of total free space. With it, neighbours merge back together and future allocations can reuse the combined space. Concretely, if three neighbouring blocks are freed back-to-back, the coalesce pass fuses them in place:



Chapter 8, Figure 5

The two intermediate headers are absorbed into A's payload, and A's `next` pointer jumps straight to D.

The Magic Canary

Every block header carries the constant `0xDEADBEEF` in its `magic` field. When `kfree` is called, it checks whether the header's magic is still correct. If a buffer overrun has trampled the header of an adjacent block, or if `kfree` was called with a bogus pointer, the magic value will no longer be `0xDEADBEEF`, and the function returns without doing any damage. It does not prevent the corruption — it is a smoke detector, not a sprinkler — but it stops the allocator from operating on already-corrupted metadata and making things worse.

The Slab Allocator

`kmalloc` is a general-purpose allocator — it handles any size, but every allocation pays the cost of a header scan and coalescing logic. When the kernel needs to allocate and free the same kind of object thousands of times (process descriptors, pipe buffers, open-file entries), there is a better option: a **slab allocator**.

The idea, introduced by Jeff Bonwick at Sun Microsystems in 1994 and adopted by Linux as SLAB (and later SLUB), is to dedicate a pool of memory to a single object type. Because every slot in the pool has the same size, there is no fragmentation and no header scan — allocation is a single pointer pop.

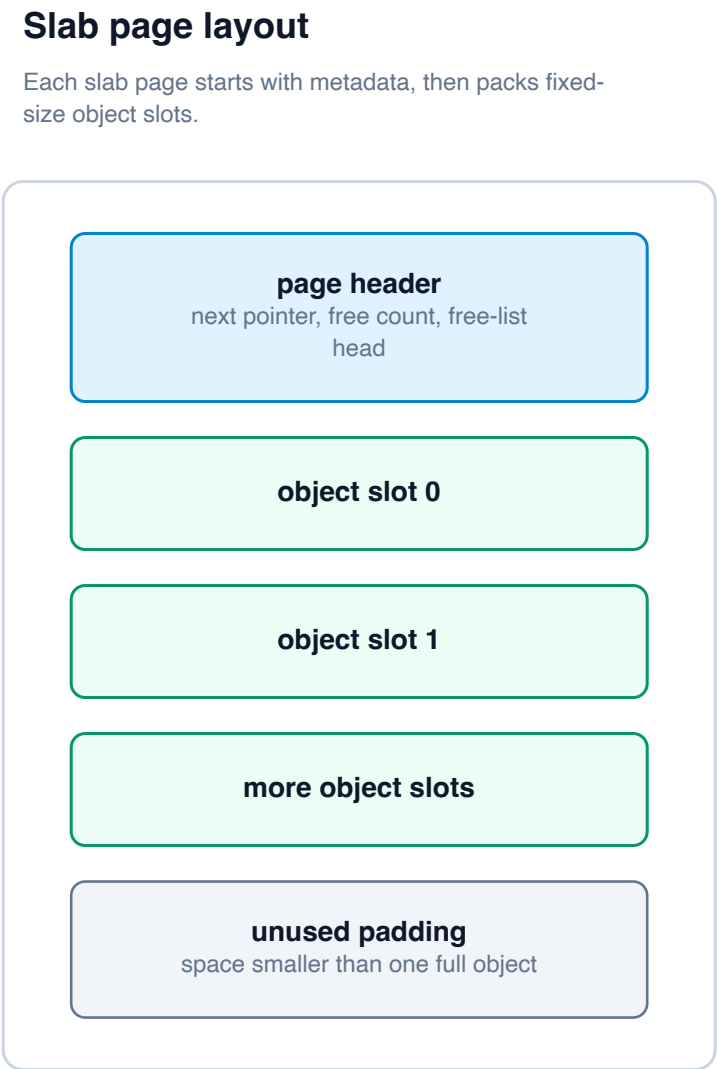
Structure

The slab allocator introduces two types:

```
typedef struct slab_hdr {
    struct slab_hdr *next;      /* intrusive list link */
    uint32_t         n_free;    /* free objects in this slab */
    void            *free_head; /* head of the embedded free list */
} slab_hdr_t;

typedef struct kmem_cache {
    const char  *name;
    uint32_t     obj_size;
    uint32_t     objs_per_slab;
    slab_hdr_t  *partial;       /* slabs with at least one free slot */
    slab_hdr_t  *full;          /* slabs with no free slots */
} kmem_cache_t;
```

A **slab** is exactly one 4 KB PMM page. The `slab_hdr_t` (12 bytes) sits at the very start; the remaining 4,084 bytes are divided into equal-sized object slots. A **cache** (`kmem_cache_t`) owns a list of these slabs for one specific object size and is itself allocated from the kernel heap.

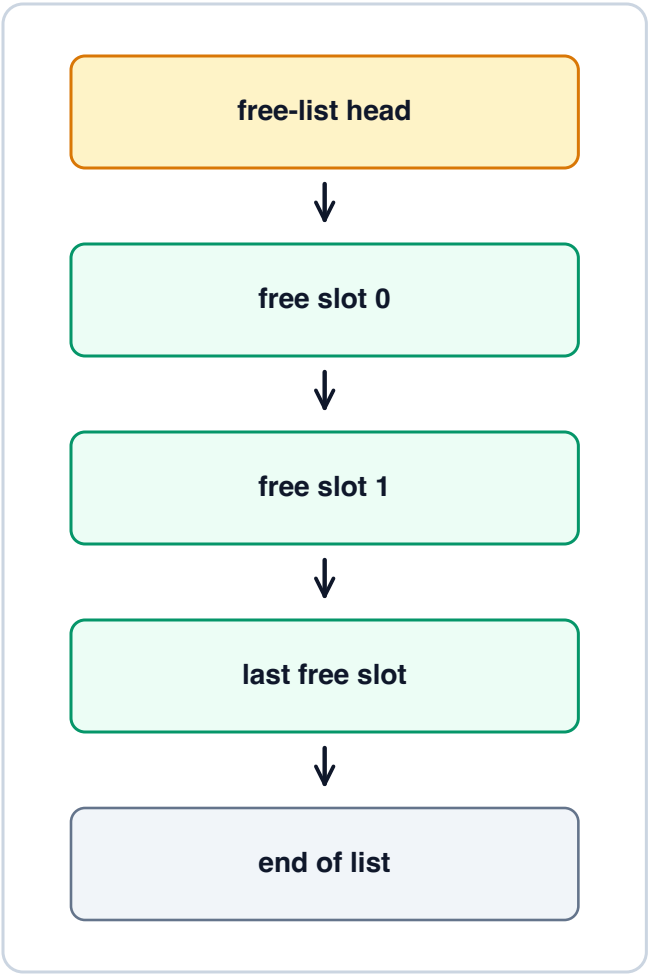
Layout Within a Slab Page



Chapter 8, Figure 6

The Embedded Free List

When a slot is free, its first four bytes hold a pointer to the next free slot. No separate array of free-slot indices is needed. After `kmem_cache_create`, the free list threads through all N slots in address order:



Chapter 8, Figure 7

Allocation pops `free_head`; deallocation pushes back. Both are O(1).

Finding the Owning Slab at Free Time

When `kmem_cache_free` is called, it needs to update the slab that contains the object — but the caller passes only the object pointer, not the slab. Because every slab is exactly one page and the `slab_hdr_t` is at the page base, the slab address is simply the object address with its low 12 bits cleared:

```
slab = (slab_hdr_t *)((uint32_t)obj & ~0xFFF);
```

This works as long as the object lies within the same page as its slab header, which `kmem_cache_create` guarantees by refusing object sizes that would not fit (`objs_per_slab == 0`).

Partial and Full Lists

The cache tracks two lists:

- **partial**: slabs with at least one free slot — the first place `kmem_cache_alloc` looks.
- **full**: slabs that are completely consumed — bypassed on allocation, moved back to partial when any object is freed into them.

When the partial list is empty and an allocation is needed, the cache claims a fresh PMM page, formats it as a new slab, and prepends it to partial. When freeing an object causes a slab to become completely empty, the slab is removed from partial and its page is returned to the PMM. This keeps the memory footprint tight: idle caches consume no physical pages.

Limits

The maximum object size the slab handles is `PAGE_SIZE - sizeof(slab_hdr_t)` = 4,084 bytes — enough for one object to fit per page. Objects larger than this must use `kmalloc` or `pmm_alloc_page` directly.

The Initialisation Order

The four memory subsystems must be initialised in a specific order:

1. **Physical page tracking** walks the firmware memory map and builds the allocation bitmap. This runs before paging is on, so it writes directly into kernel BSS.
2. **Paging** fills in the page directory and page tables and switches the CPU into virtual-address mode. From the next instruction onward, every memory access goes through the translation hardware.
3. **The heap allocator** installs its initial free list over the heap region. Paging is already on at this point, but because the heap is identity-mapped the addresses still resolve correctly.
4. **Object caches** (created on demand after the heap is up) each allocate a small descriptor from the heap and grow lazily — the first physical page for a cache is claimed only when the first object is actually allocated from it.

Trying to do any of these steps out of order is impossible. The heap depends on paging being on. Paging depends on the PMM. The PMM depends on the Multiboot memory map. The slab depends on both the heap (for the descriptor) and the PMM (for slab pages).

Where the Machine Is by the End of Chapter 8

By the end of this chapter we have four memory capabilities:

Capability	Provided by	How the kernel calls it
Track free physical pages	PMM	`pmm_alloc_page` / `pmm_free_page`

Capability	Provided by	How the kernel calls it
Translate virtual to physical addresses	Paging	Transparent — the CPU does it
Allocate and free arbitrary byte ranges	Heap	`kmalloc` / `kfree`
Allocate and free fixed-size objects efficiently	Slab	`kmem_cache_create` / `kmem_cache_alloc` / `kmem_cache_free`

Together, these four layers are the foundation for every feature in the chapters that follow. Loading programs, creating processes, managing files, buffering keystrokes — every one of those requires dynamic memory. The heap handles one-off and variably-sized allocations; the slab handles the high-frequency, fixed-size objects that dominate kernel hot paths.

Part III — Hardware Interfaces

With memory under control, the kernel can allocate data structures reliably — but it is still isolated. It cannot observe its own behaviour beyond what fits on the screen, it cannot receive input from the user, and it cannot reach the disk. A kernel that cannot read keystrokes or access persistent storage is not yet an operating system; it is a program that runs once and forgets everything the moment the power goes off.

This part is about giving the kernel senses. The problem has two dimensions. One is mechanical: each piece of hardware speaks its own protocol — the keyboard talks over the PS/2 port (the older round-connector interface for keyboards and mice) using scancodes (numeric codes the keyboard sends to indicate which key was pressed or released), the ATA disk controller (the standard interface that connects hard drives to the motherboard) exposes status and data registers through a pair of I/O port ranges, and both use interrupts to signal that they need attention. The other dimension is architectural: as the number of drivers grows, they must not interfere with each other or with the rest of the kernel. A keyboard handler that reaches into disk driver state, or a disk driver that assumes a particular interrupt vector, becomes a maintenance problem. The solution is a pair of registries — one for block devices like disks, one for character devices like the keyboard — that let drivers advertise a uniform interface without knowing anything about each other.

Chapter 9 establishes the kernel boot log: a structured, tagged logging facility that every subsequent driver uses to report its initialisation status. This is built before the drivers themselves because an uninstrumented driver is nearly impossible to debug. Chapter 10 implements the PS/2 keyboard driver: scancode decoding, modifier key tracking, a ring buffer that the rest of the kernel reads from, and the special handling for Ctrl+C and Ctrl+Z that the terminal layer will later formalise. Chapter 11 implements the ATA disk driver using Programmed I/O — a mode where the CPU directly polls the controller and moves data through I/O ports rather than using a DMA transfer — exposing a simple read/write interface over Logical Block Addressing (LBA), where every sector on disk is identified by a single sequential number rather than a cylinder, head, and sector triplet. Chapter 12 introduces the block and character device registries that abstract over individual drivers, so that the filesystem and terminal layers can talk to “a disk” and “a keyboard” without caring which specific driver is underneath.

By the end of Part III the kernel has eyes and hands. It can log structured messages to the console, accept characters from the keyboard, and read or write any sector on the disk. The building blocks for persistent storage are in place.

Chapter 9 — Kernel Boot Log

A Shared Voice for Every Subsystem

With the memory subsystem live and the heap available, the kernel can initialise subsystems that need dynamic allocation. As those subsystems come up they all need to report their status, and without a shared logging layer each one invents its own approach. One subsystem calls `print_string` directly with a literal string, another builds a message buffer inline, a third adds and removes diagnostic lines repeatedly during development. The screen becomes cluttered with inconsistent output that is hard to read and impossible to review after it scrolls away.

This chapter introduces a small logging layer called **klog**. It is not a debugging aid to be removed before shipping — it is the permanent voice the kernel uses to describe itself as it boots. Every subsystem from this chapter onward uses klog instead of calling `print_string` directly.

The Three Functions klog Provides

klog exposes three convenience functions, and between them they cover the common messages the kernel needs to produce. All three take a short **tag** — typically a three or four-character subsystem identifier — and a message string. `klog` prints the tag and message alone at `INFO` level. `klog_uint` appends a colon, a space, and a decimal number. `klog_hex` does the same but formats the number in hexadecimal. No caller has to format numbers by hand — that logic is entirely inside the logging layer.

Underneath those wrappers sits the more general `klog_log(level, tag, msg)` entry point. That is what gives the log stream explicit severity levels such as `DEBUG`, `INFO`, `WARN`, and `ERROR` while still keeping the older helper calls small and readable.

Every call produces a single line on the VGA console and on QEMU's debugcon when that output is enabled. The rendered format now includes a boot-relative timestamp, a severity, a subsystem tag, and the message text:

Kernel log format

Each log line starts with a subsystem tag, then the human-readable message.

tag	message
MEM	free pages: 31500
DISK	drive ready
FILES	filesystem mounted
FAULT	page-fault exception

Chapter 9, Figure 1

How It Is Implemented

klog still depends on `print_string`, the VGA text output function from Chapter 3, and still mirrors output to the optional debug console. But the log is no longer write-only. Each rendered message is also copied into a fixed-size in-memory ring buffer so the most recent history survives after the VGA screen has scrolled away.

The retained record is intentionally compact: it stores the uptime tick count, the chosen log level, a short tag, and a bounded message string. The timestamp comes from the same PIT-driven timebase that we advance every timer interrupt, so the log lines can be rendered later as `seconds.milliseconds since boot` without consulting the RTC again.

The numeric formatting helpers are still self-contained inside klog. `klog_uint` and `klog_hex` build a short formatted suffix and then hand the finished message to the shared logging path, so every public API ends up going through the same formatter, ring-buffer insertion, and console-mirroring code.

What a Boot Log Looks Like

With klog in use everywhere, the kernel's startup path reads as a structured transcript rather than scattered, inconsistent messages. A typical boot produces:

Typical boot log

Startup reads as a structured transcript of subsystem initialization, with one tagged line per subsystem event.

tag	message
MEM	boot memory map found
MEM	free pages: 31500
CPU	user-mode entry state installed
DISK	disk initialized
TRAPS	interrupts enabled
FILES	filesystem mounted
FILES	shell program located
PROCESS	entering user mode

Chapter 9, Figure 2

Each line is produced by a `klog` call somewhere in `start_kernel` or one of the subsystem initialisers. The PMM lines are intentionally permanent — knowing the number of free pages at boot is operationally useful and catches memory-map parsing bugs the moment they appear.

Because the recent history is retained in memory, the boot log is no longer limited to what still fits on the visible VGA screen. `procfs` exposes that retained transcript as `/proc/kmsg`, and the user-space `dmesg` utility simply opens and prints that file through the normal `SYS_OPEN` and `SYS_READ` path. We therefore have both immediate human-visible console output and a later-readable diagnostic history.

When to Add a klog Call

The rule is simple: any subsystem that completes initialisation, encounters a recoverable error, reports a numeric measurement that varies between boots, or catches a CPU exception should emit a klog line. A completed initialisation usually calls `klog`; a numeric result calls `klog_uint` or `klog_hex`; a path that needs stronger severity can call `klog_log` directly. There is still no runtime log filtering — this kernel is small enough that every message is worth keeping — but severity tags make the transcript easier to scan once it is viewed later through `/proc/kmsg` or `dmesg`.

Where the Machine Is by the End of Chapter 9

By the end of this chapter, every kernel subsystem reports its status through one shared logging interface, and the boot console reads as a consistent structured log. Later chapters build on that by exposing the retained log through `procfs`, so the same messages can be read interactively from user space long after the screen has scrolled.

Chapter 10 — Keyboard Input

From a Physical Key to a Character in a Buffer

This chapter follows the life of a single keystroke from the moment a key closes a contact on the keyboard matrix to the moment the kernel has a character ready for a user program to read. Six distinct things happen along the way:

1. The user presses a key. The keyboard controller (a small microcontroller inside the keyboard housing) detects the press and emits an electrical signal on the **IRQ** (Interrupt Request) line 1 — **IRQ1** — to the PIC (Programmable Interrupt Controller).
2. The PIC forwards the signal to the CPU as interrupt vector 33 — the vector IRQ1 was remapped to in Chapter 4.
3. The CPU saves its current state, looks up vector 33 in the IDT, and enters the common IRQ entry path.
4. The common IRQ path saves the remaining registers, converts vector 33 to IRQ line 1, and dispatches to whichever handler claimed that line.
5. The registered handler is the keyboard handler, which is the code this chapter describes.
6. The keyboard handler reads the **scancode** — the raw byte the keyboard sent — from an I/O port, converts it to an ASCII character, and appends it to a circular buffer where a later read call will find it.

Between steps 1 and 6 the rest of the system is completely passive. The keyboard driver is one of the few places in the OS where execution flows from hardware to software rather than the other way around.

Scancodes, Not ASCII

A PC keyboard does not send ASCII. It sends **scancodes** — small numeric codes that identify which physical key was pressed or released. The **PS/2** keyboard protocol (named after IBM's 1987 Personal System/2 computer line, which standardised the connector and signalling still used by modern keyboards today) defines several sets of scancodes, and by default the keyboard uses **scancode set 1**, in which each key has a **make code** sent when the key is pressed and a **break code** sent when it is released. The break code for any key is the make code with bit 7 set — that is, the make code plus `0x80`.

Our driver is deliberately simple: it ignores key releases entirely. When the handler reads a byte with bit 7 set, it returns without doing anything. Modifier keys like Shift and Ctrl are also ignored — they exist in the driver's scancode table but map to zero, meaning "no character". Because of this, typing `A` and `a` both produce the same lowercase letter. Adding proper modifier tracking (watching for shift down and shift up to toggle a flag) is a straightforward extension.

The scancode is read from **I/O port `0x60`**, the keyboard controller's data port. Reading that port both returns the most recent scancode and clears the keyboard's output buffer, making room for the next press.

The Scancode-to-ASCII Table

We map scancodes to characters with a static lookup table indexed by scancode. Scancode `0x02` through `0x0D` are the top-row number keys and their variants (`1`, `2`, `3`, …, `0`, `-`, `=`). Scancode `0x0E` is

Backspace; the table returns `\b`. Scancode `0x0F` is Tab; the table returns `\t`. The next row (`0x10` through `0x1B`) is `q` through `]`. The row after that is `a` through `` ` ``. Then the bottom row, `\` through `/`. Scancode `0x1C` is Enter, mapped to `\n`. Scancode `0x39` is the spacebar, mapped to `' '`.

Any scancode that falls outside the table, or that maps to zero, produces no character at all. This is how Shift, Ctrl, Alt, Caps Lock, and the function keys are effectively disabled.

The Ring Buffer

Our handler runs inside an interrupt context. Interrupt context has two hard rules: it must finish quickly (other interrupts may be pending), and it must not block waiting for anything. At the same time, the program that wants to read a keystroke is running at a completely different rate and may not be ready to consume a character at the exact moment one arrives.

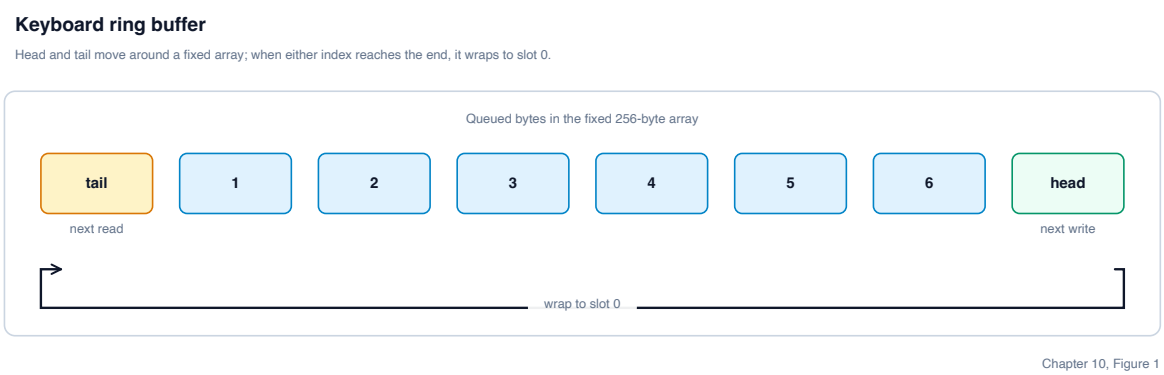
The classic solution is a **ring buffer**, also called a circular buffer. A ring buffer is a fixed-size array with two indices: a head index that points to the next slot to write, and a tail index that points to the next slot to read. The interrupt handler writes at the head and advances the head index. The consumer reads at the tail and advances the tail index. When either index walks off the end of the array, it wraps around to zero.

We declare a 256-byte buffer and the two indices:

```
#define KB_BUFFER_SIZE 256
static char kb_buffer[KB_BUFFER_SIZE];
static int  kb_head = 0;
static int  kb_tail = 0;
```

The buffer is **empty** when the head equals the tail. It is **full** when advancing the head by one would make it equal to the tail — in which case the new character is dropped rather than overwriting an old one.

Visually, the buffer is a fixed array with the two indices chasing each other around its edge. After the user types `hi`, the state looks like this:



When the consumer reads one character, `kb_tail` advances; when the interrupt pushes one, `kb_head` advances. Whichever index reaches the end of the array next wraps to zero and keeps going.

The producer path writes at the head, checks the full condition, and advances. The consumer path reads at the tail (if the buffer is not empty), advances the tail, and returns the character.

Neither side allocates memory, neither blocks, and neither requires locking because there is only one producer (the interrupt) and one consumer (the main kernel code) and the indices are updated in a safe order.

Modifier Keys and Ctrl+C

Our driver now tracks modifier key state beyond just Shift. Two modifier flags — `shift_held` and `ctrl_held` — are updated on every make code and cleared on the corresponding break code.

Shift (scancodes `0x2A` and `0x36`) has been tracked since the original implementation. We maintain separate shifted and unshifted scancode tables and select between them based on `shift_held`. Ctrl (scancode `0x1D`) follows the same pattern: the make code sets `ctrl_held = 1`; the break code (`0x9D = 0x1D | 0x80`) clears it.

When `ctrl_held` is set and scancode `0x2E` (the letter `c`) arrives, we do not push a character to the ring buffer. Instead the keyboard path hands the event to the foreground-signal logic, which decides where to send the interrupt signal based on what is currently running:

- If a foreground job is running, the kernel sends SIGINT — an interrupt signal that terminates the target process by default — to the currently running process and to the process that the waiting program is blocked on. This covers jobs with multiple processes: a pipe such as `writer | reader` forks two children and the waiting program is blocked on the writer, so both the currently scheduled reader and the writer get SIGINT. Visual feedback (`^C\n`) is printed before delivering the signals.
- If no child is running (the interactive program is at the prompt), the kernel pushes the byte `0x03` — the **ETX** (End of Text) ASCII control character — into the ring buffer. The prompt loop detects ETX, prints `^C`, discards the current input line, and re-prompts.

This design means our keyboard driver never makes policy decisions about process termination. It only detects the keystroke and reports it to the signal subsystem. Whether the signal kills a process, is caught by a user handler, or is silently ignored is determined by each process's signal disposition.

Ctrl+Z follows the same pattern: when `ctrl_held` is set and scancode `0x2C` (the letter `z`) arrives, the TTY layer prints `^Z\n` and sends a terminal-stop signal to the foreground process group. The stopped process enters a suspended state and is invisible to the scheduler until a continue signal arrives.

Sending the End-of-Interrupt Signal

After the handler has processed the keystroke, it must tell the PIC that the interrupt is complete. The PIC does not know when a handler finishes — it only knows that one is in progress. If we return from the handler without notifying the PIC, it will refuse to deliver any future interrupt of the same priority, which on IRQ1 means the keyboard stops working entirely.

The notification is a single byte written to the master PIC's command port. That byte is `0x20`, and it goes to port `0x20`. For interrupts that come from the slave PIC (IRQs 8–15), we have to send the EOI to both the slave (port `0xA0`) and the master (port `0x20`), because the slave's signal reaches the CPU by way of the master. IRQ1 is on the master, so only one write is needed.

The EOI is sent by the shared IRQ dispatch layer after the registered handler returns, not by the keyboard handler itself. Every IRQ handler benefits from this — the EOI is centralised so individual drivers cannot forget to send it.

Registering With the Kernel

Our keyboard driver does not expose a bespoke read function through a header that other subsystems include. Instead, startup registers the keyboard with two kernel-wide registries before interrupts are enabled.

First, the driver claims IRQ1 in the IRQ dispatch table. From that point on, every keyboard interrupt reaches the driver without any code outside the driver needing to know the handler's name or address.

Second, it publishes the same character-device interface under both `"stdin"` and `"tty0"`. The syscall layer reads file descriptor 0 by looking up whichever device currently owns `"stdin"` and calling through its ops-table. This means the syscall code has no compile-time dependency on the keyboard driver. A future driver could register a different device under `"stdin"` without touching the syscall code at all. Chapter 12 describes the character device registry in detail.

Extended Scancodes and Page Up / Page Down

The PS/2 keyboard uses a two-byte sequence for many keys that do not appear in the original 83-key AT layout — arrow keys, Home, End, Insert, Delete, Page Up, and Page Down among them. These keys send a prefix byte of `0xE0` followed by a second make code, instead of the single-byte make codes used by letters and digits.

We handle this with a one-bit flag, `e0_prefix`. When the interrupt fires and the byte read from port `0x60` is `0xE0`, the handler sets the flag and returns immediately — there is nothing to push into the ring buffer yet. When the very next interrupt arrives, the flag is set, so the handler treats the new scancode as the second half of an extended pair:

First byte	Second byte	Key
`0xE0`	`0x49`	Page Up
`0xE0`	`0x51`	Page Down

Rather than invent a new encoding, we map these two keys to the ASCII control codes **SOH** (Start of Header, `0x01`) and **STX** (Start of Text, `0x02`). These byte values are never produced by printable keys and are not used by any other part of the input loop, so they act as private signals that the consuming code can detect without ambiguity. Any unrecognised extended scancode is silently dropped. Key-release events (scancodes with the high bit set) clear the `e0_prefix` flag and return, the same as before.

Polling Versus Blocking

The `SYS_READ` implementation at this stage still **spins** on the keyboard character source — it polls in a tight loop until a non-zero byte appears. This is wasteful of CPU cycles (a busy-waiting loop does no useful work) but simple to implement. A more sophisticated approach would put the

process to sleep and use the `HLT` instruction (which halts the CPU until the next interrupt arrives) to avoid burning cycles. That refinement is straightforward once a proper sleep/wake mechanism is in place.

Where the Machine Is by the End of Chapter 10

Every key the user presses now produces a character in the kernel's ring buffer. Any reading program can call `SYS_READ` to retrieve those characters one at a time. Pressing Ctrl+C while a foreground job is running sends an interrupt signal to every runnable process in that job (including all stages of a pipeline); pressing Ctrl+C at the prompt cancels the current input line. Pressing Ctrl+Z sends a terminal-stop signal to the foreground process group, suspending the running program and returning control to the waiting program.

The architecture introduced in this chapter — interrupt fires, handler pushes to a ring buffer (or sends a signal), consumer pulls from the ring buffer — is the same pattern real operating systems use. The hardware-specific pieces (scancode decoding, I/O port numbers, EOI sequences) vary by device and by PIC, but the separation between the interrupt producer and the sleeping consumer is universal.

Chapter 11 — ATA Disk Driver

Why We Need to Talk to Disks Directly

Up to this point every byte the kernel has touched has been in RAM. The kernel image itself was loaded by GRUB, and once GRUB was finished, the BIOS disk services it used went away — the BIOS is a 16-bit real-mode program, and the CPU has been in 32-bit protected mode since Chapter 2. If we want to read or write persistent data (a filesystem, user programs, configuration files), we have to speak directly to the disk controller.

The disk interface on every x86 PC made before **SATA** (Serial ATA, a newer point-to-point successor that replaced the parallel bus with a serial link) became common is called **ATA** — AT Attachment, named after the IBM PC/AT. The underlying protocol is also sometimes called IDE (Integrated Drive Electronics) or PATA (Parallel ATA). QEMU's default virtual hard drive presents itself as an ATA drive on the standard controller addresses, which makes writing a driver for it the natural next step.

Two Ways to Move Data: PIO and DMA

There are two distinct strategies for transferring data between an ATA drive and RAM.

PIO (Programmed I/O) is the simpler of the two. The CPU issues a command to the drive, waits for the drive to say "data ready", and then reads or writes the 512 bytes of a sector one word at a time through an I/O port. Every byte passes through the CPU. It is slow — the CPU does nothing else during the transfer — but it requires no configuration of any other hardware.

DMA (Direct Memory Access) is the faster strategy. A separate chip called a DMA controller is programmed with a source, a destination, and a length, and it moves data between the drive and RAM without the CPU's involvement. The CPU is free to run other code during the transfer. DMA is mandatory for any serious OS but requires additional setup: the DMA controller has to be configured, the buffer has to be in memory the controller can reach, and an interrupt has to be wired up to notify the CPU when the transfer is complete.

We use **PIO** because simplicity wins at this stage. Every sector is transferred by the CPU one word at a time. Moving to DMA is a future enhancement.

The ATA Register File

The primary ATA controller is controlled through two blocks of I/O ports. The main block starts at port `0x1F0`; the control block is at port `0x3F6`.

Port	Read	Write
`0x1F0`	Data (16-bit word)	Data (16-bit word)
`0x1F1`	Error register	Features register
`0x1F2`	Sector count	Sector count
`0x1F3`	LBA bits 0–7	LBA bits 0–7
`0x1F4`	LBA bits 8–15	LBA bits 8–15
`0x1F5`	LBA bits 16–23	LBA bits 16–23
`0x1F6`	Drive/head	Drive/head

Port	Read	Write
`0x1F7`	Status	Command
`0x3F6`	Alternate status	Device control

The data port (`0x1F0`) is the only one that carries the actual contents of a sector. Every other port carries control information — which sector to read, how many sectors, which drive on the bus, and so on.

The most important register is the **status** register at `0x1F7`. After every command, we read this register in a loop to find out what the drive is doing. Its bits are:

Bit	Name	Meaning
`0`	error	Last command failed
`1`	index	Index pulse bit
`2`	fixed	Reserved status bit
`3`	data	Data is ready to transfer
`4`	seek	Seek operation completed
`5`	fault	Drive reported a fault
`6`	ready	Controller can accept work
`7`	busy	Controller is busy

We watch four of these flags:

- **BSY** (bit 7) — the controller is busy processing a command. Do not issue anything while this is set.
- **DRDY** (bit 6) — the drive is ready to accept a new command.
- **DRQ** (bit 3) — the drive has a sector buffer ready to be transferred.
- **ERR** (bit 0) — the last command failed. The error register (`0x1F1`) holds details.

Addressing Sectors With LBA

A hard drive stores data as a sequence of 512-byte **sectors**. The way those sectors are numbered has changed over the years. The original scheme was **CHS** (Cylinder/Head/Sector), which reflected the physical layout of the platters. Modern drives use **LBA** — Logical Block Addressing — which ignores physical geometry and numbers every sector starting from zero.

We use **LBA28**, a variant of LBA that fits the sector number in 28 bits — enough for 128 GB of data, more than sufficient for a 1 MB disk image. The 28 bits are split across four registers: bits 0 through 23 go into `0x1F3`, `0x1F4`, and `0x1F5`, and the top four bits go into the low nibble of the drive/head register at `0x1F6`.

Drawn out as a bit field, an LBA28 value is chopped like this:

Bits	Field	Stored in
`0–7`	Low byte	Low address byte

Bits	Field	Stored in
`8–15`	Middle byte	Middle address byte
`16–23`	High byte	High address byte
`24–27`	Top nibble	Drive-select byte

The drive/head register at `0x1F6` packs the top LBA nibble together with the drive-select bits:

Bits	Field	Meaning
`0–3`	Top sector bits	Sector bits 24 through 27
`4`	Drive	Choose master or slave
`5`	Fixed 1	Required constant bit
`6`	Address mode	Use linear sector addressing
`7`	Fixed 1	Required constant bit

Bits 7 and 5 are obsolete holdovers that historically must be set to 1. A typical drive/head value for the DUFS disk is `0xE0` ORed with the top four bits of the LBA.

Waiting for the Drive

Because PIO mode does not use interrupts (we do not wire IRQ14 into the IDT), every wait is a polling loop. The driver spins on the status register until either the drive is ready or an iteration counter runs out (a crude timeout to catch dead drives). The busy flag must clear before any command is issued — sending a command while BSY is set is a protocol violation. After a read or write command is issued, the driver waits for the data-request bit so it knows the drive's sector buffer is ready.

The 400-Nanosecond Delay

The ATA specification requires a short delay after certain operations to give the drive's internal state machine time to update the status register. The canonical way to implement this delay is to read the alternate status register (`0x3F6`) four times. Each I/O port read takes roughly one hundred nanoseconds on real hardware, so four reads give the required 400-nanosecond pause. Reading the alternate status register, as opposed to the main status register at `0x1F7`, has the additional benefit of not clearing any pending interrupt from the drive.

Initialising the Controller

Initialisation starts by performing a **software reset** on the primary bus. The driver sets the **SRST** (Software ReSeT) bit in the device-control register, clears it again, and performs the 400 ns delay. After the reset, it waits for BSY to clear and reads the main status register. On a real machine, a value of `0xFF` means the bus is floating — there is no drive attached. Anything else indicates a drive is present, and the driver continues to wait for the DRDY bit before declaring the drive ready.

Reading a Sector

Reading one sector is a sequence of seven steps, in order:

1. Wait for BSY to be clear.
2. Write the sector count (`1`) to `0x1F2`.
3. Write the low, middle, and high bytes of the LBA to `0x1F3`, `0x1F4`, and `0x1F5`.
4. Write the drive select byte (LBA mode, master drive, top four bits of the LBA) to `0x1F6`.
5. Write the READ SECTORS command (`0x20`) to the command register at `0x1F7`.
6. Wait 400 nanoseconds, then wait for DRQ to be set.
7. Read 256 16-bit words from the data port at `0x1F0` into the caller's buffer.

The seventh step is where the actual data crosses from the drive into RAM. Rather than reading one word at a time in a C loop, we use a single x86 instruction called `rep insw` ("repeat input string word"), which reads 16-bit words from an I/O port into memory under the control of the ECX register. We call this through a tiny inline-assembly wrapper that passes the port number, the buffer address, and the count of 256. One instruction copies the entire sector.

Writing a Sector

Writing a sector follows the same skeleton, but the command byte is WRITE SECTORS (`0x30`) and the data flow reverses — we use `rep outsw` to write 256 words from the buffer to the data port. There is one extra step at the end: we send a **cache flush** command (`0xE7`). Modern drives have write caches that hold data in volatile memory until they decide to commit it. Without an explicit flush, a power loss could leave the supposedly-written data unrecorded. The flush command tells the drive to wait until everything in the cache has reached the physical medium before reporting completion.

Wiring It Into the Kernel

The ATA controller is brought up during kernel startup after the heap is set up but before the full interrupt path is live. On boot, we print one of three messages through the klog layer introduced in Chapter 9: a drive was found and is ready, no drive was detected, or an error occurred during initialisation. This gives immediate visible feedback that the controller is responding.

Once the driver is initialised, it wraps its sector read/write routines in a small ops-table and publishes that table to the block device registry under the name `"hd0"`. Chapter 12 describes that registry in detail. After that, the filesystem layer and the syscall layer find the drive by name rather than calling ATA-specific code directly.

Where the Machine Is by the End of Chapter 11

We can now read and write arbitrary 512-byte sectors on the primary ATA drive. That capability is exactly what a filesystem needs — the filesystem imposes structure on top of the raw sectors, turning "sector 22" into "the file called `hello.txt`". Chapter 13 takes that step. For now, the driver is intentionally minimal: one sector at a time, polling only, master drive on the primary channel, no interrupt handling. It is not efficient, but it is correct and easy to reason about.

Chapter 12 — Block and Character Device Registries

The Coupling Problem

Chapter 11 left us with an ATA driver that can read and write sectors. Before this chapter, the filesystem read sectors by calling ATA functions directly, and the syscall layer read keystrokes by calling the keyboard's internal function. These direct dependencies meant the filesystem and syscall layer had compile-time knowledge of both the keyboard driver and the ATA driver. Adding a second disk drive or a USB keyboard would have required modifying the filesystem and syscall code — the wrong place to touch for a hardware change.

The device registries decouple these relationships. A driver publishes a small ops-table under a short name. Consumers look up the device by name and call through the function pointer. Neither side needs to know the other's implementation details at compile time.

Block Devices

A block device is one that transfers data in fixed-size chunks — on this system, 512-byte sectors. The block device registry provides a central place to publish and look up block devices.

The ops-table for a block device is a pair of function pointers:

```
typedef struct {
    int (*read_sector)(uint32_t lba, uint8_t *buf);
    int (*write_sector)(uint32_t lba, const uint8_t *buf);
} blkdev_ops_t;
```

Both functions take an **LBA** (Logical Block Address) — the zero-based sector number — and a pointer to a 512-byte buffer. They return 0 on success and non-zero on error.

The registry holds up to four entries. Each entry is a name (up to seven characters) paired with a pointer to a `blkdev_ops_t`. The table is static and zero-initialised; a slot is free when its name's first byte is zero.

Slot	Name	Purpose
`0`	`hd0`	Active disk entry
`1`	free	Empty slot for a future device
`2`	free	Empty slot for a future device
`3`	free	Empty slot for a future device

Registering a block device means finding the first free slot and filling it. Looking one up means doing a linear scan and returning the matching ops-table pointer, or NULL if no device with that name has been registered.

Our ATA driver publishes itself under the name `"hd0"` once initialisation succeeds. After that, the filesystem layer resolves `"hd0"` once during its own startup and caches the resulting ops-table pointer for all subsequent reads.

A file handle in the process open-file table also stores a pointer to the block device ops-table (resolved at `SYS_OPEN` time). When the syscall layer handles a `SYS_READ` on a file descriptor, it calls through the ops-table rather than any named function from the ATA driver.

Character Devices

A character device is one that transfers data one byte at a time, typically an interactive input or output stream. The character device registry uses the same pattern as the block device registry but with a simpler ops-table:

```
typedef struct {
    char (*read_char)(void);   /* returns 0 if no data is available */
    void (*write_char)(char c); /* may be NULL for input-only devices */
} chardev_ops_t;
```

The registry holds up to four entries and is queried the same way as the block-device table: publish a name/ops pair at startup, then resolve that name later when some other subsystem wants terminal-style byte I/O.

Our keyboard driver publishes the same character source under two names:

- `"stdin"` — used by `SYS_READ` when the calling process reads from file descriptor 0.
- `"tty0"` — the conventional name for the first terminal device, available for any future code that wants to write to or read from the system console.

When the syscall dispatcher handles `SYS_READ` on file descriptor 0, it resolves the `"stdin"` ops-table and polls `read_char()` in a tight loop until it returns a non-zero byte. Each iteration emits a `pause` instruction — an x86 hint that marks this as a deliberate spin-wait, reducing power consumption and avoiding a false memory-ordering hazard on hyperthreaded designs. A future improvement would transition the process to a blocking sleep state so the CPU is freed entirely while waiting; the current implementation is correct but wastes cycles during the wait.

Why Function Pointers Instead of Compile-Time Includes

Both registries use runtime function pointers rather than compile-time `#include` dependencies. The tradeoff is worth it:

- **Adding a new disk** requires only registering under a new name (e.g., `"hd1"`). No existing source file changes.
- **Swapping the keyboard** for a USB HID (Human Interface Device) requires only registering the new driver under `"stdin"`. The syscall layer is unchanged.
- **Testing** becomes possible without hardware: a test harness could register a mock block device that returns known sector data without touching any disk.

The cost is a single pointer indirection on each read. At the scale of a 512-byte sector read or a keyboard poll, this is completely invisible.

Where the Machine Is by the End of Chapter 12

We now route all disk I/O through the block device registry and all character I/O through the character device registry. The ATA driver is `"hd0"` in one table; the keyboard is `"stdin"` and `"tty0"` in the other. No code outside each driver reaches its internal functions directly. The coupling between subsystems has been replaced by a narrow, named interface that any future driver can implement.

Part IV — Files and Storage

The ATA driver from Part III can read and write disk sectors, but a sector number is not a name. To open a program or a document, the user — and every program they run — thinks in terms of paths like `bin/shell` or `docs/notes.txt`, not in terms of sector 1,847 on drive 0. Translating between those two worlds is the job of the filesystem.

A filesystem is fundamentally a map: given a name, find the data. Building that map on top of raw sectors requires agreeing on where the map itself is stored, how it is structured, and how it is kept consistent when files are created, grown, or deleted. The design choices made here have long-lasting consequences. An inode-based layout, where each file's metadata lives in a fixed-size record at a known location, makes lookup fast and makes file sizes independent of where the name appears in the directory. A bitmap to track free blocks keeps allocation O(1). Indirect block pointers let files grow beyond what fits in the inode itself. These are not arbitrary choices — they are the same tradeoffs that Unix filesystems have refined over fifty years.

The second problem this part addresses is abstraction. A kernel that bakes filesystem details into every piece of code that touches storage becomes fragile: adding a second filesystem type, or changing the on-disk format, means touching code everywhere. The Virtual File System layer — the VFS — solves this by placing a thin operations table between callers and the filesystem implementation. Everything above the VFS talks to `vfs_open`, `vfs_read`, `vfs_create`; everything below the VFS implements those operations for a specific on-disk format. Swapping or adding filesystem drivers becomes a matter of registering a new operations table, not rewriting the callers.

Chapter 13 builds DUFS — the project's native inode-based filesystem — from the superblock through the inode and block bitmaps, the block pointer structure, directory entries, and path resolution. Chapter 14 wraps it in the VFS layer, defining the operations table that DUFS registers against and that all higher-level code uses from this point forward.

By the end of Part IV the disk has structure. Paths resolve to inodes, inodes point to data blocks, and the kernel can create, read, write, and delete named files in a directory hierarchy. The storage layer is ready to serve the process and shell layers that come next.

Chapter 13 — An Inode-Based Filesystem (DUFS v3)

Why a Filesystem Is Necessary

Chapter 12 left us with a block device registry that routes all disk I/O through named ops-tables. Up to this point we loaded `shell` from a hard-coded sector offset on the disk. That worked because the Makefile placed the file at a known location and the loader knew exactly where to look. It stops working the moment a second program is added, because we have no way to distinguish one binary from another — every sector is just a sequence of bytes.

A **filesystem** is the layer of software that turns a numbered sector into a named file and back again. This chapter builds DUFS v3 — "OS File System, version 3" — a compact inode-based filesystem modelled after the classical UNIX design with Linux-compatible limits. Understanding it requires following the path a single `open("shell")` call takes from the kernel's VFS layer down to a raw ATA sector read.

Why Inodes?

The very first DUFS stored each file's name, size, and starting sector together in a single flat directory entry. That worked for a tiny proof-of-concept but has four fundamental problems:

Renaming is expensive. Moving a file between directories means rewriting the entry's parent field, which in the v1 design is an index into the same directory table, making renames entangled with directory state.

Hard links are impossible. A hard link is two names pointing at the same file content. With metadata embedded in the directory entry, giving one file two names would require duplicating all the metadata and keeping both copies in sync — it cannot be done reliably.

Contiguous allocation is a dead end. V1 stores only a `start_lba`, so every file must live in one contiguous run of sectors. A file cannot grow past an adjacent occupied sector without being relocated. With block pointers in the inode, a file can be assembled from non-contiguous blocks anywhere on the disk.

Directory entries bloat with metadata. When a directory entry holds all the metadata, scanning it for a name lookup also reads timestamps, sizes, and permissions — data that the name lookup does not need.

UNIX (Bell Labs, 1969) separated name from metadata by introducing the **inode** (index node): a small fixed-size record on disk that holds the file's type, size, link count, and an array of block pointers. Directory entries become `(name, inode_number)` pairs only. Two names can point at the same inode number — that is a hard link. Renaming a file within the same directory only rewrites the directory entry, not the inode. The inode is the ground truth; the directory is just the index.

Linux's `ext2` filesystem, the direct descendant of the original UNIX design, works exactly this way. DUFS v3 follows the same principles and matches Linux's key limits: `NAME_MAX = 255`, `PATH_MAX = 4096`, and 4096-byte data blocks.

The On-Disk Layout

DUFS v3 divides the disk into six regions. The ATA driver works in 512-byte **sectors** (LBAs). DUFS groups eight consecutive sectors into one 4096-byte **block** — the unit at which file data and directory content are allocated and read. **LBA** (Logical Block Addressing) numbers sectors from zero.

Chapter 13, Figure 1

The first 274 LBAs are entirely overhead — superblock, bitmaps, and the inode table. File data

and directory content begin at LBA 274. The block bitmap can track up to 32,768 data blocks (128 MB); the inode table holds up to 1,024 inodes. The disk image is 50 MB (102,400 sectors).

On-Disk Structures

The superblock, inode, and directory entry are defined as packed C structs so the kernel can read a raw sector buffer and cast it directly to the right type.

Superblock — exactly one sector at LBA 1. It identifies the filesystem and records the locations of all other regions:

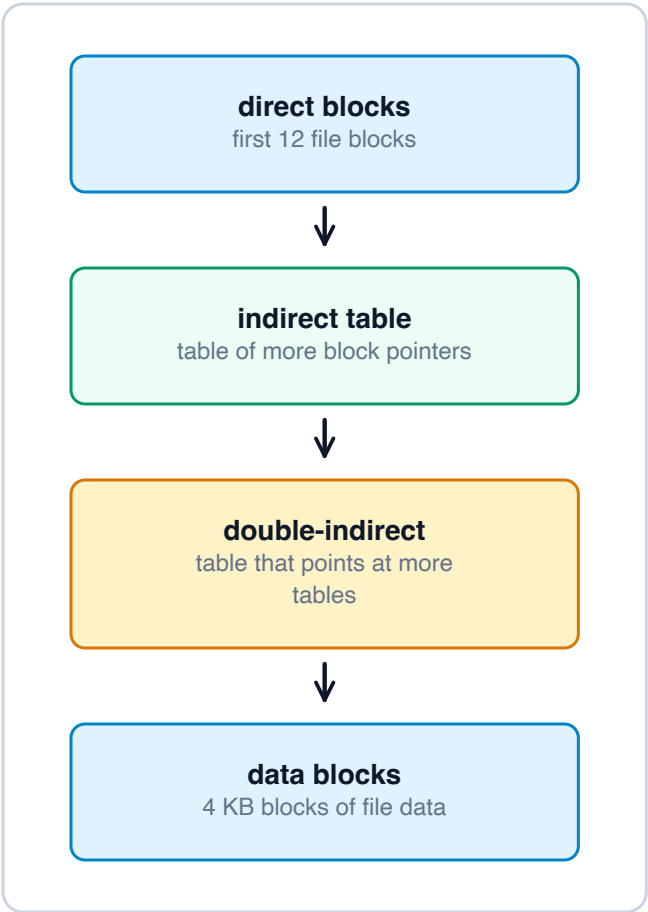
```
typedef struct {
    uint32_t magic;             /* 0x44554603 — "DUF\x03" (DUFS v3)         */
    uint32_t total_sectors;     /* total sectors in the disk image           */
    uint32_t inode_count;       /* number of inode slots (1024)              */
    uint32_t inode_bitmap_lba;  /* start LBA of the inode bitmap block (2)  */
    uint32_t block_bitmap_lba;  /* start LBA of the block bitmap block (10) */
    uint32_t inode_table_lba;   /* start LBA of the inode table (18)        */
    uint32_t data_lba;          /* start LBA of the first data block (274)  */
    uint8_t  pad[484];          /* zeroed padding to fill the sector         */
} __attribute__((packed)) dufs_super_t;
```

The **magic** field is the filesystem's fingerprint. When `fs_init` reads the superblock it compares the magic against the expected constant. A mismatch means the disk image was built by a different version — we refuse to mount rather than interpret random bytes as a filesystem.

Inode — 128 bytes, four per sector. An inode never stores the file's name; it stores everything else about the file:

```
typedef struct {
    uint16_t type;            /* DUFS_TYPE_FILE = 1, DUFS_TYPE_DIR = 2      */
    uint16_t link_count;      /* directory entries naming this file          */
    uint32_t size;            /* file size in bytes (0 for directories)      */
    uint32_t block_count;     /* data blocks currently allocated             */
    uint32_t mtime;           /* last-modified Unix timestamp, UTC seconds   */
    uint32_t atime;           /* last-accessed Unix timestamp, UTC seconds   */
    uint32_t direct[12];      /* direct block LBAs (0 = unallocated)         */
    uint32_t indirect;        /* single-indirect block LBA (0 = none)        */
    uint32_t double_indirect; /* double-indirect block LBA (0 = none)        */
    uint8_t  pad[52];         /* zeroed padding to reach 128 bytes           */
} __attribute__((packed)) dufs_inode_t;
```

The block pointers are the heart of the inode. Each block is 4096 bytes. Twelve `direct` entries cover up to 48 KB directly. When a file exceeds that, `indirect` holds the LBA of a 4096-byte **indirect block** containing 1024 LBA values — adding up to 4 MB more. For files larger still, `double_indirect` holds a 4096-byte block whose 1024 entries each point to their own indirect block, covering up to 4 GB. On a 50 MB disk, the practical ceiling is the disk itself.



Chapter 13, Figure 2

The capacity at each tier is: 12 direct blocks × 4096 bytes = **48 KB**; one indirect block × 1024 entries × 4096 bytes = **4 MB**; one double-indirect block × 1024 × 1024 × 4096 bytes ≈ **4 GB**. On a 50 MB disk image the practical ceiling is the disk itself.

A block pointer value of 0 means "not allocated". LBA 0 is always unused on DUFS disks, so 0 is a safe sentinel that costs nothing to detect.

Directory entry — 260 bytes, fifteen per 4096-byte block. Every directory is a flat array of these:

```
typedef struct {
    char     name[256];  /* null-terminated filename (max 255 characters)    */
    uint32_t inode;      /* inode number (0 = empty / deleted slot)          */
} __attribute__((packed)) dufs_dirent_t;
```

The 256-byte name field matches Linux's `NAME_MAX = 255`: 255 characters plus a NUL terminator. Fifteen entries pack into one 4096-byte block ($15 \times 260 = 3900$ bytes; 196 bytes unused per block). The inode number links the name to all the file's metadata.

Inode 0 and Inode 1

Inode 0 is permanently reserved and never allocated to any file or directory. Its bitmap bit is set at image-build time and the allocation scan starts at inode 2, so inode 0 is invisible to the rest of the code. Using 0 as the "no inode" sentinel in directory entries is safe precisely because inode 0 is never valid.

Inode 1 is always the **root directory** — the directory that every path without a leading slash is resolved against. This convention matches the classical UNIX design: Linux's `ext2`, `ext3`, and `ext4` all fix inode 2 as the root (inode 1 is reserved for a different purpose in those filesystems; DUFS uses 1 directly). The root directory's inode is written by the image-building tool at build time; we read it on mount to locate the root's data blocks.

Initialisation

When the filesystem is mounted at boot, the driver opens the disk, reads the superblock, and validates a magic number embedded in it. A mismatch means the disk either has no DUFS filesystem or is corrupted, and the mount fails immediately.

If the magic is correct, the inode bitmap and the block bitmap are both read into kernel memory, where they remain for the rest of the kernel's lifetime. Keeping them in RAM means every allocation or lookup can check or update a bitmap with a simple in-memory bit operation rather than a disk read.

We keep no in-memory cache of inodes or directory contents. Every directory lookup, every read, and every write goes through the block device for its I/O.

Block I/O

All data I/O in DUFS v3 operates on 4096-byte blocks. Because the ATA driver transfers exactly one 512-byte sector at a time, every block-level read or write internally calls `read_sector` or `write_sector` eight times in sequence:

Chapter 13, Figure 3

Inode table I/O is the exception: inodes are 128 bytes with four per sector, so inode reads and writes still use single-sector calls. This keeps inode I/O efficient — updating one inode costs one sector read and one sector write, not eight of each.

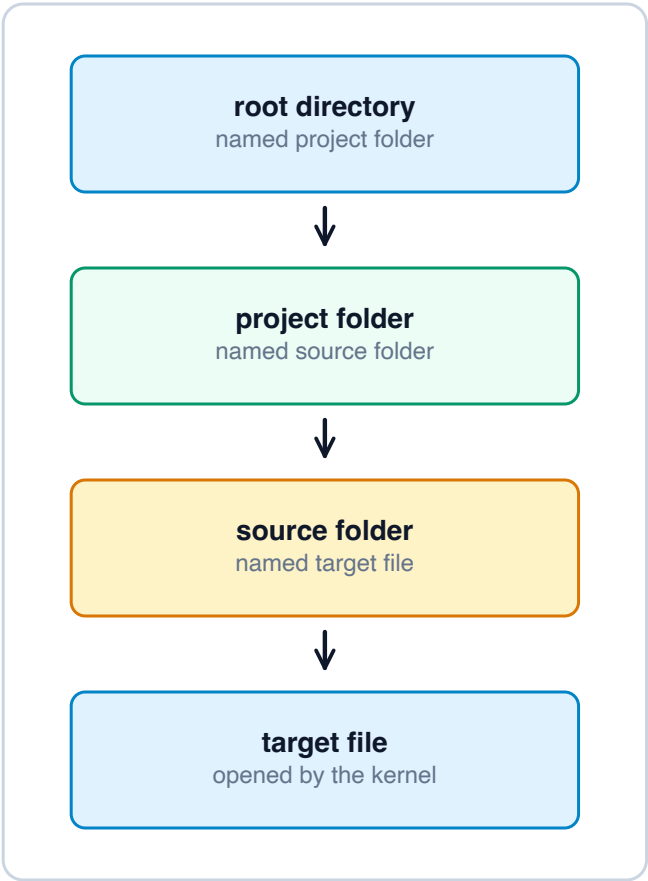
Path Resolution

DUFS supports paths of arbitrary depth. A bare name like `"shell"` names a root-level file. A slash-separated path like `"projects/src/main.c"` names a file nested three levels deep. A leading `/` is ignored — `"/a/b"` and `"a/b"` are equivalent.

DUFS resolves a path with an iterative component walk modelled after Linux's path-walk logic. The important split is the same in both systems: one layer walks components from left to right, and the filesystem-specific lookup step only has to answer "inside this directory, which inode goes with this name?"

The walk begins at inode 1 (root) and processes each `/`-separated component left to right. For each intermediate component, the filesystem looks that name up in the current directory, reads the resulting inode to confirm it is itself a directory, and then advances into it. When there are no more slashes, the remaining string becomes the **leaf name** and the last directory reached becomes the **containing directory inode**. The leaf itself is not looked up yet, because the caller may be creating it rather than opening it.

The walk for a path like `projects/src/main.c` looks like this:

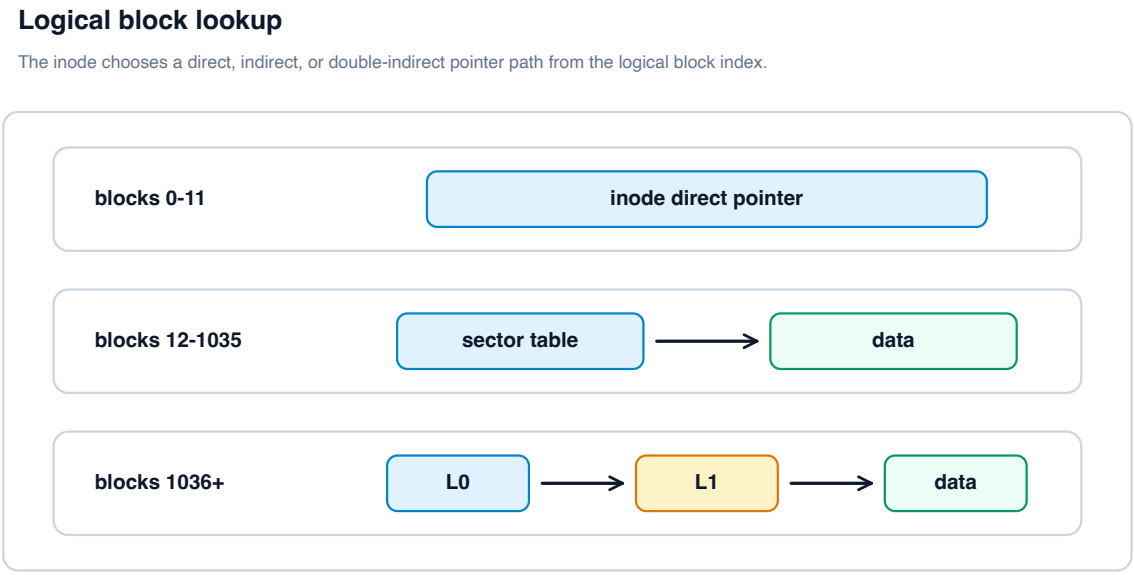


Chapter 13, Figure 4

Reading File Data

Reading from a file maps a byte range within that file to a sequence of block reads. The filesystem reads the inode first to get the file size; if `offset >= size`, the read returns 0 — **EOF** (End of File, the condition that no further data exists beyond this point). Otherwise the requested range is capped at `size - offset` bytes.

The byte range is then walked block by block. For each block, the filesystem computes a **logical block index**: `block_idx = cur_offset / 4096`. It translates that logical index to a physical LBA by examining the direct, indirect, and double-indirect pointers in the inode, reads the 4096-byte block from disk, copies the relevant bytes into `buf`, and advances the cursor.



Chapter 13, Figure 5

Writing File Data

Writing extends a file, allocating new data blocks on demand. For each block in the affected range, the filesystem first ensures that a real block exists. If the relevant direct or indirect pointer is still zero, it finds a free block in the block bitmap, marks it allocated, zeroes it on disk, and stores the new LBA in the inode. For writes that land in the indirect or double-indirect range, the intermediate table blocks are allocated first.

The block bitmap is held in memory during the write. The allocation bits are updated immediately there, but the bitmap itself is flushed to disk only once at the end of the whole write, along with the updated inode. That keeps a large multi-block write from re-writing the bitmap on every single block allocation.

Creating and Deleting Files

Creating a file starts by walking the path to its parent directory. If an entry with the same name already exists and is a file, the existing inode's data blocks are freed, its `mtime` is refreshed from the kernel wall clock, and the same inode number is returned — a truncation. If no entry exists, the filesystem allocates a fresh inode, stamps it with the current Unix timestamp, and inserts a new `(leaf_name, inode_number)` record into the parent directory.

Adding an entry to a directory scans the parent's existing data blocks for an empty slot. If one is found, it is filled and the parent's modification timestamp is updated. If every existing block is full, a new block is allocated for the directory and the entry goes there.

Removing a file resolves the path, reads the file's inode, and decrements `link_count`. When that count reaches zero, the filesystem frees every data block the inode owns. For double-indirect files this means walking the whole pointer tree: the data blocks themselves, the L1 blocks that point at them, and finally the top-level L0 block.

Creating and removing directories follows the same path-walk logic. A new directory gets a fresh inode marked as a directory and is inserted into its parent, but it owns no data blocks until the first directory entry is added to it. Removing a directory is allowed only when it is empty.

Block and Inode Bitmaps

Both bitmaps live in memory as 4096-byte arrays. Bit `i` of the inode bitmap corresponds to inode `i`. Bit `i` of the block bitmap corresponds to the data block starting at `LBA = data_lba + i × 8`.

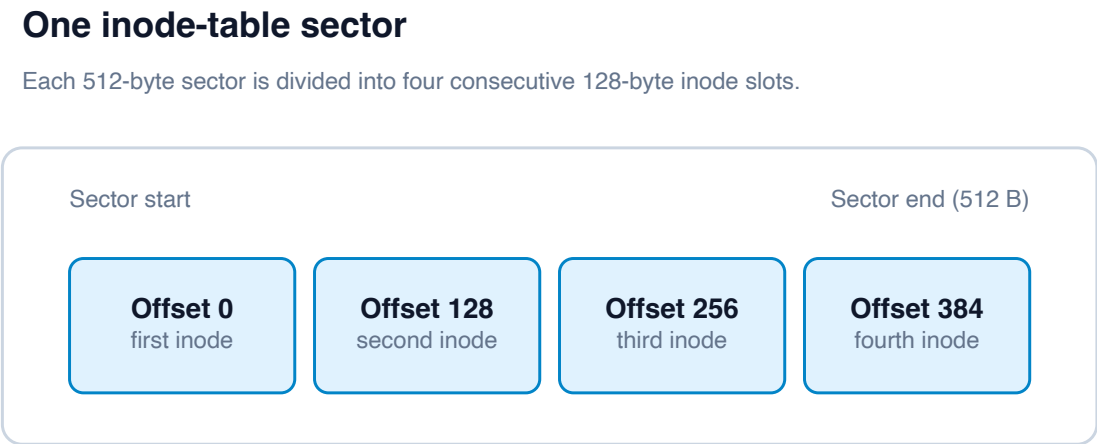
Block allocation scans from bit 0 and returns the first free 4096-byte data block. Block freeing performs the inverse mapping from LBA back to bitmap bit and clears that bit.

Flushing a bitmap to disk writes all eight sectors of the bitmap block in sequence.

The Inode Table on Disk

The inode table spans LBAs 18 through 273 (256 sectors = 1024 inodes × 128 bytes). Each sector holds four 128-byte inodes at offsets 0, 128, 256, and 384.

Reading inode `n` requires computing the sector: `inode_table_lba + n / 4`, and the slot within that sector: `n % 4`. `inode_read(n, out)` performs one sector read and copies the appropriate 128-byte slice. `inode_write(n, in)` performs a read-modify-write: it reads the sector, replaces the target slot, and writes the sector back.



Chapter 13, Figure 6

Building the Disk Image

The image-building tool constructs a valid DUFS v3 image from a list of source files and their destination paths. It runs at build time.

Destination paths may contain `/` at any depth: `"usr/bin/hello"` causes the script to create both `usr/` and `usr/bin/` automatically. The directory tree is built by splitting each destination path on `/`, creating every intermediate component. Inode numbers are assigned in breadth-first order — parents always receive lower inode numbers than their children.

Initial image timestamps are populated at build time. File inodes receive the host file's modification time, while directory inodes receive the image build time. Runtime changes use the kernel wall clock instead.

Chapter 13, Figure 7

The VFS Interface

DUFS v3 registers itself with the **VFS** (Virtual File System, the abstraction layer that lets multiple filesystem implementations coexist behind one interface) through an ops-table:

Operation	DUFS v3 function	Meaning
`init`	`fs_init`	Mount the filesystem and load its on-disk metadata
`open`	`fs_open`	Resolve a path to an inode number and file size
`getdents`	`fs_list`	Enumerate the entries in a directory
`create`	`fs_create`	Create or truncate a file and return its inode number

Operation	DUFS v3 function	Meaning
`unlink`	`fs_unlink`	Remove a file name and free the inode when its link count reaches zero
`mkdir`	`fs_mkdir`	Create a new directory inode and attach it to its parent
`rmdir`	`fs_rmdir`	Remove an empty directory
`rename`	`fs_rename`	Move or rename a directory entry
`stat`	`fs_stat`	Return file metadata such as type, size, link count, and timestamps

All layers above the VFS identify files by inode number rather than by LBA. `file_handle_t` in the process descriptor stores `inode_num`; the read and write syscalls pass that number directly to `fs_read` and `fs_write`.

`fs_flush_inode(inode_num)` is a no-op in DUFS v3. The inode is always written back at the end of every `fs_write` call, so `sys_close` has nothing extra to flush.

Where the Machine Is by the End of Chapter 13

The DUFS v3 image is now mounted and the filesystem layer is fully operational. We have read the superblock from LBA 1, verified the magic number, and loaded both 4096-byte bitmaps into memory. The root directory inode (inode 1) is our anchor for every subsequent path resolution.

From here, any filename — whether a bare name in the root or a slash-separated path of arbitrary depth — resolves to an inode number through the filesystem's path walk. That inode number is our stable handle for the file: it outlives renames (the inode number never changes when a directory entry is rewritten), and two names pointing at the same inode number coexist correctly because the inode tracks a `link_count` rather than a single owning entry.

File data is stored in 4096-byte blocks, addressed by up to twelve direct pointers plus single-indirect and double-indirect chains. A small file lives entirely in direct blocks; a large file extends transparently through indirect addressing without any change to the callers above.

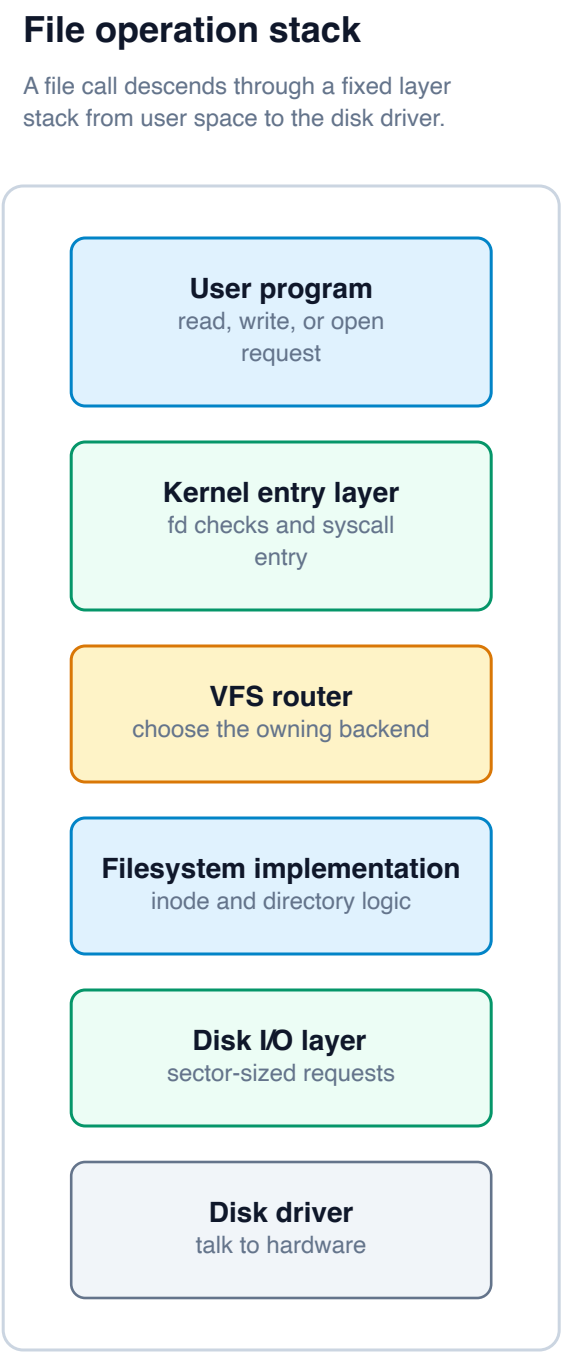
The inode number has replaced the raw LBA as our handle for an open file. Every layer from the ELF loader through the process open-file table and the read/write syscalls now speaks in inode numbers, with DUFS v3 handling the translation to physical disk sectors internally.

Chapter 14 — The Virtual Filesystem Switch

Why the Kernel Uses a VFS Layer

Chapter 13 left us with DUFS v3 fully operational and an inode number standing in as our stable handle for any open file. The code that calls filesystem operations does not call DUFS functions directly. Instead it talks to a narrow indirection layer. That keeps the ELF loader, `start_kernel`, and the rest of the kernel decoupled from DUFS-specific details, and it allows the namespace to be assembled from more than one backend.

The call stack for any file operation flows downward through a fixed set of layers. Each layer knows about exactly the one below it:



Chapter 14, Figure 1

We currently mount DUFS at `/`, a synthetic device filesystem at `/dev`, and a synthetic process-information tree at `/proc`. Path resolution therefore walks a small mount tree rooted at `/`, always choosing the deepest mounted prefix.

The Ops-Table

The VFS interface is defined as:

```
typedef struct {
    uint32_t type;
    uint32_t size;
    uint32_t link_count;
    uint32_t mtime;
} vfs_stat_t;

typedef struct {
    int (*init)(void);
    int (*open)(const char *path, uint32_t *inode_out, uint32_t *size_out);
    int (*getdents)(const char *path, char *buf, uint32_t bufsz);
    int (*create)(const char *path);
    int (*unlink)(const char *path);
    int (*mkdir)(const char *name);
    int (*rmdir)(const char *name);
    int (*rename)(const char *oldpath, const char *newpath);
    int (*stat)(const char *path, vfs_stat_t *st);
} fs_ops_t;
```

The `mtime` field is a filesystem-provided Unix timestamp in UTC seconds. DUFS fills it from the kernel wall clock.

The important point is that the VFS traffic is inode-oriented, not sector-oriented:

- `open` returns an inode number and a byte size.
- `create` returns the inode number of the created or truncated file.
- `stat` reports inode metadata.
- The VFS never exposes a starting LBA to its callers.

That matches the current DUFS implementation in Chapter 13.

Registration and Mounting

The registry is still a tiny fixed array, but the namespace now has two separate tables:

```
#define VFS_MAX_FS     4
#define VFS_MAX_MOUNTS 8
```

Registering a backend copies the filesystem name into the first free registry slot and stores the ops-table pointer beside it. Mounting then resolves the mount's parent directory through the existing namespace, looks up the backend by name, runs its `init()` hook, and records the resulting mount point in the mount table.

Two backends are special: `"devfs"` and `"procfs"` are synthetic and are handled directly by the VFS layer rather than through registered ops-tables. `devfs` contributes nodes like `stdin` and `tty0` under `/dev`; `procfs` synthesises live process and module state under `/proc`.

At boot the sequence is:

1. register DUFS as an available backend,
2. enable interrupts so ATA I/O is safe,
3. mount DUFS at `/`,
4. mount `devfs` at `/dev`,
5. mount `procfs` at `/proc`.

DUFS therefore becomes the root mount only after interrupts are live and ATA reads can complete safely. The synthetic `/dev` and `/proc` subtrees are then layered on top of that root namespace.

The Public API

The public API is deliberately small:

- `vfs_resolve(path, &node)`
- `vfs_open(path, &inode, &size)`
- `vfs_getdents(path, buf, bufsz)`
- `vfs_create(path)`
- `vfs_unlink(path)`
- `vfs_mkdir(name)`
- `vfs_rmdir(name)`
- `vfs_rename(oldpath, newpath)`
- `vfs_stat(path, st)`
- `vfs_mount(path, name)`

Path resolution is the core service: it normalises the path, finds the deepest matching mount, and reports whether the result is a regular file, a directory, a character device, a TTY node, or a synthetic procfs file. `SYS_OPEN` uses that richer result to install either a DUFS file descriptor, a device-backed one, or a synthetic procfs-backed one.

Mutation helpers first identify the mount that owns the target path. They reject mount points themselves as mutation targets, and renames reject cross-mount moves. Optional operations such as `mkdir`, `rmdir`, `rename`, and `stat` are still checked for `NULL` before the function pointer is called.

DUFS Through the VFS

The DUFS registration installs this table:

```
static const fs_ops_t dufs_ops = {
    .init     = fs_init,
    .open     = fs_open,
    .getdents = fs_list,
    .create   = fs_create,
    .unlink   = fs_unlink,
    .mkdir    = fs_mkdir,
    .rmdir    = fs_rmdir,
    .rename   = fs_rename,
    .stat     = fs_stat,
};
```

So the live kernel supports more than lookup and directory listing. Through VFS, user space can

create files, delete them, create and remove directories, rename or move entries within a mount, and fetch metadata.

Directory enumeration is where the mount tree shows most clearly. `vfs_getdents` asks the owning backend for the directory's entries, filters out any names shadowed by child mounts, and then appends those child mount points as synthetic directory entries. Listing `/` therefore shows `dev/` and `proc/` even though those names come from the mount table rather than from DUFS itself.

`procfs` also shows why the VFS returns richer node types than just "file or directory". `/proc/<pid>/status`, `/proc/<pid>/vmstat`, `/proc/<pid>/fault`, `/proc/<pid>/maps`, `/proc/<pid>/fd/<n>`, `/proc/modules`, and `/proc/kmsg` are openable files, but they do not have DUFS inode numbers. The syscall layer therefore treats them as synthetic read-only files whose contents are rendered on demand from scheduler, page-table, file-descriptor, module-loader, and kernel-log state.

At the top level, `/proc` enumerates one numeric directory per live PID plus the synthetic `modules` and `kmsg` files. Each `/proc/<pid>/` directory contains `status`, `vmstat`, `fault`, `maps`, and an `fd/` subdirectory. `vmstat` and `fault` provide compact summaries, while `maps` remains the detailed virtual-memory layout view; all three are generated from the same process-memory forensics model, so they stay internally consistent. The VFS does not cache any of that output as on-disk metadata; every open, stat, getdents, and read call asks `procfs` to re-render the current kernel state. That is why `/proc/<pid>/fd/` naturally reflects descriptor duplication, pipes, TTY bindings, and even previously opened procfs files without any separate update path, and why `/proc/kmsg` always reflects the kernel's current retained log buffer rather than a stale snapshot written to disk.

What the VFS Does Not Do

This VFS is intentionally much smaller than Linux's:

- no per-file descriptor objects inside the VFS
- no **dentry** (directory entry cache — Linux's in-memory index of recently looked-up path components) or inode cache beyond whatever the concrete filesystem provides
- no unmount operation
- no symlinks, bind mounts, or mount options
- no cross-mount rename or move

It is still enough to keep the rest of the kernel from depending on DUFS's on-disk layout.

Where the Machine Is by the End of Chapter 14

Our file-facing code is now structured around a stable interface instead of a concrete filesystem implementation. `start_kernel`, the ELF loader, and the module loader all talk to `vfs_*` helpers and receive inode-oriented or node-oriented answers. DUFS provides the root filesystem at `/`, while the VFS layer itself synthesises `/dev` and `/proc`; all three are reached through the same path-resolution machinery rather than through backend-specific call sites.

Part V — Running User Programs

Everything built so far runs in the kernel. The interrupt handlers, the memory allocator, the filesystem, the ATA driver — all of it executes at ring 0 with unrestricted access to every memory address and every I/O port. That is appropriate for the kernel itself, but it is not acceptable for programs written by users. A bug in a user program must not be able to corrupt the kernel's data structures, overwrite another process's memory, or halt the machine. The CPU's privilege system exists precisely to enforce this boundary, but enforcing it requires the kernel to build the infrastructure to cross it safely.

Running a user program is not a single operation — it is a pipeline of decisions and data structures that must all be in place before the CPU can execute the first instruction of user code. The kernel must parse the ELF binary (the standard executable file format on Linux and Unix systems) from disk, allocate a private page directory that maps the program's code and data at the virtual addresses the linker chose, build a user-mode stack with the program's arguments, configure segment descriptors for user-mode code, update the Task State Segment — a hardware structure the CPU reads to find the kernel stack when switching from user code to kernel code — and finally issue a return-from-interrupt instruction with the right register values to drop from ring 0 (kernel privilege) to ring 3 (user privilege). Once that program is running, every request it makes — writing output, opening a file, creating a child process — must travel back through a controlled entry point into the kernel and out again.

Chapter 15 builds the full process model: the ELF loader, per-process page directories, the `process_t` descriptor, the round-robin preemptive scheduler driven by the timer tick established in Part I, and `fork` which clones a running process completely. Chapter 16 establishes the system call interface — the software interrupt entry point, the calling convention, the blocking-state model for calls that must wait for external events, and the complete table of syscalls the kernel exposes. Chapter 17 builds the file descriptor layer on top of the Virtual File System: the per-process open-file table, the unified read and write path that handles both terminal and file I/O, pipes, `dup2`, and seek.

By the end of Part V the kernel is a true operating system. It can load an ELF binary from disk, run it in a private address space at ring 3, preempt it on a timer tick, schedule multiple processes concurrently, and serve their requests through a stable system call interface. The next part will give those processes a way to interact with the user.

Chapter 15 — ELF Loader and Process Management

Why the Kernel Needs to Separate Itself From User Code

Chapter 14 left us with a VFS layer in place and the kernel able to open and read files by name through a stable interface. Everything described in the book so far runs at **ring 0** — the CPU's most privileged execution level, where any instruction is legal and any memory address can be accessed. A bug in ring-0 code can corrupt anything in the system, because nothing is protected from it. That is acceptable while the kernel is the only code running, but the moment you want to run a user program, the rules have to change.

A real operating system runs user programs at **ring 3** — the least privileged execution level on x86. Ring 3 code cannot talk to hardware directly, cannot read or write kernel memory, cannot modify control registers, and cannot disable interrupts. Any attempt to do any of those things raises a fault that the kernel catches. This boundary — between the kernel in ring 0 and the user in ring 3 — is enforced by hardware on every single instruction and every single memory access, not by software checks the kernel has to remember to make.

This chapter adds everything required to cross that boundary. By the end, we will read an **ELF** file (Executable and Linkable Format, the standard binary format on Linux and most Unix systems) off the disk, unpack its contents into a fresh private address space, and hand control to ring 3 with a single CPU instruction called `iret`. From there, the user program communicates with the kernel exclusively through **system calls** triggered by `int 0x80` — the same interface the 32-bit Linux kernel used for decades.

What the CPU Requires Before It Will Enter Ring 3

Four hardware structures must be in place before the `iret` instruction can legally transfer from ring 0 to ring 3.

First, the GDT must contain ring-3 segment descriptors. The GDT set up in Chapter 2 had only kernel code and kernel data segments at DPL 0 (Descriptor Privilege Level 0 — accessible only by ring 0). The kernel now rebuilds the GDT in C code, expanding it from three entries to seven:

Index	Offset	Selector	DPL	Type	Purpose
0	`0x00`	`0x00`	—	Null	Required trap entry
1	`0x08`	`0x08`	0	Code	Kernel code segment
2	`0x10`	`0x10`	0	Data	Kernel data & stack segment
3	`0x18`	`0x1B`	3	Code	User code segment
4	`0x20`	`0x23`	3	Data	User data & stack segment
5	`0x28`	`0x28`	0	TSS	Runtime task state segment

Index	Offset	Selector	DPL	Type	Purpose
6	`0x30`	`0x30`	0	TSS	Dedicated double-fault task state segment

The selector value has the requested privilege level encoded in its low two bits, so a user selector is not the same number as the offset: `0x18 | 3 = 0x1B` and `0x20 | 3 = 0x23`. When the CPU sees a selector with DPL 3 loaded into `CS`, it knows the code running under that selector is ring-3 code.

Second, the kernel has to install a Task State Segment. The **TSS** is a 104-byte structure whose main job is to tell the CPU which kernel stack to switch to when an interrupt fires while ring-3 code is running. Ring-3 code has its own stack (the user stack), and the kernel cannot safely run on that stack — a user program could have corrupted it, and the kernel handler would corrupt itself trying to save state. So the CPU reads two fields from the runtime TSS, `ESP0` and `SS0`, which contain the kernel stack pointer and kernel stack segment, and it switches to that stack automatically before pushing the interrupt frame. Without a valid TSS, the first `int 0x80` from ring 3 would fail with a general protection fault.

The runtime TSS is installed as a **system segment** in the GDT at index 5 (selector `0x28`), and the CPU is told about it with the `ltr` ("load task register") instruction.

The current kernel also keeps a second TSS in the GDT specifically for `#DF` (Double Fault). `TSS.ESP0` only helps when the CPU is crossing from ring 3 into ring 0. It does **not** help if the CPU is already in ring 0 and the current kernel stack is bad. That is exactly the situation a double fault usually represents. To survive long enough to print diagnostics, the kernel installs IDT vector 8 as a task gate to a dedicated double-fault TSS whose `EIP` points at a panic routine and whose `ESP` points at a fixed emergency stack. When `#DF` occurs, the CPU performs a hardware task switch into that TSS and the panic code can still run even if the ordinary kernel stack is unusable.

Third, the INT 0x80 gate must be marked as callable from ring 3. Every IDT entry has a privilege level, and the CPU refuses to let user code trigger a software interrupt whose gate is more privileged than the caller. For IDT entry 128, we use the `type_attr` byte `0xEF`, whose bit pattern means: present, DPL 3 (ring-3 code is allowed to invoke this), 32-bit trap gate. All other IDT entries use `0x8E` (DPL 0, interrupt gate). The difference between a trap gate and an interrupt gate is that a trap gate leaves hardware interrupts enabled while the handler runs — essential so the keyboard can continue to fire its IRQ while we are inside a `SYS_READ` call.

Fourth, each process needs its own page directory. The kernel cannot use a single shared page directory because user code must not be able to read or write kernel memory. Each process gets a fresh page directory whose entries cover the kernel's 0–128 MB region *without* the `PG_USER` flag set, and whose user-segment pages (mapped starting at `0x400000`) are set with `PG_USER | PG_WRITABLE | PG_PRESENT`. When a ring-3 instruction tries to reach a kernel address, the missing `PG_USER` bit causes a page fault; when ring-0 kernel code runs in the same address space, it ignores `PG_USER` and can reach anything it needs.

Rebuilding the GDT in C

`gdt_init` builds a seven-entry GDT in static memory and hands it to a tiny assembly routine called `gdt_flush`. The assembly code is necessary because reloading segment registers involves a sequence that cannot be expressed in C without stepping on itself: it has to execute `lgdt`, then write the new kernel data selector into `DS`, `ES`, `FS`, `GS`, and `SS`, then perform a **far jump** to reload `CS`. A plain C assignment to `CS` is impossible on x86 — the only way to change it is through a jump, call, or return that specifies a segment. After the far jump lands, the assembly routine also executes `ltr` with the runtime TSS selector to register that TSS with the CPU.

One subtle detail: all parameters must be saved into registers *before* the segment reloads begin. Reloading `SS` in the middle of a `push/pop` sequence would leave a brief window where `SS` and `ESP` disagreed about which stack was active, and a stray interrupt in that window would crash the system.

Per-Process Page Directories

The paging module from Chapter 8 is extended with two new functions.

`paging_create_user_space` allocates a fresh 4 KB page directory from the physical memory manager and copies the kernel's thirty-two page directory entries into it (those entries cover the kernel's 0–128 MB identity mapping). Because the kernel is identity-mapped, the physical address of the new page directory is also its virtual address, and we can write to it directly without switching `CR3`.

`paging_map_page` installs a single page table entry in an arbitrary page directory. If there is no page table yet for the target virtual address, it allocates one and marks the page directory entry with `PG_USER` so the CPU will descend into it on ring-3 lookups. If the existing page table was inherited from the kernel (that is, shared between processes), the function makes a private copy first — a crucial point, because otherwise a second process would overwrite the first process's user mappings in the shared kernel table and crash the first process the next time it ran.

The ELF Loader

An ELF file begins with a header that describes the layout of the file. The header contains, among other things, a pointer to a table of **program headers**, each of which describes a region of memory the loader must map before jumping to the entry point. For our purposes, only one type of program header matters: **`PT_LOAD`** (a Loadable segment — one that the ELF loader must copy into memory at a specific virtual address before execution begins).

`elf_load` reads the binary from the ATA disk one 512-byte sector at a time into a static buffer. For each `PT_LOAD` segment it performs five steps:

1. Round the virtual address range up to 4 KB page boundaries, since every mapping must be page-aligned.
2. Ask the physical memory manager for one physical page per virtual page the segment covers.
3. Map each virtual page into the process's page directory with the flags `PG_PRESENT | PG_WRITABLE | PG_USER`.
4. Zero every byte in every page — this handles the common case where the segment's in-memory size (`p_memsz`) is larger than its on-disk size (`p_filesz`), which happens when the segment contains a BSS region.

5. Copy the segment's on-disk bytes into the newly allocated physical pages. Because the kernel is identity-mapped, we write directly to the physical page through its identity-mapped kernel address without switching `CR3`.

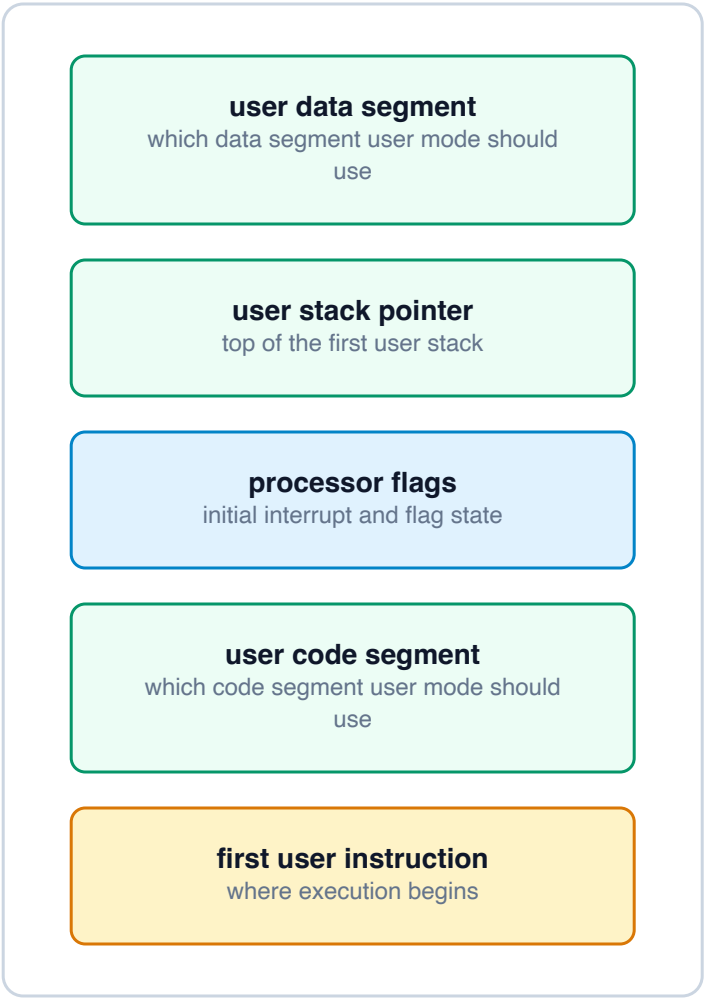
After every `PT_LOAD` segment has been processed, the loader records three addresses: the ELF header's entry point, the lowest mapped `PT_LOAD` address, and the page-rounded end of the highest mapped segment. The first is where ring 3 begins execution. The second and third let the kernel remember the loaded image range explicitly; the page-rounded high end also becomes the initial `heap_start` and `brk`, and the image range is what `/proc/<pid>/maps` later labels as the executable image.

The Moment the CPU Enters Ring 3

The final step is an `iret` instruction, which in this context does something it was never originally designed for: it performs a controlled drop from ring 0 to ring 3. The trick is that `iret` always pops a frame from the stack containing `EIP`, `CS`, and `EFLAGS`, and *when the popped `CS` has a different privilege level from the current one*, it also pops `ESP` and `SS`. We exploit this by manually constructing a fake interrupt frame on the stack that looks exactly like what the CPU would have pushed if it had interrupted a ring-3 program, and then executing `iret` to "return" to that fake state.

The assembly helper `process_enter_usermode` builds the frame and executes the `iret`. The frame it pushes contains, from bottom to top: the user data selector `0x23` (as `SS`), the user stack pointer the new process should start with, the flags register with the interrupt enable bit set so the user program will receive interrupts, the user code selector `0x1B` (as `CS`), and the ELF entry point (as `EIP`).

The stack just before the `iret` executes looks like this:



Chapter 15, Figure 1

When `iret` executes, the CPU atomically:

- Switches the current privilege level from 0 to 3.
- Loads the new `CS` and `EIP`, so the next instruction fetched is the user program's first instruction.
- Pops the new `ESP` and `SS`, so the user program has its own stack.

The data segment registers (`DS`, `ES`, `FS`, `GS`) are also set to `0x23` before the `iret`, because ring-3 code needs consistent segment selectors and `iret` does not touch those four.

System Calls: The Only Way Back Into the Kernel

Once the CPU is in ring 3, the only way the user program can reach the kernel is by triggering an `int 0x80`. The IDT entry for vector 128 points at an assembly stub that saves all registers, loads the kernel data segment, and branches into the syscall dispatcher.

The handler reads the saved frame to find the syscall number (in `EAX`) and up to three arguments (in `EBX`, `ECX`, and `EDX`) and dispatches to the right handler. The trampoline runs with the **process's page directory still active** in `CR3`, which means we can follow pointers the user program passed without switching address spaces — the user's pages are still mapped for ring-0 access because the kernel PDEs were copied without `PG_USER` but ring-0 ignores that flag.

The C handler returns a `uint32_t`. The trampoline takes that return value and writes it into the saved `EAX` slot on the kernel stack before executing `popa`, so when the `iret` at the end of the stub returns to the user program, the return value appears in user-visible `EAX` without any extra work on the caller's part.

The relevant syscalls at this stage are `SYS_EXIT` (terminate a process), `SYS_WRITE` (print a string), `SYS_READ` (block until a keystroke arrives), `SYS_EXEC` (replace the current process image with another program), and `SYS_WAIT` (block until a child exits).

Waiting for a Child to Exit

Without a wait primitive, the shell's `fork()` + `SYS_EXEC` launch path would leave the parent shell runnable while the child was still printing, producing interleaved output. `SYS_WAIT` fixes this.

When the shell calls `SYS_WAIT` with the child's PID, the kernel blocks the shell on that specific child's embedded `state_waiters` queue. The shell's process state becomes `PROC_BLOCKED`, and the process descriptor records both the queue it is sleeping on and the intrusive `wait_next` link needed to sit in that queue. When the child later exits or stops, the scheduler wakes every waiter on that child's queue, and the shell resumes from `sched_waitpid` to re-check the child's state. This still mirrors Linux's `TASK_INTERRUPTIBLE`/`wake_up` pattern, but the wait channel is now a reusable queue object rather than a handwritten `wait_pid` field.

Preemptive Round-Robin Scheduling

With the ring-3 machinery in place the next step is running **multiple processes concurrently**. The **scheduler** uses a hardware timer to interrupt the running process every ten milliseconds and hand the CPU to the next one waiting.

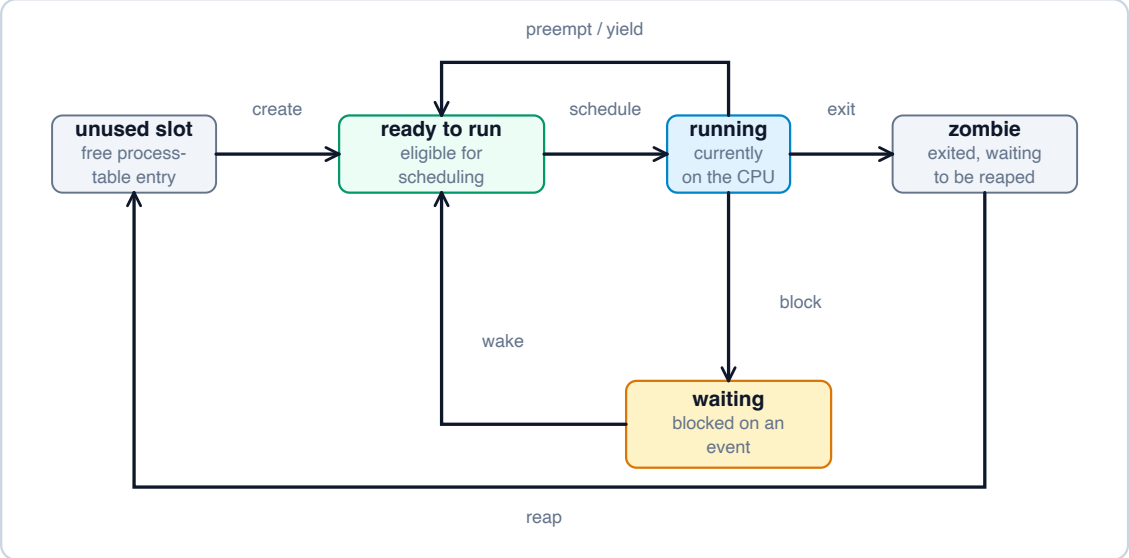
The `process_t` struct gains several new fields to support this:

```c
typedef struct __attribute__((aligned(16))) {
    uint32_t pd_phys;
    uint32_t entry;
    uint32_t user_stack;
    uint32_t kstack_bottom;
    uint32_t kstack_top;
    uint32_t saved_esp;     /* kernel ESP captured at last preemption */
    uint32_t pid;
    uint32_t state;         /* proc_state_t — one of the values below */
    wait_queue_t *wait_queue;
    process_t    *wait_next;
    uint32_t wait_deadline;
    uint32_t wait_deadline_set;
    wait_queue_t state_waiters;
    uint8_t  fpu_state[512];
} process_t;
```

We keep a static `proc_table` of eight entries and a pointer `g_current` to the currently running one. Every process moves through a state machine as syscalls and interrupts fire:

Process state machine

Processes move between ready, running, blocked, zombie, and unused states as the scheduler and syscalls act on them.



Chapter 15, Figure 2

When `RUNNING` code makes a blocking call or receives a stopping signal, the process now moves through a smaller set of scheduler states:

State	Meaning	Leaves state when
`READY`	Runnable, waiting for a CPU slice	the scheduler selects it
`RUNNING`	Currently executing on the CPU	preempted, blocked, stopped, or exited
`BLOCKED`	Asleep on a wait queue or timed deadline	a waiter is woken, a deadline expires, or a signal interrupts the sleep
`STOPPED`	Suspended by `SIGSTOP` or `SIGTSTP`	`SIGCONT` or `SIGKILL` arrives
`ZOMBIE`	Exited, waiting for a parent to reap status	`waitpid` reaps the slot

`ZOMBIE` is a terminal state: the process has exited but its slot is kept alive so the parent can collect the exit status via `SYS_WAITPID`.

The key change is that the scheduler no longer needs a different enum value for each blocking reason. A pipe embeds one `wait_queue_t`, a TTY embeds another, and every process embeds `state_waiters` for parents waiting on child state changes. `sched_block(&queue)` links the current process into that queue and marks it `PROC_BLOCKED`; `sched_block_until(deadline)` records a wake tick without any queue at all.

When every slot is blocked and nothing is `RUNNING`, the scheduler enters an idle loop — it enables interrupts, executes `hlt`, and retries when any interrupt fires. This is the same idle pattern Linux uses.

Every process has its own kernel stack. When a hardware interrupt fires while ring-3 code is running, the CPU reads `TSS.ESP0` and switches to that address before pushing the interrupt frame. If two processes shared a single kernel stack, saving the first process's frame and then taking an interrupt for the second would overwrite the first process's state. `process_create` therefore allocates a dedicated 16 KB stack for each process, and every context switch updates `TSS.ESP0` to point at the new current process's stack top. The double-fault path is separate: it does not use the current process's `ESP0`, but instead switches into the dedicated emergency TSS described earlier.

The timer is driven by the PIT. The **PIT** (Programmable Interval Timer, specifically the Intel 8253/8254) is a legacy chip found on every x86 PC. Its channel 0 is programmed with a divisor that produces a periodic interrupt on IRQ0 — in this case roughly 100 Hz, or every ten milliseconds. The PIT is configured in `pit_init` using port writes to `0x43` (mode register) and `0x40` (channel 0 data), and the PIC mask is updated so that both IRQ0 (timer) and IRQ1 (keyboard) are enabled.

The wall clock starts from the RTC and advances on PIT ticks. During boot, `clock_init` reads the **RTC** (Real-Time Clock), a battery-backed timekeeping chip housed in the **CMOS** (Complementary Metal-Oxide Semiconductor) chip on the motherboard, through I/O ports `0x70` and `0x71`, waits for a stable sample, converts the RTC date into Unix time, and stores it as UTC seconds since 1970-01-01. Each PIT interrupt calls `clock_tick` before `sched_tick`, so the wall-clock counter advances once every 100 timer ticks even though the RTC is only read at boot.

The timer interrupt is where the handoff happens. When the periodic timer IRQ arrives, the CPU has already saved a register frame on the current process's kernel stack. After the IRQ handler finishes, the scheduler gets one chance to decide whether the current process should continue or whether some other ready process should run next. If it chooses a different process, the kernel switches to that process's saved kernel stack and returns from the interrupt there instead.

The context switch itself does four things in order:

1. The outgoing process's floating-point and SSE register state is saved into its process descriptor.
2. The outgoing process's kernel stack pointer is recorded in its descriptor and the process is marked as ready to run again.
3. The process table is scanned round-robin for the next ready process.
4. The incoming process's page directory, kernel stack, and FPU state are installed, and execution is resumed from that process's saved register frame.

From the CPU's point of view, the final return from interrupt is ordinary. It restores registers from whichever kernel stack is active at that moment, so once the kernel has switched stacks, the rest of the return path naturally resumes the incoming process as if the timer interrupt had always been destined for it.

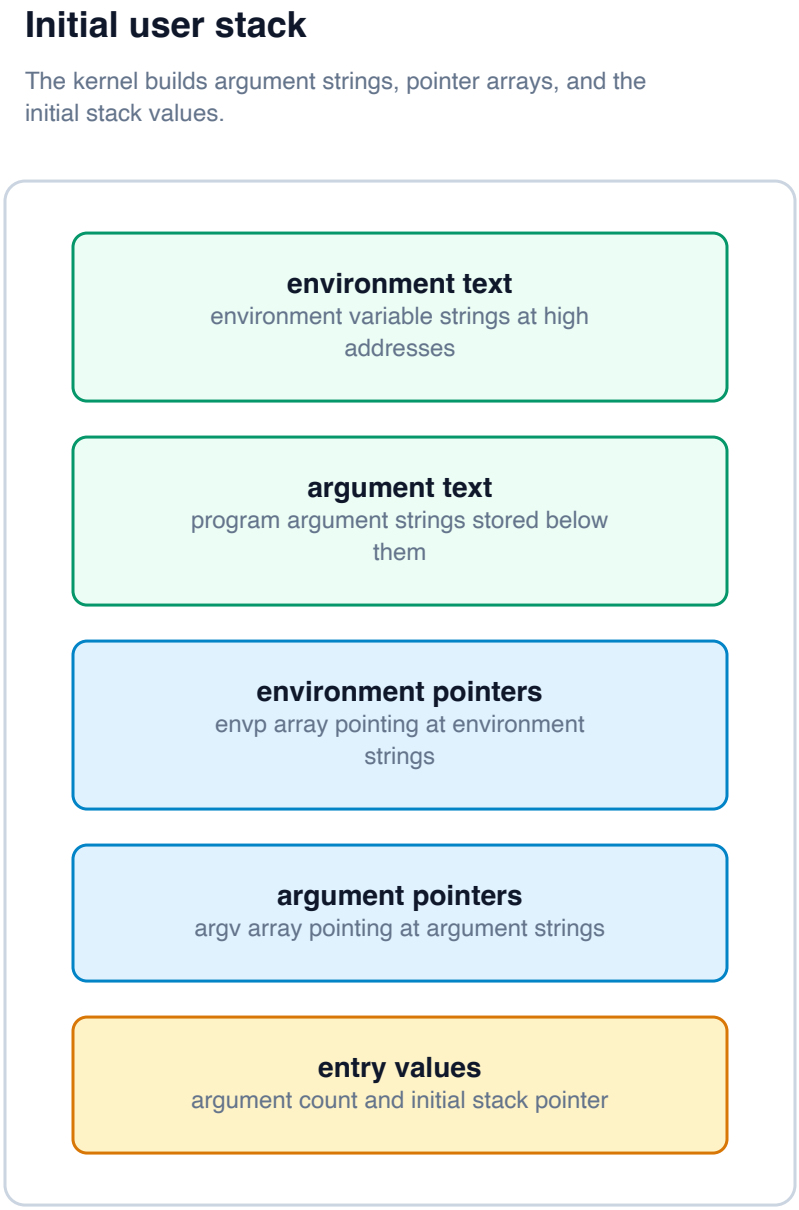
A process running for the first time needs a synthetic frame. A freshly created process has never been preempted, so there is no real saved frame to resume yet. The scheduler manufactures one on the new process's kernel stack: the saved segment registers are set to the user data selector, the general-purpose registers are zeroed, and the `iret` frame is built to drop into ring 3 at the ELF entry point with interrupts enabled. The kernel then records the stack pointer to that synthetic frame as the place where the process should later resume.

A new process enters `main` with `argc`, `argv`, and `envp` already on its stack. When the kernel allocates and maps the four user stack pages, it does not stop at the empty stack top — it lays down a full System V i386 **ABI** (Application Binary Interface, the set of register and stack conventions the compiler assumes at function-call boundaries) argument frame first and hands back the resulting `ESP` as the process's initial stack pointer. That way, by the time the `iret` transfers control to ring 3, the user stack already looks like a conventional Unix process image, including an environment array.

The frame is built from the top of the stack page downward:

Region	Content
Argv+env strings	All `argv` strings followed by all `envp` strings, concatenated and NUL-terminated, written at the top of the page.
0–3 bytes of zero padding	Enough to 4-byte align the pointer arrays below.
`envp[]` pointer array	`envc` user-space pointers into the string region above, followed by a NULL terminator.
`argv[]` pointer array	`argc` user-space pointers into the string region above, followed by a NULL terminator.
`char **envp`	A pointer to the first entry of the envp array.

Region	Content
`char **argv`	A pointer to the first entry of the argv array.
`int argc`	← initial ESP



Chapter 15, Figure 3

Because the page directory and page tables live in the kernel's identity-mapped 0–128 MB region, the kernel can resolve the physical frame backing the top stack page and write the whole initial stack image through that kernel-visible pointer. No **CR3** reload is required, and no temporary mapping has to be torn down. The same path handles `argc == 0` and `envc == 0` gracefully,

writing zero counts and NULL-terminated empty arrays in each case, so the ring-3 startup stub can always pop all three words off the stack without a special case.

To keep the frame inside a single 4 KB stack page, the kernel enforces four caps: at most 32 argv entries, at most 1 024 bytes total across all argv strings, at most 32 environment entries, and at most 1 024 bytes total across all environment strings. The `SYS_EXEC` path copies every argv and envp string into fixed-size kernel scratch buffers before building the new image, because the caller's address space is about to be replaced and any raw user pointers would then reference the wrong pages.

The same context-switch path is used for voluntary yields. Timer-driven preemption is not the only way one process hands the CPU to another. `SYS_YIELD` (voluntary yield), `SYS_WAIT` (park the caller until a child exits), and `SYS_EXIT` (terminate the caller) all leave the current process unable or unwilling to continue. In each case, the kernel saves the same kind of frame, chooses the next `PROC_READY` entry, and resumes somewhere else. The asynchronous timer path and the voluntary handoff path therefore converge on one mechanism.

Process Observability Through `/proc`

Core dumps are still the deep post-mortem debugging tool, but the live kernel now exposes lighter-weight process state through `procfs`. The VFS mounts `procfs` at `/proc`, and the synthetic files under that tree are rendered directly from scheduler, fd-table, memory-management, module-loader, and kernel-log state rather than from DUFS inodes.

At the top level, `/proc` contains one numeric directory per live PID plus the synthetic `modules` and `kmsg` files. For each live PID, `/proc/<pid>/status` reports the fields that are already present in `process_t`: executable name, run state, PID, PPID, process-group and session IDs, controlling TTY, image range, heap range, current mapped stack floor, pending and blocked signals, open-fd count, and the best-effort command line captured in `psargs`.

`/proc/<pid>/maps` walks the process's user page tables, coalesces adjacent present PTEs, and labels the ranges that overlap the loaded ELF image, the heap, and the stack. Because the output is generated from the live page tables, it reflects current committed mappings and current PTE permissions, so permission changes applied with `mprotect()` show up once there is a present page to report. The same implementation detail is also the current limitation: untouched lazy reservations that exist only in the VMA table, such as a never-faulted anonymous `mmap()` region, do not appear until at least one page has actually been materialized. `/proc/<pid>/fd` is a directory whose numbered entries describe the process's open descriptors — DUFS files, pipes, TTYs, VGA stdout, and even synthetic procfs descriptors that were opened earlier — and each `/proc/<pid>/fd/<n>` file renders a one-line description of that fd's current target.

`/proc/kmsg` complements those per-process views with a system-wide diagnostic stream. The `klog` layer now keeps a fixed-size ring buffer of recent kernel log records, each tagged with a boot-relative timestamp, a severity level, a subsystem tag, and the message text. Reading `/proc/kmsg` renders that retained history in order, so the user can recover context that has already scrolled off the VGA console.

That matters for shell tooling because `ls /proc`, `cat /proc/3/status`, `cat /proc/3/maps`, `cat /proc/3/fd/1`, `cat /proc/modules`, and `dmesg` now work through the ordinary `SYS_OPEN` and `SYS_READ` paths. The shell no longer has to infer process state only from blocking behaviour or

wait statuses, and the kernel no longer relies solely on a fatal crash or whatever still happens to be visible on screen to make system state inspectable.

Forking a Process

`SYS_EXEC` replaces the current process image in place by loading a fresh ELF binary. `SYS_FORK` does something fundamentally different: it makes a second process that resumes at the same instruction pointer with the same registers and the same open files. The memory is not copied eagerly any more. Parent and child leave `fork()` sharing the same user frames, and only the first write to a shared writable page triggers a private copy. The only immediate architectural difference between them is the return value of `fork()` itself: the parent receives the child's PID, and the child receives zero.

This is the mechanism Linux programs rely on to spawn new work: fork first, then decide which branch you are in by checking the return value, and have the child call `exec()` if you want it to run a different program. This section describes how we make the split happen.

Cloning the Address Space

The first job of `fork()` is still to give the child its own page directory and its own user page tables. The difference is that the user frames underneath those page tables are now shared initially rather than copied immediately.

To do that, the kernel allocates a new page directory and private user page tables for the child, but it leaves the user frames underneath them shared at first. Writable user pages are rewritten in both address spaces as read-only copy-on-write mappings, and the physical frame's reference count is incremented so the memory manager knows the frame now has two owners. Read-only user pages are also shared into the child, but they stay genuinely read-only so later writes still fault as true protection violations.

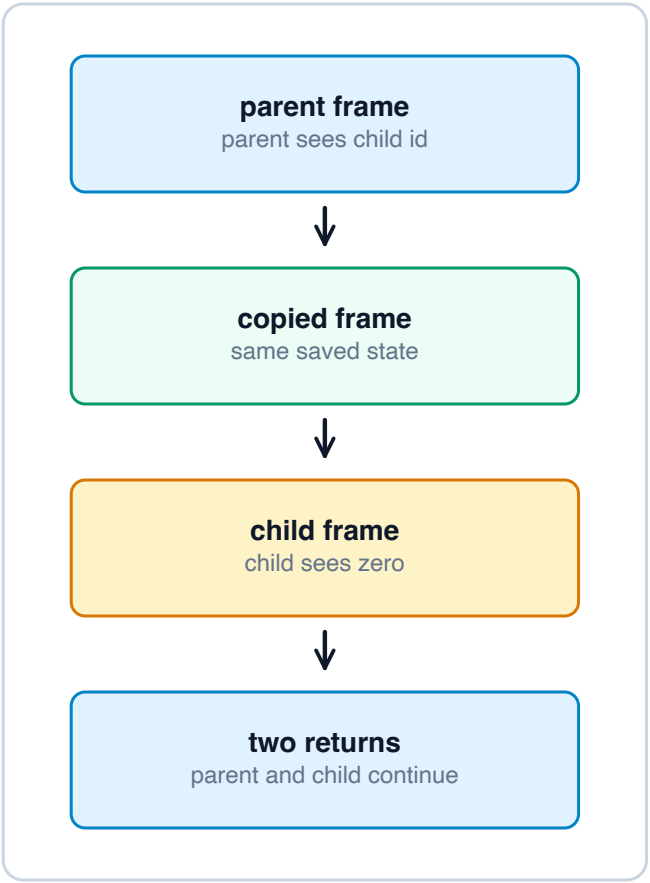
This is the copy-on-write model Linux uses too: the expensive physical copy is deferred until a later write fault proves one side needs a private version. The immediate consequence is that `fork()` can now return quickly without copying every page in the process up front.

Splitting the Return Value

After cloning the address space, the kernel must arrange for the parent and child to resume execution at the same place but see different values in `EAX`. This is the tricky part of fork.

The trick is that the kernel already has exactly the right data structure for this moment: the saved syscall-return frame sitting on the parent's kernel stack. That frame contains the user registers that will be restored when the kernel returns to ring 3, including the return register `EAX`.

Fork works by copying that saved frame onto the child's kernel stack and then editing only one field in the child's copy: the saved `EAX` value is changed to zero.



Chapter 15, Figure 4

The next time the scheduler chooses the child, the normal return-from-kernel path restores that copied frame, so the child resumes at the same user `EIP` as the parent with `EAX = 0`. The parent returns through its own original frame and sees the child's PID in `EAX`. No special second return path is needed; both branches reuse the ordinary syscall-return mechanism.

One additional detail: before the kernel copies the process descriptor, it forces a fresh save of the parent's floating-point and SSE state. Otherwise the child would inherit whatever stale snapshot happened to be left from the last preemption rather than the parent's actual live FPU state.

Where the Machine Is by the End of Chapter 15

We can now:

- Read an ELF file from disk, parse its program headers, and load its segments into a fresh

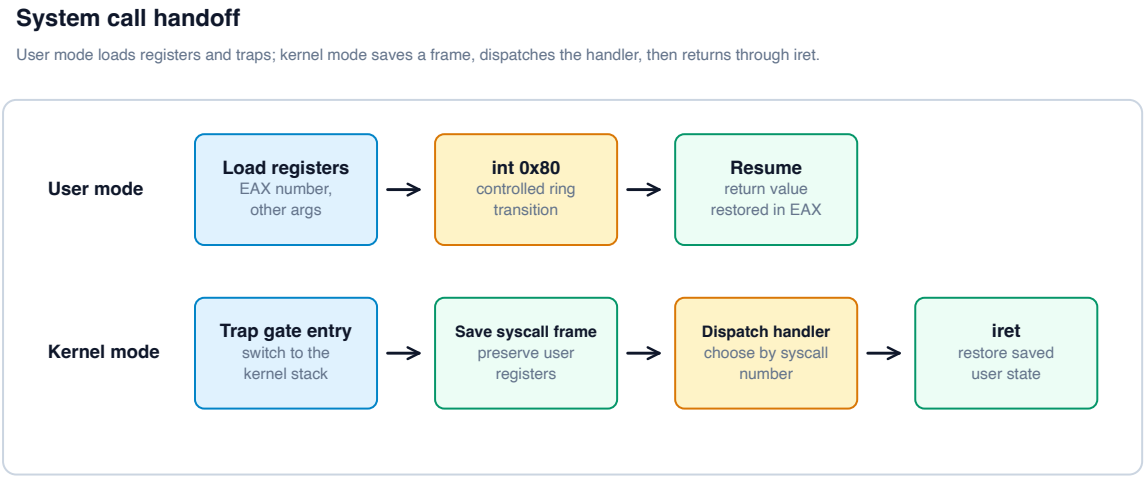
private address space.
- Enter ring 3 via a manufactured `iret` frame, with user-mode segment selectors and a fresh user stack.
- Service system calls through `int 0x80`, reading arguments from the user's registers and writing return values back.
- Preempt running processes every ten milliseconds via the PIT and round-robin schedule up to eight concurrent processes, saving and restoring register state, page directories, kernel stacks, and FPU/SSE state on every switch.
- Wait for a child process to exit with `SYS_WAIT`, parking the caller until the scheduler wakes it.
- Fork a running process via `SYS_FORK`, duplicating its page tables, sharing its user frames copy-on-write, and arranging for the parent and child to see different return values at the same return address.

With this machinery in place, the first user program runs as a ring-3 process, can launch any ELF file on the DUFS disk, and waits for each child to exit before returning to the prompt.

Chapter 16 — System Calls

The Path of a System Call

Chapter 15 left us with a ring-3 process running ELF binaries and a context-switch path driven by the PIT. When a user program needs a service that only the kernel can perform — reading a keystroke, writing a file, spawning a child process — it cannot call the kernel directly. Ring-3 code cannot jump into ring-0 memory, and even if it could, the CPU would fault. The only legal crossing point is a software interrupt: the user program loads a syscall number and arguments into registers, then executes `int 0x80`. The CPU sees that this matches a DPL-3 trap gate in the **IDT** (Interrupt Descriptor Table) and performs a controlled ring transition — switching to the kernel stack, loading the kernel code segment, and jumping to the ISR stub. The stub saves registers, calls `syscall_handler`, writes the return value back into the saved frame, and executes `iret` to return to ring 3.



Chapter 16, Figure 1

The **ABI** (Application Binary Interface — the register and stack conventions both sides agree on) is the classic 32-bit Linux-style register convention: `EAX` holds the syscall number on entry and the return value on exit; `EBX`, `ECX`, `EDX`, `ESI`, and `EDI` hold up to five arguments. The IDT entry for vector 128 uses a trap gate rather than an interrupt gate, so hardware interrupts remain enabled while `syscall_handler` runs — essential for calls like `SYS_READ` that block waiting for keyboard input to arrive.

Blocking Calls and Process States

Most syscalls complete immediately, but several must suspend the calling process until an external event occurs. `SYS_READ` on a TTY blocks until input is available. `SYS_WAIT` and `SYS_WAITPID` park the caller until a child exits or stops. Pipe reads and writes block on the shared pipe wait queue when the buffer is empty or full. `SYS_SLEEP` suspends the caller until a future scheduler tick.

In each case we use the same mechanism: we change the calling process's state from `PROC_RUNNING` to `PROC_BLOCKED`, record either a wait queue or a wake deadline in the process descriptor, and call the scheduler to switch to another ready task. A TTY reader sleeps until keyboard input arrives;

a `waitpid` caller sleeps until a child exits; a pipe endpoint sleeps until the buffer drains or fills; a sleep call simply sleeps until a deadline expires. When the relevant event fires, the sleeping process is transitioned back to ready and the scheduler picks it up on the next tick.

This is still the same Linux idea: blocking calls are state transitions plus wakeups, not busy loops. A **signal** — an asynchronous notification delivered to a process from the kernel or from another process — can also make a blocked syscall return early. `SYS_SLEEP` reports the remaining whole seconds in that case, matching Linux `sleep(3)` semantics.

Path Resolution Per Process

Every filesystem-touching syscall — open, create, unlink, mkdir, rmdir, rename, stat, exec, getdents, modload — resolves its path argument against the calling process's **cwd** (current working directory) before passing it to the VFS. The rules are simple: a path starting with `/` is root-relative; an empty cwd means the process is at the root; otherwise the cwd is prepended. This means a process whose cwd is `projects/src` can call `open("main.c")` and the kernel opens `projects/src/main.c` without the user program constructing the full path.

The cwd lives in the process descriptor, not in user space. It is inherited across `fork`, so a child starts in the same directory as its parent. `SYS_CHDIR` validates the target through the VFS — confirming that the path exists and is a directory — before writing the new cwd into the descriptor. `SYS_GETCWD` copies the current cwd string back to a user-supplied buffer.

Pipes and File Descriptor Duplication

`SYS_PIPE` creates an anonymous pipe — a unidirectional in-kernel ring buffer with a read-end and a write-end **fd** (file descriptor). Both ends are installed in the calling process's open-file table as distinct entries sharing the same buffer. When the write end is closed by all holders, a subsequent read from the read end returns 0 (EOF), allowing a pipeline to terminate naturally.

`SYS_DUP2` duplicates one fd onto another, closing the destination first if it was already open. The shell uses this in a forked child before calling `SYS_EXEC` to redirect stdin or stdout through a pipe. Because `SYS_EXEC` preserves the calling process's fd table, the exec'd program sees its standard streams already connected to the pipe without any further setup.

Virtual Memory Mappings

`SYS_BRK` is no longer the kernel's only user-space memory-management interface. The current syscall layer also exposes the classic Linux-shaped trio `SYS_MMAP`, `SYS_MUNMAP`, and `SYS_MPROTECT`, backed by a small per-process **VMA** (Virtual Memory Area — a tracked half-open virtual-address range with a shared policy for permissions and behaviour) table.

The implementation is intentionally narrow for now. `SYS_MMAP` accepts only anonymous private mappings: the caller passes an `old_mmap_args` structure through a user pointer in `EBX`, and the kernel currently accepts only `MAP_PRIVATE | MAP_ANONYMOUS`, `fd = -1`, and `offset = 0`. If the optional hint is page-aligned and the requested hole is free, the kernel uses it; otherwise it scans downward from the current stack window and places the mapping high in user space, above the `brk()` heap and below the reserved stack range. No physical page is allocated at `mmap()` time. The VMA is recorded immediately, and the first access later raises a page fault that the kernel handles by allocating a physical frame and installing the mapping on demand.

SYS_MUNMAP removes only generic anonymous mappings, not the ELF image, the brk() heap, or the reserved stack VMA. It first updates the VMA table, then walks the covered page-table entries and drops any present frames that had already been committed. SYS_MPROTECT likewise updates only generic anonymous mappings. It rewrites the VMA metadata first, then adjusts any present PTEs in place so later faults and /proc/<pid>/maps output observe the new permissions. Together, the three calls are enough for user space to reserve, release, and retag anonymous regions without abusing brk().

Process Creation and the exec/fork Split

SYS_EXEC replaces the calling process in place. The PID, parent linkage, process-group/session membership, cwd, signal mask, and open-file table survive; the user address space, heap, user stack, entry point, and process metadata derived from argv are rebuilt from the new ELF image. On success the old image never resumes. This is what makes the shell's fork(); exec(); waitpid(); path genuinely Unix-shaped rather than a spawn wrapper in disguise.

SYS_FORK takes the opposite approach: it clones the calling process's register frame, FPU state, open-file table, and paging structures, but it shares the existing user frames copy-on-write rather than copying them eagerly. Parent and child resume at the same instruction; the parent's return value is the child's PID, and the child's return value is 0. Fork plus exec together form the classic Unix process-creation idiom that every shell relies on.

SYS_EXIT stores the exit code in the process descriptor, marks the process PROC_ZOMBIE, and transfers to the scheduler. The zombie slot is kept alive until the parent calls SYS_WAITPID to collect the exit status, at which point the slot is freed.

Syscall Reference

The complete syscall table:

Number	Name	Description
1	SYS_EXIT	Mark process zombie with exit status
2	SYS_FWRITE	Write bytes to an open writable fd
3	SYS_READ	Read from stdin or an open fd
4	SYS_WRITE	Write a counted buffer to the VGA console
5	SYS_OPEN	Open a file read-only
6	SYS_CLOSE	Close a file descriptor
7	SYS_WAIT	Block until a PID exits
8	SYS_CREATE	Create or truncate a file, open writable
9	SYS_UNLINK	Delete a file
10	SYS_FORK	Clone the current process
11	SYS_EXEC	Replace the current process image with an ELF file
12	SYS_CLEAR	Clear the VGA console
13	SYS_SCROLL_UP	Page backward through terminal history
14	SYS_SCROLL_DOWN	Page forward toward the live console
15	SYS_MKDIR	Create a directory
16	SYS_RMDIR	Remove an empty directory

Number	Name	Description
17	`SYS_CHDIR`	Change the process working directory
19	`SYS_LSEEK`	Reposition the file offset of an open fd
20	`SYS_GETPID`	Return the calling process's PID
37	`SYS_KILL`	Send a signal to a PID or process group
38	`SYS_RENAME`	Rename or move a file or directory
45	`SYS_BRK`	Query or move the heap break; pages commit on fault
64	`SYS_GETPPID`	Return the calling process's parent PID
67	`SYS_SIGACTION`	Install or query a per-signal handler
90	`SYS_MMAP`	Create an anonymous private mapping
91	`SYS_MUNMAP`	Remove an anonymous mapping range
106	`SYS_STAT`	Retrieve file or directory metadata
119	`SYS_SIGRETURN`	Return from a user-space signal handler
125	`SYS_MPROTECT`	Change protections on an anonymous mapping range
126	`SYS_SIGPROCMASK`	Modify the signal blocking mask
141	`SYS_GETDENTS`	Enumerate directory entries
158	`SYS_YIELD`	Voluntarily yield the CPU
162	`SYS_SLEEP`	Suspend the process for N seconds
170	`SYS_MODLOAD`	Load a relocatable kernel module
171	`SYS_PIPE`	Create a pipe; return two fds
172	`SYS_DUP2`	Duplicate one fd onto another
183	`SYS_GETCWD`	Copy the process cwd into a user buffer
184	`SYS_TCGETATTR`	Read the TTY termios configuration
185	`SYS_TCSETATTR`	Write a new TTY termios configuration
186	`SYS_SETPGID`	Set a process's process group
187	`SYS_GETPGID`	Return a process's process group
188	`SYS_WAITPID`	Wait for a PID with WNOHANG/WUNTRACED flags
189	`SYS_TCSETPGRP`	Set the TTY foreground process group
190	`SYS_TCGETPGRP`	Get the TTY foreground process group
265	`SYS_CLOCK_GETTIME`	Read a kernel clock into a `timespec`

Where the Machine Is by the End of Chapter 16

We now expose a stable ring-3 ABI. Every service user programs depend on — console I/O, process creation and waiting, heap growth, anonymous memory mappings, filesystem operations, pipes, directory enumeration, metadata queries, and signal delivery — is available through a single `int 0x80` entry point. The generic wait-queue model ensures that waiting processes consume no CPU while they sleep, regardless of whether the event source is a TTY, a pipe, a child process, or a timeout. The per-process cwd makes relative paths work naturally, and the per-process VMA table gives the memory-management syscalls a coherent view of heap, stack, and `mmap()` regions.

Chapter 17 — File I/O and Open File Descriptors

The File Descriptor Table

Chapter 16 left us with a complete syscall ABI and blocking wait queues. Every open file in a Unix-like system is identified by a small integer called a **file descriptor** (fd). The fd is not a pointer into the filesystem — it is an index into a per-process table. Each slot in the table holds our view of an open file: the inode number, the current read/write position, the file's cached size, and a flag indicating whether the descriptor is writable.

Each process carries this table inside its `process_t` descriptor. The table has a fixed number of slots. By convention, the first three are pre-reserved: fd 0 is standard input (connected to the keyboard character device), fd 1 is standard output (connected to the VGA console), and fd 2 is standard error (not yet implemented). Regular disk-backed files start at fd 3 and occupy higher slots.

When a process has two files open, the table might look like this:

fd	Entry	Target	Position
`0`	stdin device	Console input	Not seekable
`1`	stdout device	Console output	Not seekable
`2`	stderr device	Console errors	Not seekable
`3`	Open for reading	File A	Next read starts at byte 256
`4`	Open for writing	File B	Next write starts at byte 0

Identifying files by inode number rather than by raw disk **LBA** (Logical Block Address) is what connects this layer to the inode-based filesystem in Chapter 13. The inode number is stable — it does not change when a file is renamed or when a directory entry is updated — so the open descriptor remains valid even if the file's name changes while it is open.

Opening and Closing Files

`SYS_OPEN` resolves the caller's path string through the VFS (Virtual File System) layer, which returns an inode number and file size. The kernel then scans the current process's `open_files[]` array for the first free slot and installs that metadata with `offset = 0` and `writable = 0`. The returned value is the slot index — the fd number the caller uses for all subsequent operations on this file.

`SYS_CREATE` does the same but requests a writable slot. On the filesystem side, if a file with the given name already exists, it is truncated: its data blocks are freed and the inode size is reset to zero. If no such file exists, a fresh inode is allocated and a directory entry is created. Either way the descriptor starts at `offset = 0` and `size = 0`, representing an empty file ready to receive writes. DUFS stamps the file with the current wall-clock time as the creation or modification timestamp.

`SYS_CLOSE` marks the slot as free. If the descriptor was opened writable, we flush the inode before releasing the slot. In the current DUFS implementation, `fs_write` already persists the inode on every write, so the flush at close time is a no-op — but it exists as the correct hook for future caching layers.

Reading

When a process calls `SYS_READ`, we dispatch based on the fd number. A read from fd 0 goes directly to standard input and blocks until terminal data is available. A read from fd 3 or higher goes through the file table.

For a file read, we first check whether the current `offset` has reached `size` — if so we return 0, signalling **EOF** (End of File). Otherwise the kernel asks the filesystem to read from that inode at the current offset into the caller's buffer. The filesystem layer performs the actual block-level I/O (Chapter 13), translating the logical byte offset into block indices and LBAs. On return, the descriptor's `offset` advances by the number of bytes read. Subsequent reads automatically continue from where the previous one stopped, because the offset is maintained across calls in the fd table entry.

`SYS_LSEEK` repositions the offset of an open file descriptor without reading or writing. It accepts a signed displacement and a mode — set to an absolute position (`SEEK_SET`), advance relative to the current position (`SEEK_CUR`), or move relative to the end of the file (`SEEK_END`). After a seek, the next read or write starts from the new position. Seeking is rejected on non-file descriptors such as pipes or the TTY.

Creating and Writing

`SYS_FWRITE` writes through a writable descriptor. The kernel passes the descriptor's inode number, current offset, user buffer, and byte count down to the filesystem layer. There the write may allocate new data blocks on demand, perform read-modify-write for partial-block updates, advance through the inode's direct and indirect block chains, update the inode's size and modification timestamp, and flush the updated metadata to disk. On return, the descriptor's `offset` and cached `size` advance by the number of bytes written.

The write path is necessarily heavier than the read path because it may change the filesystem's physical layout — new blocks must be allocated, new indirect table entries may need to be written, and the bitmaps on disk must reflect the current allocation state. These writes happen synchronously: when `SYS_FWRITE` returns, the data is on disk.

Deletion, Rename, and Directories

`SYS_UNLINK` removes a file by name. We resolve the path to an inode, decrement the inode's link count, and when the count reaches zero, free all data blocks (including any indirect and double-indirect chains) and clear the inode's bitmap bit. The directory entry is then cleared, making the name available for reuse.

`SYS_RENAME` moves a file or directory. Both the source and destination paths are resolved relative to the calling process's cwd. Moving a regular file between the root and a subdirectory is supported. Replacing an existing regular file at the destination is permitted. Replacing an existing directory is rejected.

`SYS_MKDIR` and `SYS_RMDIR` create and remove directories. Both paths are resolved relative to the cwd. `SYS_RMDIR` refuses to remove a directory that still contains entries.

`SYS_GETDENTS` enumerates the entries in a directory. When called with a path, it resolves to the corresponding directory inode and reads its directory blocks. When called with a null path and

the process is at the root, it lists the root. Each returned entry carries the name, inode number, type, and size — enough for `ls` to display a complete directory listing without a separate `stat` call for each entry.

`SYS_STAT` fills a small metadata structure in user space with the file's type, byte size, link count, and last-modified timestamp. The timestamp is the Unix time in UTC seconds stored in the inode's `mtime` field.

Pipes and the fd Table

`SYS_PIPE` also installs entries in the fd table. It allocates a shared ring buffer in kernel heap memory and creates two fd-table entries — one read-only, one write-only — both referencing the same buffer through a pointer rather than an inode number. Reads from the read end block when the buffer is empty; writes to the write end wake any blocked reader. When the last write-end descriptor is closed, the next read from the read end returns 0, signalling EOF to the reader.

Where the Machine Is by the End of Chapter 17

The process descriptor now holds a live fd table that spans keyboard input (fd 0), console output (fd 1), and up to eight disk-backed or pipe-backed file descriptors starting at fd 3. The open, read, write, seek, and close operations all operate through inode numbers resolved at open time, so the filesystem layer and the syscall layer are cleanly separated: the syscall layer advances offsets and validates access flags; the filesystem layer handles the block allocation and disk I/O.

Part VI — The User Environment

Part V established that programs can run. What it did not establish is that they can do anything useful from a user's perspective. A process that launches, computes silently, and exits is not an interactive system — it is a batch job. For the OS to feel like an operating system in the way users experience one, processes need to respond to keystrokes in a principled way, react to events that arrive while they are in the middle of something else, and exist within a shell that lets the user compose, launch, and manage them.

The gap between "programs run" and "users can work" spans four distinct problems. The first is the terminal: raw keyboard characters are not enough for interactive programs. A text editor needs to know about line boundaries, backspace, and arrow keys. A shell needs to echo what the user types, accumulate a full line before acting on it, and hand the terminal off to the foreground program without losing control of it. The TTY line discipline — the layer of software that sits between the keyboard driver and the programs reading from the terminal — formalises all of this into a configurable set of rules governing how characters flow. The second problem is signals: a running program needs a way to receive asynchronous notifications — that the user pressed Ctrl+C, that a pipe it was writing to has no reader, that a child process has exited — without constantly checking for them. Signals are the kernel mechanism that delivers those notifications at the exact moment the process is about to return to user code. The third problem is the user-space foundation: before any user program can be written as ordinary C, there must be a startup routine that sets up the stack, calls `main`, and issues an exit syscall when `main` returns, plus a library of syscall wrappers so programs do not have to write raw assembly. The fourth is the standard library: programs expect `printf`, `malloc`, `fopen`, and `strcmp` to simply exist. The fifth is the shell itself, which ties all of the above together into the surface the user actually touches.

Chapter 18 builds the TTY line discipline: raw and canonical input modes, the `termios` configuration interface, non-busy blocking reads, and the foreground process group mechanism that determines which process owns the terminal at any moment. Chapter 19 builds the signal subsystem: the pending and blocked signal bitmasks in the process descriptor, the delivery window just before every return to user code, the signal frame that redirects execution to a user handler and returns cleanly, and the stop and continue signals that underpin job control. Chapter 20 builds the user-space runtime: the `crt0` entry point that the linker calls before `main`, the syscall wrapper library, and the `sbrk`-based heap. Chapter 21 builds a minimal C standard library on top of that runtime: string and memory functions, character classification, a sink-based `printf`, `FILE` streams, and POSIX wrappers. Chapter 22 builds the interactive shell: readline with tab completion, command tokenisation, `fork/exec/wait`, pipe construction, environment management, and the `jobs`, `fg`, and `bg` builtins that let the user manage stopped and background processes.

By the end of Part VI the OS has a face. The user sits at a prompt, runs programs, interrupts them with Ctrl+C, suspends them with Ctrl+Z, and resumes them with `fg`. Programs written in C with ordinary standard library calls compile and run without modification. The system is usable.

Chapter 18 — The TTY Line Discipline

From File Descriptors to a Line Discipline

Chapter 17 left us with a working fd table and `SYS_READ` dispatching fd 0 reads to the keyboard ring buffer. Two shortcomings remained: reads busy-waited, and terminal-generated signals relied on a kernel-side heuristic rather than explicit process-group ownership. We fix both by adding a proper **TTY** (teletypewriter) abstraction between the keyboard driver and the programs that read from it.

What a TTY Is

The word "TTY" is an anachronism — it originally referred to a physical Teletype machine, a combination keyboard and paper printer that a programmer used to communicate with a mainframe in the 1960s. The term has survived in Unix systems to mean the software layer that sits between the keyboard hardware and the programs that read from it.

A TTY has three jobs. The first is *line discipline*: in its default mode it buffers incoming characters until the user presses Enter, then hands the complete line to whichever program is waiting. This is called **canonical mode** (ICANON). Programs that want to react to every keystroke individually — a text editor, a game, a readline loop in a shell — switch the TTY into **raw mode** by clearing the ICANON flag, at which point characters are delivered immediately as they arrive.

The second job is *signal generation*. Certain control characters carry meaning that goes beyond text: pressing Ctrl+C on a Unix terminal has always meant "interrupt the foreground process". The TTY, not the keyboard driver, is the right place to translate that keypress into a **signal** — a kernel-mediated asynchronous notification that causes a process to take some action, by default terminating it — because the TTY knows which process group is currently in the foreground. This chapter focuses on the TTY's role in generating and targeting those signals.

The third job is *echo*. When the ECHO flag is set, the TTY prints each incoming character back to the screen so the user can see what they are typing. In raw mode a program usually handles its own echo.

The `termios` Structure

Unix programs configure a TTY's behaviour through a structure traditionally called `termios`. Our implementation keeps only the fields that are currently useful:

```
typedef struct {
    uint32_t c_lflag;   /* local flags */
} termios_t;
```

The `c_lflag` field is a bitmask. The flags we support at this stage are:

Flag	Bit	Meaning
`ICANON`	0	Buffer input by line; deliver on newline
`ECHO`	1	Echo each character as it is typed
`ECHOE`	2	Echo backspace as BS SP BS (erase the character visually)

Flag	Bit	Meaning
`ISIG`	3	Generate terminal signals (`SIGINT` on Ctrl+C, `SIGTSTP` on Ctrl+Z)

We default to `c_lflag = 0` — raw mode, no echo. That keeps the shell's own readline loop in control of echo and editing until a program explicitly requests canonical mode.

Programs call `sys_tcgetattr` and `sys_tcsetattr` to read and write these settings. The `TCSANOW` action applies the change immediately; `TCSAFLUSH` additionally discards any unread input in the TTY's buffers before applying the new settings.

Process Groups and the Foreground Group

A **process group** is a collection of related processes — typically a shell command pipeline. Every process has both a **process group ID** (PGID) and a **session ID** (SID) stored in its `process_t`. When a process is created with `process_create`, both fields start at zero; the scheduler fills them in with the process's own PID as soon as the process enters the process table, making every new process both a group leader and a session leader by default unless a parent later re-groups it.

When a process forks (via `SYS_FORK`), the child inherits the parent's PGID — it joins the same group automatically.

The TTY structure records two ownership fields:

- `ctrl_sid`: the session that owns this terminal
- `fg_pgid`: the process group inside that session that currently has the foreground

When Ctrl+C or Ctrl+Z arrives and `ISIG` is set, the terminal sends `SIGINT` or `SIGTSTP` to the foreground process group recorded in the TTY. Every process in that group sees the signal; any process outside it is untouched.

A program calls `sys_setpgid(0, 0)` to make itself a new group leader (its own PGID becomes its PID). A shell calls `sys_setpgid(0, 0)` at startup to establish its own group, then uses `sys_tcsetpgrp` to hand the terminal to a foreground child or pipeline. `SYS_TCSETPGRP` enforces the same session boundary Unix does: the caller must already belong to the terminal's controlling session, and the target PGID must exist inside that same session. The first successful caller claims the terminal for that session.

When no data is available in the TTY, the reader blocks on the terminal's wait queue and the scheduler switches to something else. When the keyboard IRQ later delivers input that the current line discipline considers readable — either a byte in raw mode or a completed line in canonical mode — the terminal wakes those blocked readers.

This replaces the busy-wait spin loop in `SYS_READ` with genuine blocking, freeing the CPU for other processes while the user is thinking.

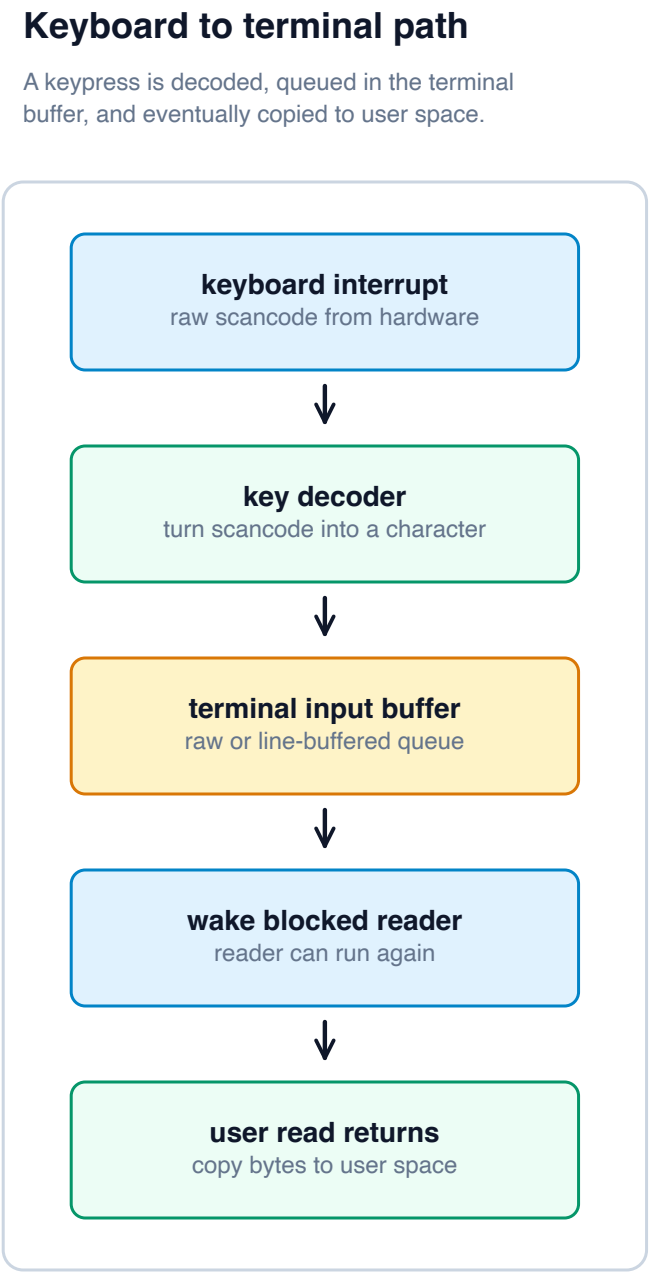
The TTY Structure

```c
typedef struct {
    char     raw_buf[256];   /* ring buffer for raw mode */
    uint32_t raw_head;
    uint32_t raw_tail;

    char     canon_buf[256]; /* line buffer for canonical mode */
    uint32_t canon_len;
    uint32_t canon_ready;    /* 1 = a complete line is waiting */

    termios_t termios;
    uint32_t  ctrl_sid;
    uint32_t  fg_pgid;
    wait_queue_t read_waiters;
    uint32_t  in_use;
} tty_t;
```

There is currently one TTY, `tty_table[0]`, referred to as **tty0**. Every process's standard input file descriptor (`fd 0`) is an `FD_TYPE_TTY` entry pointing at `tty_idx = 0`.

The Data Path

The journey of a keystroke through the revised system looks like this:

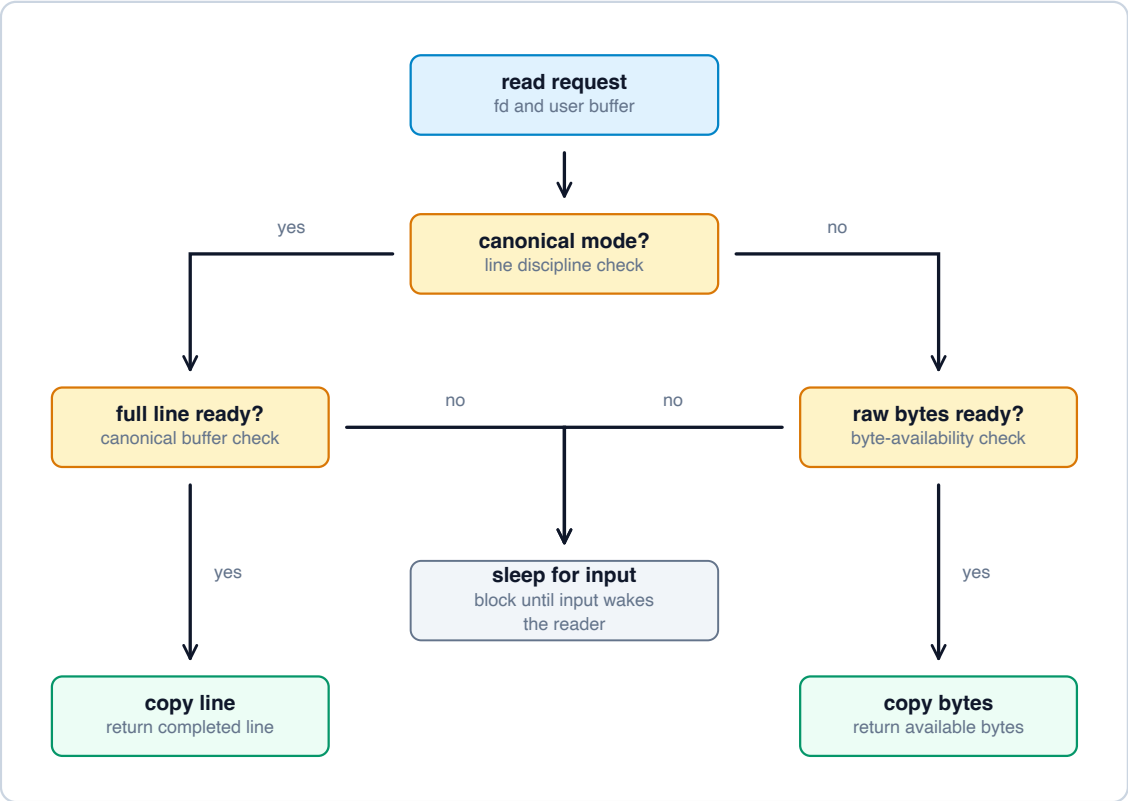


Chapter 18, Figure 1

On the read side, an `FD_TYPE_TTY` descriptor eventually asks the terminal layer to produce input according to the current mode:

Terminal read behavior

A terminal read branches on the active input mode and on whether enough buffered input is already available.



Chapter 18, Figure 2

Prompt-Time Shell Signals

We no longer keep a fallback "guess the foreground job" path. Instead, the boot shell becomes a real process-group leader at startup, claims the foreground terminal for its own PGID, and installs `SIGINT` and `SIGTSTP` handlers for prompt time. While the shell is reading a command line, those handlers simply mark that prompt input should be abandoned and redrawn.

When the shell launches a foreground job, it hands the TTY to the child's PGID with `SYS_TCSETPGRP` and restores the default signal dispositions in the child side. That means Ctrl+C and Ctrl+Z go to the job while it owns the terminal, and to the shell while the shell owns the terminal. No kernel-side heuristic is needed.

System Calls Added for the TTY

The TTY layer required several new syscalls. `SYS_TCGETATTR` and `SYS_TCSETATTR` read and write the `termios_t` configuration for a given file descriptor. `SYS_TCSETATTR` takes an action argument: `TCSANOW` applies the new settings immediately, while `TCSAFLUSH` additionally drains any unread bytes from the TTY's input buffers before applying the change — important when a program

wants to ensure stale keystrokes from before a mode switch are not misinterpreted under the new settings.

`SYS_SETPGID` and `SYS_GETPGID` read and write a process's group membership. `SYS_TCSETPGRP` and `SYS_TCGETPGRP` control which group the TTY currently treats as foreground. `SYS_WAITPID` extends the existing wait mechanism with option flags: `WUNTRACED` causes the call to return when a child enters `PROC_STOPPED` state (so the shell can detect Ctrl+Z), and `WNOHANG` returns immediately with zero if no child has changed state.

User-space wrappers for all these calls are available in the syscall library.

Switching Modes at Runtime

A program that wants line-buffered input with echo calls `sys_tcgetattr` to obtain the current settings, sets the `ICANON`, `ECHO`, `ECHOE`, and `ISIG` bits in `c_lflag`, and calls `sys_tcsetattr` with `TCSANOW`. From that point forward, characters accumulate in the canonical line buffer until the user presses Enter. If a reader asks for data before a full line is ready, the process blocks and the scheduler switches away. When Enter arrives, the line discipline marks the line complete, wakes the blocked readers, and the next read returns the buffered line in one piece.

The shell itself keeps its own raw readline loop unchanged — the TTY defaults to raw mode so the existing shell behaviour is preserved without any shell-side modifications.

Where the Machine Is by the End of Chapter 18

At the end of this chapter the keyboard's characters no longer flow directly to a ring buffer — they pass through a stateful line discipline first. We have a single TTY (`tty0`) that every process's standard input is connected to. Processes that call `sys_read(0, ...)` and find no data available now sleep on the terminal instead of spinning; the CPU is genuinely free while they wait. Ctrl+C sends SIGINT and Ctrl+Z sends SIGTSTP to the foreground process group recorded in the TTY, and the TTY itself records which session owns that terminal. The `termios` API gives user-space programs the ability to switch between raw and canonical input modes. Process groups and sessions now line up with the shell's `jobs`/`fg`/`bg` builtins, so job control no longer depends on any kernel fallback path.

Chapter 19 — Signals

What a Signal Is

Chapter 18 left us with a TTY that tracks a foreground process group and generates control-character events, but nothing to act on them yet. A **signal** is a kernel-mediated asynchronous notification delivered to a process. The word "asynchronous" is the key distinction from every other form of process-to-process or kernel-to-process communication: an ordinary read, write, or syscall takes place at a predictable moment the process chose. A signal can arrive at any point — between two instructions, in the middle of a system call, whenever the interrupted process is about to return to user space.

Linux defines signals using a standard numbering. We adopt the same numbers for the eight signals we support:

Number	Name	Default action	When sent
2	SIGINT	terminate	User presses Ctrl+C
9	SIGKILL	terminate (uncatchable)	`SYS_KILL` with signum 9
13	SIGPIPE	terminate	Write to a broken pipe
15	SIGTERM	terminate	`SYS_KILL` with signum 15
17	SIGCHLD	ignore	Child process exits or stops
18	SIGCONT	continue	Resumes a stopped process
19	SIGSTOP	stop (uncatchable)	`SYS_KILL` with signum 19
20	SIGTSTP	stop	User presses Ctrl+Z

Every process can choose to handle or ignore each signal (except SIGKILL and SIGSTOP, which cannot be caught, blocked, or ignored). Our signal subsystem has four jobs: recording that a signal is pending, choosing the moment to deliver it, killing the process or handing control to its handler, and — for the stop/continue pair — suspending or resuming execution.

Signal State in the Process Descriptor

Each process tracks three signal-related fields in its descriptor:

```
uint32_t  sig_pending;          /* bitmask: bit N = signal N is pending */
uint32_t  sig_blocked;          /* bitmask: bit N = signal N is masked  */
uint32_t  sig_handlers[NSIG];   /* per-signal disposition               */
```

`NSIG` is 32 — the size of a `uint32_t` bitmask. Bit N in `sig_pending` means signal N has been sent to this process but not yet delivered. Bit N in `sig_blocked` means delivery of signal N is postponed until the bit is cleared.

Each entry in `sig_handlers` holds one of three values:

- `SIG_DFL` (0) — take the platform default action. For most signals this means terminate. SIGCHLD is ignored by default. SIGSTOP and SIGTSTP stop the process. SIGCONT resumes it.
- `SIG_IGN` (1) — silently discard the signal.

- Any other value — the virtual address of a user-space function with signature `void handler(int signum)`.

Signal state is initialized to all-zero in `process_create` (no pending signals, nothing blocked, every disposition SIG_DFL). `process_fork` copies `sig_handlers` from the parent so installed handlers survive across `fork`, but clears `sig_pending` in the child — POSIX specifies that pending signals are not inherited.

Sending a Signal

`sched_send_signal(pid, signum)` is the universal signal-sending function. It locates the target process in the process table, sets the appropriate bit in `sig_pending`, and if the process is in the generic `PROC_BLOCKED` state it calls the same wakeup helper the rest of the scheduler uses, transitioning the process back to `PROC_READY` and clearing its wait-queue linkage or timeout. Setting `g_need_switch = 1` ensures the scheduler runs at the next opportunity. This is why a sleeping process woken by a signal returns from `SYS_SLEEP` early, with the number of remaining seconds as its return value.

Two signals receive special treatment inside `sched_send_signal`:

- **SIGCONT**: if the target is in `PROC_STOPPED` state, it is immediately transitioned to `PROC_READY`. Any pending SIGSTOP or SIGTSTP bits are cleared — a continue cancels a queued stop (Linux semantics).
- **SIGSTOP / SIGTSTP**: any pending SIGCONT bit is cleared — a queued stop cancels a queued continue.

SIGKILL also receives special handling: if the target is stopped (`PROC_STOPPED`), SIGKILL forces it back to `PROC_READY` so the kill can be delivered. No other signal can wake a stopped process — only SIGCONT and SIGKILL.

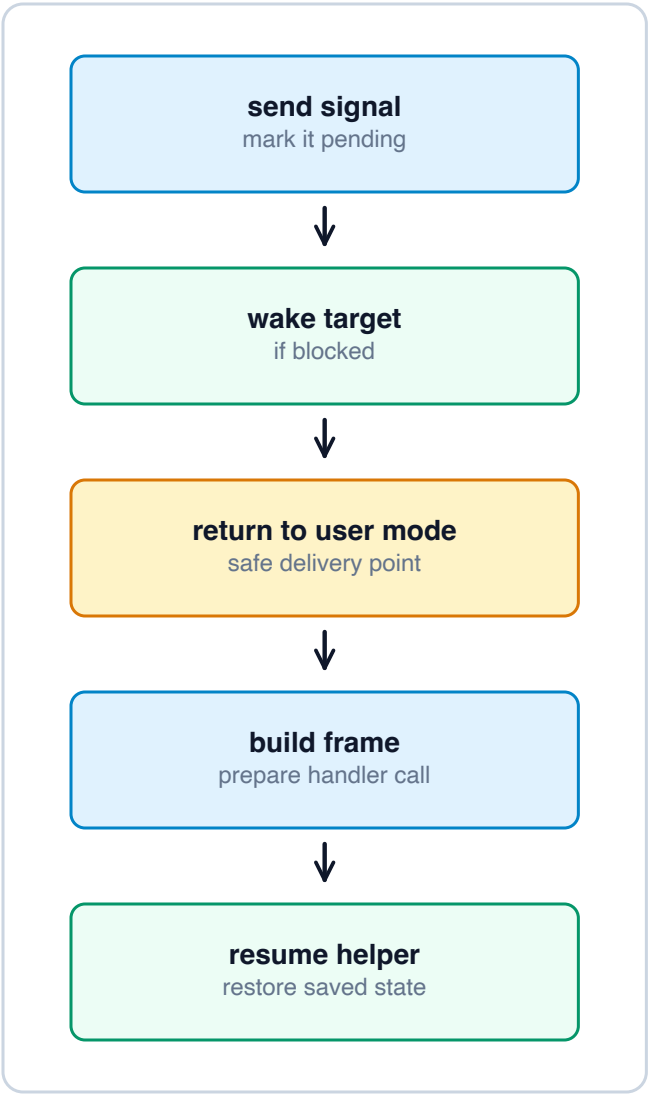
From the keyboard driver, Ctrl+C and Ctrl+Z now flow through the TTY layer rather than through a foreground heuristic. `tty_ctrl_c(0)` sends `SIGINT` to `tty0`'s foreground process group; `tty_ctrl_z(0)` sends `SIGTSTP` to that same group. While the shell owns the foreground TTY, those signals are delivered to the shell itself and its prompt-time handlers redraw the prompt. While a job owns the foreground TTY, the entire foreground process group receives the signal, including both sides of a pipeline.

User programs send signals through `SYS_KILL` by passing either a positive PID or a negative process-group ID and a signal number. The call returns zero on success and -1 if the signal number is out of range.

The Delivery Window

A signal's pending bit is set the moment `sched_send_signal` runs — possibly during an interrupt, possibly during another process's syscall. The signal is not delivered at that instant. Delivery happens at one specific location: **the moment a process is about to return to ring-3 code after a syscall or IRQ**.

The life of a signal, from send to handler return:



Chapter 19, Figure 1

This is the only safe point for delivery because it is the only moment when the full user-space register context is sitting in a complete, well-defined frame on the kernel stack and can be copied and redirected without corrupting anything.

Both `syscall_common` and `irq_common` call `sched_signal_check` immediately before their final restore-and-`iret` sequence. They pass the current stack pointer as an argument; `sched_signal_check` returns the stack pointer to use for the restore — unchanged if no signal is

delivered, or adjusted to point at a newly built signal frame if one was pushed onto the user stack. The trampoline loads the returned value into ESP, then falls through to the same register-pop and `iret` it would have executed anyway. The signal delivery mechanism is therefore invisible to the rest of the trampoline — it sees only "here is the frame to restore."

`sched_signal_check` computes `sig_pending & ~sig_blocked` to find deliverable signals. It picks the lowest-numbered one, clears its pending bit, and looks up the handler:

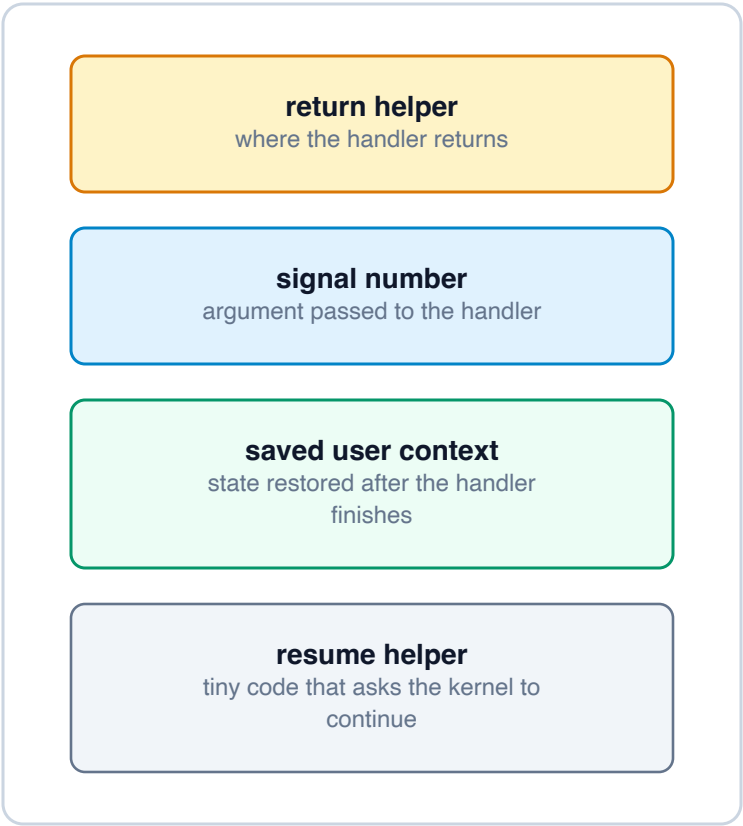
- **SIG_IGN**: nothing to do; returns unchanged `frame_esp`.
- **SIG_DFL** and fatal: calls `sched_mark_signaled(signum, dumped_core)` to encode a signal-style wait status, then calls `schedule()` to switch immediately to the next READY process. The trampoline eventually restores some other process's register frame and `iret`s to it.
- **User handler**: calls `build_signal_frame`, which constructs a signal frame on the user stack and redirects the saved EIP to the handler. Returns the unchanged `frame_esp` — the trampoline restores the (now-modified) frame and `iret`s to the handler instead of the original user code.

The ring-3 check (`frame[15] & 3 == 3`) ensures signals are never delivered when returning to kernel-mode code. Signals are strictly a user-space concept.

One subtlety remains when a process is asleep inside a blocking syscall. A signal wakeup moves the process from `PROC_BLOCKED` back to `PROC_READY`, but the resumed context is still inside the kernel loop that was implementing the syscall. Pipe reads, pipe writes, and TTY reads therefore check `cur->sig_pending` after each wakeup and return early with an interrupt-style error if a signal is pending. Control then reaches `syscall_common`, which calls `sched_signal_check` with the syscall's ring-3 iret frame, and the signal is delivered normally.

The Signal Frame

When a user handler must be called, the kernel pushes a **signal frame** onto the user stack to save the context that was about to be restored. The frame is 32 bytes and contains everything needed to resume normal execution after the handler returns.



Chapter 19, Figure 2

`ret_addr` points to the trampoline bytes at offset 24 within the same frame. After the frame is built, the kernel modifies two fields in the saved kernel register frame:

- `frame[14]` (saved EIP) is set to the handler's virtual address.
- `frame[17]` (saved ESP) is set to the new, lower user stack pointer.

When `iret` runs, the CPU jumps to the handler with `[esp]` = `ret_addr` and `[esp+4]` = `signum`. To the handler, this looks exactly like an ordinary C function call.

Returning From a Signal Handler

The handler is an ordinary C function. When it executes `ret`, the CPU pops `ret_addr` from the stack and jumps to the embedded trampoline code. The trampoline executes:

```
mov eax, 119    ; SYS_SIGRETURN
int 0x80
```

`SYS_SIGRETURN (119)` in the kernel reads back the saved context from the signal frame. At the moment of the `int 0x80`, the user's ESP points to the `signum` slot (offset 4 in the frame), so the frame base is `user_esp - 4`. The kernel restores:

- `frame[14]` ← `sig_frame[2]` (original EIP)
- `frame[16]` ← `sig_frame[3]` (original EFLAGS)
- `frame[11]` ← `sig_frame[4]` (original EAX — the syscall return value before the signal)
- `frame[17]` ← `sig_frame[5]` (original ESP)

After this restore, the process resumes exactly where it was interrupted, with registers in the same state they had before the signal arrived.

Registering and Masking Signals

A process changes the disposition of a signal through `SYS_SIGACTION`. Passing `SIG_DFL` (zero) restores the platform default; passing `SIG_IGN` (one) silently discards the signal. Any other value is treated as the virtual address of a handler function with signature `void handler(int signum)`. The call also accepts an optional output pointer where the previous disposition is written before the new one is installed, which lets library code chain handlers. SIGKILL and SIGSTOP have fixed dispositions — the kernel refuses to change them.

The signal mask is modified through `SYS_SIGPROCMASK`. The caller specifies both a mode and a bitmask. Mode `SIG_BLOCK` ORs the supplied bits into the current mask, adding signals to the blocked set; `SIG_UNBLOCK` clears those bits; `SIG_SETMASK` replaces the mask entirely. Regardless of mode, the kernel always strips bits 9 (SIGKILL) and 19 (SIGSTOP) from the resulting mask — those two signals cannot be blocked by any process, ever.

A signal that is both pending and blocked is held indefinitely in `sig_pending`. It will be delivered the first time the process's `sig_blocked` mask is cleared at that bit, either by an explicit `SYS_SIGPROCMASK` call or by returning from a signal handler that temporarily blocked other signals.

Stop and Continue (Job Control Signals)

Three signals form the stop/continue mechanism that enables job control in the shell:

SIGSTOP (19) is the uncatchable stop. Like SIGKILL, it cannot be caught, blocked, or ignored. When `sched_signal_check` encounters a pending SIGSTOP, it calls `sched_mark_stopped(SIGSTOP)`, which transitions the process to `PROC_STOPPED`, encodes the stop signal in `exit_status` using Linux-compatible encoding `(stop_signal << 8) | 0x7F`, sends SIGCHLD to the parent, and wakes the parent if it is waiting. The scheduler then calls `schedule()` to switch away. The stopped process will not be picked by `sched_pick_next` until it receives SIGCONT.

SIGTSTP (20) is the terminal stop — sent to the foreground process group when the user presses Ctrl+Z. Unlike SIGSTOP, SIGTSTP *can* be caught or ignored. If the handler is SIG_DFL, the process is stopped exactly as with SIGSTOP. If the process has installed a handler, the signal is delivered as a normal signal frame, and the handler can choose to do cleanup before stopping (or to ignore the stop entirely). If the disposition is SIG_IGN, the signal is discarded.

SIGCONT (18) resumes a stopped process. The wake-up happens inside `sched_send_signal` itself — the target is transitioned from `PROC_STOPPED` back to `PROC_READY` immediately, before any signal-check pass. If the process has installed a SIGCONT handler, the handler is delivered the

next time the process returns to user space. Otherwise the signal is silently discarded after the wake-up.

The Ctrl+Z delivery chain mirrors the Ctrl+C chain. The keyboard IRQ handler detects scancode 0x2C with Ctrl held, and calls `tty_ctrl_z(0)`. The TTY driver prints `^Z\n` and, if `fg_pgid` is set, sends SIGTSTP to the foreground process group via `sched_send_signal_to_pgid`.

Wait Status Encoding

When a parent waits for a child via `SYS_WAIT` or `SYS_WAITPID`, the kernel returns a Linux-compatible encoded status word rather than the raw exit code. This encoding lets the caller distinguish between a child that exited normally and one that was stopped by a signal:

Condition	Encoding	Macro
Normal exit (code N)	`(N << 8) | 0x00`	`WIFEXITED(s)` true, `WEXITSTATUS(s)` = N
Stopped by signal S	`(S << 8) | 0x7F`	`WIFSTOPPED(s)` true, `WSTOPSIG(s)` = S

`SYS_WAITPID (188)` extends `SYS_WAIT` with option flags:

- **WUNTRACED**: also return when a child enters `PROC_STOPPED` (needed for the shell to detect Ctrl+Z).
- **WNOHANG**: return 0 immediately if no child has changed state (used by `jobs` to check background processes without blocking).

The shell uses `sys_waitpid(pid, WUNTRACED)` when running a foreground program. If the child is stopped, the shell adds it to its job list and re-prompts. The `fg` builtin later calls `sys_kill(pid, SIGCONT)` and `sys_waitpid(pid, WUNTRACED)` to resume and re-wait.

Terminal Ownership

Two syscalls manage which process group owns the terminal. `SYS_TCSETPGRP` sets the TTY's `fg_pgid` to a given process group ID; the shell calls it before launching a foreground child to hand the terminal over, and calls it again after the child exits or stops to reclaim it. `SYS_TCGETPGRP` reads the current `fg_pgid` back — useful when a process wants to check whether it is still in the foreground before performing terminal I/O.

Where the Machine Is by the End of Chapter 19

Every process now carries a 32-entry signal disposition table and two bitmasks. The keyboard driver sends SIGINT on Ctrl+C and SIGTSTP on Ctrl+Z. Signals are held as pending bits until the target process is about to return to user space, at which point `sched_signal_check` delivers them — terminating the process (SIG_DFL for fatal signals), stopping it (SIGSTOP/SIGTSTP), or building a signal frame on the user stack to redirect execution to an installed handler.

A stopped process enters `PROC_STOPPED` and is invisible to the scheduler until SIGCONT (or SIGKILL) arrives. The shell detects this via `SYS_WAITPID` with `WUNTRACED`, adds the stopped child to its job list, and offers `fg`, `bg`, and `jobs` builtins for resuming or inspecting jobs.

The signal path now covers the full lifecycle: asynchronous delivery, handler invocation and return, process stop and continue, and wait-status reporting that distinguishes exit from stop. Together with the TTY's foreground process group tracking (Chapter 18), this gives us a complete job control subsystem.

Chapter 20 — User-Space Runtime Library

What a Runtime Library Is, and Why User Programs Need One

Chapter 19 left us with a complete signal subsystem and a process that can handle asynchronous notifications. Until now, every user program we wrote has been responsible for its own `_start` entry point and its own inline-assembly wrappers for `int 0x80`. That boilerplate has to be copied into every new program, and the programs themselves cannot be written as plain C with a `main` function.

On Linux, a user program written in C looks like this: you write `main`, you call `printf`, you return 0, and the compiler does the rest. What the compiler actually does is invisible to the programmer, but a lot is happening behind the scenes. A small startup object called **crt1.o** (the "C runtime, version 1") is linked into every executable. That object defines `_start`, the real entry point the kernel jumps to, which calls `main` and then passes `main`'s return value to `exit`. **libc** (the C standard library) provides `printf`, `write`, `read`, and every other standard C function, and each of those eventually issues a system call to ask the kernel to do the actual work.

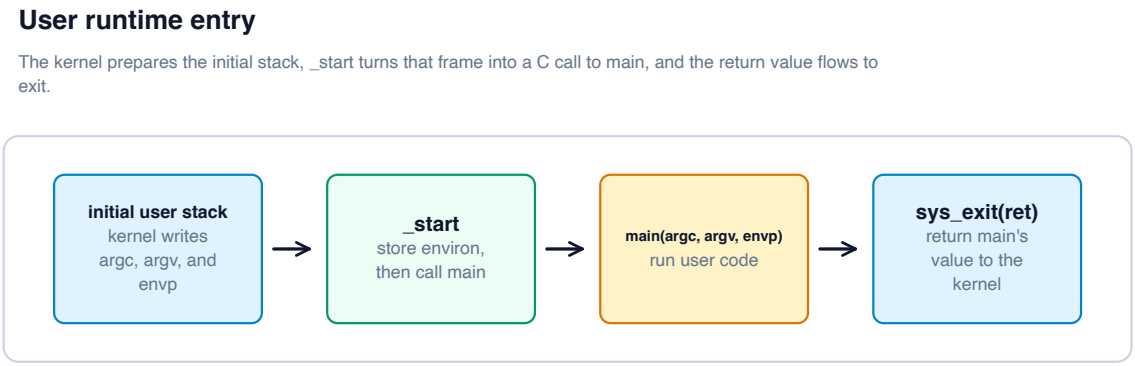
We build with `-nostdlib -ffreestanding`, two compiler flags that explicitly turn off every automatic library. Nothing is linked in by default — not crt1.o, not libc, not even the runtime helpers GCC would normally provide. We build a shared user-space runtime library — a minimal equivalent of crt1.o and libc — so that every user program can be written as plain C with a `main` function.

The Startup File

The **ELF** entry point `_start` is declared in the user linker script and read by the ELF loader from the file's header. Defining `_start` in a shared startup assembly file means every user program gets it for free.

By the time `_start` begins executing, the kernel's `process_create` has already laid a full argument and environment frame at the top of the user stack in **SysV i386 ABI** (System V Application Binary Interface for 32-bit x86, the calling-convention standard that defines how arguments are passed, registers are used, and stacks are managed) order. The word at the lowest stack address is `argc`, the word above it is a `char **argv` pointer, and the word above that is a `char **envp` pointer. The pointer arrays and raw strings live at higher addresses.

The bridge from the kernel's `iret` to user `main` looks like this:



Chapter 20, Figure 1

`_start` pops all three words, stores the `envp` pointer into the global `environ` variable (so that library functions like `getenv` can find it without being passed it explicitly), then pushes all three back in cdecl order before calling `main`. After `main` returns, `_start` discards the three argument slots, pushes the return value as the argument to `sys_exit`, and calls `sys_exit`. An unreachable infinite loop follows in case something goes wrong.

Because the kernel always writes `argc`, `argv`, and `envp` — even for processes launched with no arguments, where it emits `argc = 0` and NULL-terminated empty arrays — `_start` can pop all three unconditionally. A user program declaring `int main(int argc, char **argv)` still works correctly: the third stack word is never read by the C code, and the cdecl calling convention makes `_start` responsible for cleaning up the extra slot.

The Syscall Library

All `int 0x80` inline assembly now lives in exactly one library file. User programs include the companion header and call the wrappers as ordinary C functions — no inline assembly, no register constraints visible to the caller.

The convention the wrappers follow is straightforward: `EAX` holds the syscall number, `EBX` holds the first argument, `ECX` the second, and `EDX` the third. The wrappers use **GCC** (GNU C Compiler) extended-assembly syntax to bind C variables to those specific registers before executing `int 0x80`, and to extract the return value from `EAX` afterwards. The complete table of syscall numbers and their argument conventions was documented in Chapter 16.

One wrapper deserves a special note: `sys_read`. It passes a buffer pointer in `ECX`, and the kernel writes the character it reads into that buffer through the pointer. If the wrapper were written with an output constraint on the buffer, GCC might believe the buffer is an output to be regenerated by compiler-generated code, overwriting the character the kernel just deposited. The fix is to capture the return value in a separate local variable and leave the buffer as a plain pointer input. The result is that `EAX` carries the byte count back to the caller and the buffer contains the byte the kernel wrote, with no compiler interference.

`sys_exit` ends with an infinite loop. This tells the compiler the function never returns, suppressing warnings when the startup stub calls it and then reaches what would otherwise look like unreachable fall-through code.

Rewriting the User Programs to Use the Library

With the runtime in place, every user program shrinks to an ordinary C file that declares `int main(int argc, char **argv)`, includes `lib/syscall.h`, and calls the wrapper functions. There is no inline assembly, no `_start`, and no duplicated boilerplate.

The shell undergoes the same transformation. Its original version had six duplicated inline-asm wrappers. Those are deleted, and the old `_start` is renamed to `main`. The shell gains nothing and loses nothing in behaviour, but the code is shorter, and adding a new user program no longer requires copying assembly from an existing one.

The Heap: malloc and sbrk

User programs built so far can only allocate memory from the stack. Stack allocation works for buffers whose size is known at compile time, but it cannot support data structures that grow at runtime — variable-length inputs, linked lists, dynamically sized tables.

The heap fills that gap. Every user process has a heap region that sits between the end of the ELF binary's **BSS** (uninitialised data segment) and the bottom of the user stack. At the moment a process starts, the heap region is empty — no physical pages back it. As the process requests memory, the kernel maps fresh pages into the region on demand. Two addresses per process define the heap's extent:

- **`heap_start`**: the first virtual address of the heap — the page-aligned end of the last loaded ELF segment. Set once when the process is created.
- **`brk`**: the **program break** — the current boundary between the backed heap and the unmapped region above it. Advancing `brk` is how the heap grows.

The mechanism that moves the break is the `SYS_BRK` system call. The kernel now treats `brk()` as an address-space reservation rather than an eager allocation: it validates that the new break stays below the reserved stack window, stores the new value, and returns. Physical pages are committed later, one page at a time, when the process first touches the reserved range and the page-fault handler installs the missing mapping. The `SYS_BRK` contract still mirrors Linux: passing zero queries the current break without changing it; any other value requests a new break; on success the return equals the requested value; on failure the return equals the unchanged old break.

The user-space bridge is `sbrk`. It queries the current break, requests `old_brk + increment`, checks whether the kernel accepted the request, and returns either the start of the newly usable region or `(void *)-1` on failure.

On top of `sbrk`, the user heap implements a first-fit free-list allocator. Every allocation is preceded by an 8-byte block header:

```c
typedef struct {
    uint32_t size;    /* payload bytes, not including this header */
    uint32_t flags;   /* bit 0: 1 = allocated, 0 = free */
} block_hdr_t;
```

Free blocks store a pointer to the next free block in the first four bytes of their payload, threading them into a singly-linked free list. `malloc` walks this list for the first block large enough to satisfy the request. If the leftover space after carving out the requested bytes is large enough to hold a new header and a minimum payload, the tail is split into a new free block. If no existing block fits,

`malloc` calls `sbrk` to extend the heap. `free` prepends the released block to the front of the free list. `realloc` returns the existing pointer unchanged if the block is already large enough, otherwise allocates a new block, copies the data, and frees the old one.

Build System Changes

The build system compiles the runtime library once and links it into every user program. Three components are produced: the startup stub, the syscall wrappers, and the heap allocator. Each user ELF is linked against all three, with the startup stub listed first so `_start` lands at the very beginning of the executable — which is where the ELF entry point must be.

Where the Machine Is by the End of Chapter 20

At this point the user-space boundary is clean and self-contained. Every `int 0x80` instruction in user programs has been moved into the syscall library; all user programs are plain C files with a `main` function. Adding a new user program requires only writing a C file — no boilerplate assembly to copy, no entry-point glue to write.

The `_start` stub receives `argc`, `argv`, and `envp` from the kernel-built stack frame, stores `envp` into the global `environ` pointer so library functions can find it, and forwards all three to `main`. After `main` returns, `_start` passes the return value to `sys_exit`, which transfers control back to the kernel and never returns.

User programs also have access to `malloc`, `free`, `realloc`, and `sbrk`, backed by `SYS_BRK` heap-growth. The heap starts empty; `sbrk` reserves more virtual space as the allocator exhausts its current region, and the CPU's page-fault path commits physical pages lazily on first touch.

Chapter 21 — A Minimal C Standard Library

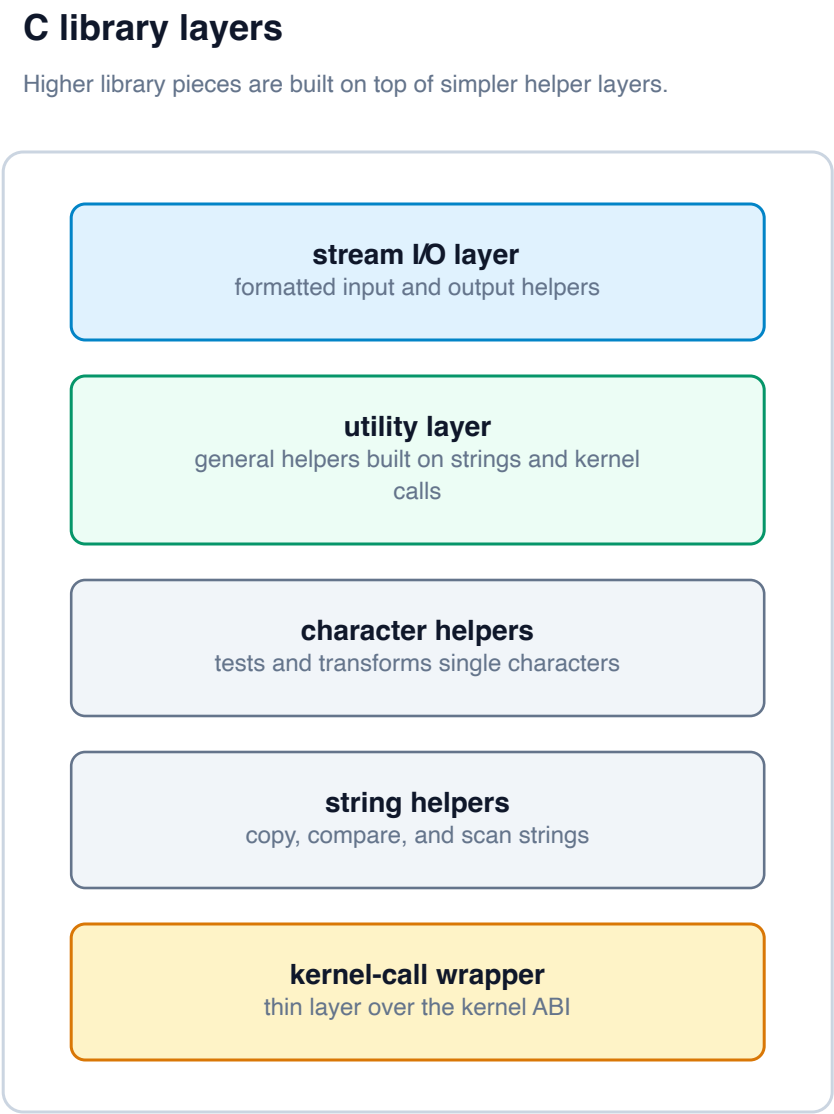
The Foundation a libc Builds On

By the end of Chapter 20, user programs have three things: a startup stub that bridges the kernel's `iret` into `main`, a syscall wrapper library that hides every `int 0x80` behind ordinary C function calls, and a heap allocator backed by the `SYS_BRK` syscall. Those three pieces are enough to write programs that allocate memory and talk to the kernel. They are not enough to write programs that look like ordinary C — programs that call `printf`, compare strings with `strcmp`, classify characters with `isdigit`, or open files with `fopen`.

The **libc** (C standard library) is the layer that provides those familiar names. On a Linux system it is `glibc` or `musl`; our libc implementation sits on top of the raw syscall wrappers and heap allocator.

The Dependency Hierarchy

The modules form a strict layering. Each module is only allowed to call modules below it. That rule prevents circular dependencies and keeps the linker from being surprised.



Chapter 21, Figure 1

The two bottom layers — `string.c` and `ctype.c` — are **pure**: no system calls, no global state, no dependencies on anything but `<stddef.h>` for the `size_t` type. Everything above them may call downward but never sideways or upward.

Why string.c Must Exist at the Bottom

`string.c` is not optional. GCC in `-ffreestanding` mode — the flag we use for all user code — still emits implicit calls to `memcpy`, `memset`, `memmove`, and `memcmp` for struct assignments and local zero-initialisation. A user program that does `struct point p = origin;` will compile into code that calls `memcpy` under the hood. If those four symbols are not linked in, the program fails at link time with an undefined reference — even if the programmer never wrote `memcpy` anywhere in the source. Shipping `string.c` is therefore not just a convenience for string-handling code; it is a

linker requirement for any non-trivial C program.

One implementation subtlety deserves mention: `memmove` must produce the correct result even when the source and destination regions overlap. The straightforward forward copy that `memcpy` uses would overwrite source bytes before they are read when the destination falls above the source in memory. `memmove` detects this case with a pointer comparison and copies backward — from the highest address downward — so each source byte is consumed before the destination pointer reaches it.

Character Classification Without Double-Evaluation Bugs

The standard C library on most systems implements character-classification functions as macros that index into a 256-entry lookup table. That approach is fast but carries a well-known hazard: macros evaluate their argument more than once, so `isdigit(*p++)` would advance `p` twice. We use out-of-line functions instead. For a project of this scale the call overhead is irrelevant, and the single-evaluation guarantee means the functions compose safely with expressions that have side effects.

The classification ranges are defined entirely by the **ASCII** (American Standard Code for Information Interchange) character encoding — to the CPU they are just integers. Because uppercase and lowercase ASCII letters are exactly 32 positions apart, `toupper` subtracts `0x20` from a lowercase letter and `tolower` adds it. No lookup table is needed.

One Format Engine, Multiple Destinations

Every `printf` variant does the same thing: walk a format string, substitute arguments, and send the result somewhere. The "somewhere" varies — `printf` goes to `stdout`, `sprintf` goes into a caller-supplied buffer. A naive implementation would be one copy of the format walker per destination type. The libc here uses one shared format engine that writes to an abstract **sink**.

```
typedef struct {
    char  *buf;    /* chunk buffer (stream mode) or destination buffer (buffer mode) */
    size_t cap;    /* buffer capacity */
    size_t pos;    /* current write position within buf */
    int    total;  /* total characters that would have been emitted */
    int    fd;     /* >= 0: stream mode; < 0: buffer mode */
} pf_sink_t;
```

When `fd >= 0` the sink is in stream mode: output accumulates in a small stack-allocated chunk buffer, and when the chunk fills up the buffer is written to the file descriptor and the position resets. When `fd < 0` the sink is in buffer mode: output accumulates into the caller's buffer up to its capacity, but `total` keeps counting beyond the limit. At the end, `total` is the return value — the number of characters that *would* have been written — which is how C99's `snprintf` contract lets callers size a buffer on a dry run and fill it on a second pass.

The number-to-ASCII conversion writes digits into a temporary buffer *in reverse order* (least significant digit first) using repeated division, then reads the buffer backwards when composing the final field. This avoids needing to know the digit count before starting to write.

The FILE Type and Standard Streams

A `FILE` is a small struct holding a kernel file descriptor and a flags word. The three standard streams are static instances defined inside `stdio.c`:

```
typedef struct {
    int          fd;
    unsigned int flags;
} FILE;
```

The flags word carries direction bits, error and EOF indicators, and a buffering mode. The pointers `stdin`, `stdout`, and `stderr` are global variables that point at those three static instances. Every program that includes `stdio.h` sees them; they are pre-wired to the file descriptors the kernel gives every new process (fd 0 for TTY input, fd 1 for VGA output, fd 2 for unbuffered error output).

`fopen` maps a POSIX mode string to the corresponding kernel file operation: mode `"r"` opens an existing file, mode `"w"` creates or truncates one. It allocates a `FILE` on the heap with `malloc`. Closing a heap-allocated `FILE` closes the underlying descriptor and frees the allocation, but closing one of the three static standard streams is a no-op — they are statically allocated and must never be freed.

The POSIX Wrapper Split

The POSIX wrapper layer is deliberately thin: it provides POSIX names for the same operations already in the syscall library, as one-line forwarding calls. The split serves two purposes. First, it keeps all `int 0x80` inline assembly in exactly one place. Second, it gives programs written to POSIX conventions a header they can include without pulling in any assembly knowledge. The kernel ABI lives entirely below the syscall wrappers.

The same pattern now extends to memory mapping. A tiny adapter layer exposes `mmap`, `munmap`, and `mprotect` as libc-style names while still delegating all register choreography to the raw syscall wrappers. User code therefore gets the familiar interface shape without spreading inline assembly or syscall-number knowledge beyond one file.

One wrapper requires special treatment: `write` must route through the fd-aware kernel write path rather than the old raw-console shortcut. The original console write path ignores file descriptor redirection; the fd-aware path dispatches through the per-process fd table, so `write(1, ...)` correctly follows a `dup2(pipe_fd, 1)` in a pipeline. Any program that calls the POSIX `write` function therefore works transparently inside a shell pipeline.

`isatty` also deserves a note: there is no dedicated kernel syscall for it. Instead, it probes whether an fd refers to a terminal by calling `sys_tcgetattr` and checking whether the call succeeds. If the fd is a TTY the call succeeds; if it is a file or pipe it fails. This is the same technique Linux's glibc uses.

Calendar Time in User Space

The kernel provides exactly one time value: `clock_unix_time()`, accessible from user space through `SYS_CLOCK_GETTIME`. It returns seconds since the Unix epoch (midnight 1970-01-01 UTC). Everything else — converting that timestamp into year, month, day, hour, minute, second; determining

the weekday; formatting it as a human-readable string — happens entirely in user space through `time.c`.

This mirrors Linux's design. The kernel has no concept of months, leap years, or time zones. It just keeps a counter. The C library turns that counter into calendar fields by counting the days from 1970 to the present year (accounting for leap years with the Gregorian rule: divisible by 4 but not 100, or divisible by 400), then counting days within the year to determine the month and day-of-month. No data tables are needed — leap year detection and cumulative month-day counts are computed arithmetically.

`getenv` relies on the global `environ` pointer that the startup stub initialises from the envp word on the initial stack. The function does a linear scan of the `NULL`-terminated `environ` array, comparing each `NAME=VALUE` string's prefix to the requested name. If it finds a match, it returns a pointer directly into that string at the `=` + 1 position. No copy is made; the caller reads directly from the inherited environment block.

Where the Machine Is by the End of Chapter 21

User programs now have a complete, self-contained C runtime with no dependency on any host system library. The strict dependency hierarchy means modules can be linked individually — only the layers a program actually uses end up in the binary. The format engine, the sink abstraction, the POSIX wrapper split, and the in-userspace calendar conversion are all architectural decisions that eliminate duplication, keep assembly in one place, and correctly separate what the kernel knows (a second count) from what libc knows (months, weekdays, time zones). The raw `int 0x80` interface is completely hidden behind the syscall wrappers.

Chapter 22 — The Interactive Shell

What the Shell Is in the Current System

Chapter 21 left us with a complete C runtime library and every user program able to call `printf`, `malloc`, and `fopen` without touching assembly. The shell is an ordinary ring-3 process stored on the filesystem as `bin/shell`. The kernel starts exactly one instance during boot: it looks up `bin/shell` in DUFS, creates a process for it with an initial environment of `PATH=/bin`, and schedules that process first.

After that, the shell stays alive as a parent process. It does not get replaced by child programs and is not automatically relaunched by the kernel after every command exits. Instead it follows the classic read-eval-print loop that every Unix shell has used since 1971: print a prompt, read a command line, execute the command, and repeat. If the shell itself exits and no other process is runnable, the scheduler halts the machine.

The Read-Eval-Print Loop

By the time `_start` hands control to `main`, the kernel has already placed the user at a shell prompt. From that moment on, everything the user sees is driven by one tight loop.

`readline` blocks inside a `SYS_READ` call, waiting for the keyboard driver to produce a character. When a character arrives it is echoed immediately and appended to the line buffer. A backspace removes the last character from the buffer and overwrites it on screen with a space. Page Up and Page Down invoke the VGA scrollback syscalls without disturbing the input buffer. `ETX` (End-of-Text, the byte value `0x03` produced when the user holds Ctrl and presses C) discards the current buffer and re-displays the prompt. When Enter arrives the buffer is returned to the caller as a complete line.

The returned line is tokenised in place. The shell scans for whitespace, replaces each run of it with a null byte, and fills a `char *tokens[]` array with pointers into the original buffer. No heap allocation is involved: each pointer directly addresses a substring of the original line, which is already a valid C string because the null replacement terminates it. The first token is the command name; the rest are its arguments.

The command name is checked against a table of built-in commands. Built-ins run directly inside the shell process — they can modify the shell's own environment table and working directory in ways that an external child process cannot. Any name that is not a built-in reaches the external program launcher, which is described later in this chapter.

Current Working Directory

Every process in the kernel carries a working directory string inside its descriptor, initialised to the root `/`. Two syscalls manage it: one validates a target path and, if it names an existing directory, writes the new path into the process descriptor; the other copies the current path into a user-supplied buffer. Because the working directory lives in the kernel descriptor rather than in user space, it is automatically inherited when a process forks — the child's descriptor is a copy of the parent's, working directory included.

The shell's `cd` built-in calls the first syscall directly. The prompt-generation code calls the second before each prompt to display the current directory. The kernel-side implementation handles

`NULL`, empty strings, and `"/"` (go to root), `".."` (strip the last path component), absolute paths (beginning with `/`), and paths relative to the current directory — all resolved inside the kernel without requiring the shell to perform any path arithmetic itself.

Environment Variables

The shell maintains a private variable table — a fixed-size array where each entry stores a name, a value, an exported flag, and a read-only flag. At startup, it walks the `envp` (environment pointer) array that the startup stub receives from the kernel and populates the table from any `NAME=VALUE` strings already present. The initial environment the kernel provides is `PATH=/bin`.

The exported flag determines whether a variable is passed to child processes. Before launching any child program the shell walks the table and constructs a flat null-terminated `char *envp[]` array from all exported entries. A forked child inherits that vector through the global `environ` pointer, and a subsequent `SYS_EXEC` uses it to build the replacement image's initial environment frame. Because the kernel lays the environment strings out on the new user stack alongside the argument strings (Chapter 15), the program sees a fully populated environment the moment its `_start` runs.

The read-only flag causes the shell to reject subsequent assignments to a variable. Shell-local variables exist in the table but are omitted from the exported `envp[]` rebuild, so they are invisible to children.

PATH-Based Command Resolution

When the user types a name that does not match any built-in, the shell searches for a corresponding **ELF** (Executable and Linkable Format, the binary format described in Chapter 15) file on disk. The search follows the standard `PATH` algorithm.

If the name contains a `/`, it is treated as a literal path and probed directly. Otherwise, the shell splits the `PATH` variable on `:` to obtain a list of directories and, for each directory, concatenates the directory path with the command name and probes whether the result exists. The first match wins.

Because the kernel initialises `PATH=/bin`, all programs installed in `/bin` on the DUFS image are reachable by name without qualification. `PATH` is a regular environment variable, so a user can extend it with additional directories the same way any other variable is set.

Running an External Program

Launching an external program requires four kernel operations in sequence.

First, `SYS_FORK` clones the shell. The child resets its foreground-job signal handlers to defaults so Ctrl+C and Ctrl+Z affect the program it is about to run rather than bouncing back into shell prompt logic.

Second, the child calls `SYS_EXEC` with the path, argument vector, and exported environment. The kernel loads the **ELF** binary from disk, allocates a fresh page directory for it, maps its segments into user space, rebuilds the initial user stack, and replaces the child process image in place. The child's PID does not change.

Third, the shell calls `SYS_SETPGID` to place the child in its own **process group** — a collection of processes that share a common group identifier, the **PGID** (Process Group ID). This is the foundation of job control: signals sent to a PGID reach every process in the group simultaneously, and the TTY driver directs keyboard input only to the group that currently holds the terminal foreground.

Fourth, the shell calls `SYS_TCSETPGRP` to hand the terminal to the child's process group. The TTY's foreground PGID is updated to the child's; from this moment Ctrl+C and Ctrl+Z signals are directed at the child rather than the shell. The shell then calls `SYS_WAITPID` and blocks until the child either exits or stops.

When the child terminates, the shell reclaims the terminal by calling `SYS_TCSETPGRP` with its own PGID before printing the next prompt.

Pipe Support

The shell supports a single `|` operator connecting two external programs. After tokenising the line the shell scans for a `|` token; when found, everything to the left is the writer command and everything to the right is the reader command.

To connect them, the shell first asks the kernel to allocate a **pipe** — a unidirectional in-kernel ring buffer with two **fd** (file descriptor) handles: a read end and a write end. It then forks twice.

The left child replaces its standard output (fd 1) with the write end of the pipe using `SYS_DUP2`, closes the raw pipe fds, and execs the left command. The right child replaces its standard input (fd 0) with the read end using `SYS_DUP2`, closes its pipe fds, and execs the right command. The parent closes both pipe ends.

Closing the write end in the parent is the critical detail. When the left child exits, both the child's copy and the parent's copy of the write end are closed. The right child then sees **EOF** (End of File — the condition that no further data will ever arrive) on its read end and terminates naturally, rather than blocking forever waiting for input that can never come.

Both children run in the same process group so a Ctrl+Z stops the entire pipeline together. `SYS_EXEC` preserves the calling process's fd table, so the `dup2` redirection performed in the forked child before exec is transparently visible to the exec'd program.

Job Control

The shell maintains a job table of up to eight entries. Each entry records a child PID, its stopped or running state, and the original command name. The built-in job-management commands (`jobs`, `fg`, `bg`) operate on this table.

When the shell starts, it establishes itself as its own process group leader, claims the foreground TTY for that PGID, and installs prompt-time handlers for `SIGINT` and `SIGTSTP`. That means Ctrl+C and Ctrl+Z are real signals even at the prompt; the shell's handlers simply abandon the current input line and redisplay the prompt instead of terminating or stopping the shell itself.

When the shell launches a foreground child and calls `SYS_TCSETPGRP`, the TTY's foreground PGID changes to the child's. Now Ctrl+C sends `SIGINT` to the entire child process group and Ctrl+Z sends `SIGTSTP`. If the child stops due to `SIGTSTP`, `SYS_WAITPID` returns with a stopped status. The

shell detects this — the status word encodes whether the child exited or was merely stopped — adds the child to the job table, reclaims the terminal for its own PGID, and re-prompts. The stopped child can be resumed in the foreground with `fg` or continued in the background with `bg`.

Prompt and ANSI Colours

Before printing each prompt the shell queries the kernel for the current working directory and assembles the prompt string using **ANSI** (American National Standards Institute) terminal escape sequences. An ANSI escape sequence is a byte string beginning with the ESC character (`0x1B`) followed by `[`, a numeric attribute code, and `m`. The VGA console driver interprets these sequences, applying the requested colour to subsequent characters until a reset sequence is encountered.

The shell prints the hostname in cyan, the working directory in green, and the `>` separator in cyan, then resets the colour before the cursor — keeping the user's typed input in the default white-on-black style rather than inheriting the prompt colour.

Tab Completion

When the user presses Tab, the shell treats it as a completion request rather than echoing the character. The completer examines the buffer to find the word being completed — the contiguous run of non-space characters that ends at the cursor — and dispatches to one of two strategies.

If the word is the first token on the line (no space has been typed yet), the completer performs **command completion**: it searches a table of all built-in names, then walks each directory in `$PATH` and enumerates its files via `SYS_GETDENTS`, collecting every entry whose name starts with the typed prefix.

If at least one space precedes the word, the completer performs **path completion**: the word is split at its last `/` to obtain a directory component and a name prefix. `SYS_GETDENTS` is called on that directory (or on the process's working directory when no `/` is present), and every entry that starts with the name prefix is collected as a candidate. Directory entries are returned by the kernel with a trailing `/`, which is preserved in the match so the user can see immediately whether a completion names a file or a subdirectory.

Once the match list is assembled, the completer picks a course of action based on how many candidates were found:

Matches	Action	Result
`0`	Ring bell	There is nothing to complete
`1`	Insert suffix	The word is completed immediately
`2+`	Longest prefix or list	The word extends if possible, or choices are shown

The longest-common-prefix step mirrors the behaviour of every POSIX shell: the first Tab extends as far as possible without guessing, and the second Tab (or a Tab when no extension is possible) reveals the ambiguous options.

All of this happens entirely inside the shell's line editor — no new syscalls are involved. The completer writes directly to standard output using the same output path the rest of the line editor uses for echo and backspace, keeping the screen state consistent with the buffer at every step.

Where the Machine Is by the End of Chapter 22

We now have a real interactive parent process with job control, environment management, and PATH-based command resolution. The shell can navigate an arbitrary directory tree, launch external programs with argument vectors and an inherited environment, connect two programs through a kernel pipe, and suspend and resume jobs with terminal handoff. Every user-visible action goes through the syscall layer, making the shell the place where the process, filesystem, driver, environment, and console subsystems all meet for the first time.

Part VII — Extending the Kernel

At the end of Part VI the OS is complete in the sense that matters to a user: programs run, the shell is interactive, files persist across reboots. But the kernel is a closed system. Adding new functionality — a driver for a new device, a new filesystem, a new system call — means modifying kernel source code, recompiling the entire kernel, and rebooting the machine. For a developer iterating on kernel components, or for a system that needs to load drivers on demand without restarting, that is an unacceptable constraint. And when something does go wrong — when a program crashes — there is currently no record of what the machine state looked like at the moment of the fault. The process disappears cleanly, and whatever went wrong with it disappears too.

This part addresses both the closed nature of the compiled kernel and the opacity of process crashes. The two problems share a common thread: both require the kernel to understand ELF binaries — the standard executable and object file format — more deeply than just loading their program segments. A loadable module is an ELF relocatable object (a `.o` file, the kind a compiler produces before linking) with unresolved symbol references that the kernel must patch at load time by looking up the target addresses in a table of exported kernel symbols. A core dump is an ELF core file — a binary the kernel *writes*, not reads — whose structure must match what a debugger like GDB expects so it can reconstruct the exact state of the crashed process without any assistance from the running system.

Chapter 23 builds the runtime module loader: the ELF relocatable section parser, the two relocation types the x86 calling convention requires (one for absolute address references and one for relative ones), the kernel symbol export table, the `module_init` call that runs after relocation completes, and the `SYS_MODLOAD` syscall that user programs use to trigger a load. Chapter 24 builds the core dump writer: the ELF core file layout, the register-state note that carries the full CPU register snapshot at the moment of the fault, the process-info note that carries the process name and command line so GDB can identify the binary automatically, and the memory segments that snapshot every user-space page. The process name and argument string are captured at process creation time so they survive whatever the program may have done to its own stack before crashing.

By the end of Part VII the kernel can grow without recompilation, and crashed processes leave behind a complete, analysable record. The gap between the kernel as a static artifact and the kernel as a living, inspectable system is closed.

Chapter 23 — Runtime Loadable Modules

Extending the Kernel at Runtime

Chapter 22 left us with a working interactive shell that can launch ELF programs, manage jobs, and pipe output between processes. Every built-in driver is still linked into the kernel image, though. We add one extension point: user space can point the kernel at a relocatable ELF object already stored in DUFS, and we can load and execute it at runtime without rebuilding the kernel image.

The Input Format

We accept ordinary 32-bit relocatable ELF objects — the `.o` files that the compiler produces before a linker combines them into an executable. Before doing any work we validate four conditions: the file begins with the four-byte ELF magic sequence (`0x7F 'E' 'L' 'F'`), the object type field is `ET_REL` (the ELF type constant for a relocatable object, as opposed to an executable or shared library), the machine field is `EM_386` (the ELF architecture constant for 32-bit x86), and the file size does not exceed `MODULE_MAX_SIZE`.

The load request starts like any other file access: the kernel resolves the module path through the **VFS** (Virtual File System), identifies the backing inode and file size, and reads the bytes from DUFS into kernel memory. The module loader is therefore not a side door around the filesystem; it consumes the same on-disk objects the rest of user space sees.

How a Module Is Loaded

Once the bytes are in memory, the kernel reads the ELF header and section-header table so it can understand the object's layout before committing any permanent runtime memory.

The first pass over the section-header table is a sizing pass. Every section marked `SHF_ALLOC` — the ELF section flag that means "this section must occupy memory at runtime" — contributes its size and alignment requirement to a running total. Text and read-only data sections, the initialized data section, and any zero-filled BSS regions all carry this flag. Non-allocatable sections such as debug information and the relocation records themselves do not. Once the total is known, we call `kmalloc` once for the whole module and zero-fill the resulting buffer. Zero-filling is important because BSS sections are never stored on disk; the ELF format simply records their required size and expects us to provide zeroed memory.

The second pass copies each allocatable section into its calculated position inside the module buffer, records the runtime address at which each section landed, and reads the symbol table. The symbol table lists every name the object defines or references — local functions, global data, and any external names the module expects the kernel to supply. Defined symbols are resolved first: we add each section's base address to the symbol's section-relative offset, so every locally defined function and variable gets its runtime address in the kernel heap.

Undefined symbols — those the object references but does not define — are matched against the kernel export table. That table is a fixed array of `(name, address)` pairs listing every kernel function a module is allowed to call. The matching is a linear scan by name. Any symbol that appears in the object's symbol table but not in that export table is a hard failure: the load aborts and the module buffer is freed. This bounded visibility is our small-scale equivalent of Linux's

`EXPORT_SYMBOL` mechanism — we don't expose every internal function automatically, and modules cannot reach symbols that were not explicitly published.

Applying Relocations

A relocatable object produced by the compiler contains placeholder values wherever it references a symbol whose final address is not yet known. A **relocation record** tells us exactly where one of these placeholders sits, which symbol's address should go there, and what arithmetic to apply. We handle the two relocation types that a simple C module produces.

An `R_386_32` record describes a 32-bit absolute address reference. We add the symbol's runtime address to whatever value the compiler placed at the patch site — typically zero — and write the result back. A `call` or `jmp` to an absolute address uses this form.

An `R_386_PC32` record describes a 32-bit PC-relative address reference — the signed displacement that an `E8 call` instruction encodes. We add the symbol's runtime address and then subtract the address of the patch site itself, producing the offset the CPU computes at execution time. Encountering any other relocation type is a hard failure; we have no arithmetic for it and refusing to continue is safer than leaving a partially patched module in memory.

Memory Layout and Lifetime

All allocatable sections for one module are packed into a single contiguous heap allocation. Because this is a 32-bit kernel whose heap pages are executable, we don't need to create a separate executable mapping — text sections land in the same buffer as data sections and the CPU can fetch instructions from them directly.

Once the relocation pass completes without error, we look up the symbol named `module_init` in the resolved symbol table and call it. If `module_init` returns zero the load is considered successful; a non-zero return signals that the module itself detected a problem during its own initialization.

There is no unload path. Once `module_init` succeeds, the heap allocation remains resident for the life of the kernel. A module that registers itself with a device registry or interrupt table cannot be safely removed without knowing whether any code currently running holds a pointer into the module's text or data — a reference-counting problem we deliberately defer. We also record a small `(name, base, size)` descriptor for each successful load so `/proc/modules` can enumerate the live module set.

How User Space Requests a Load

User space reaches the loader through the `SYS_MODLOAD` syscall (number 170). The shell's built-in `modload` command names a file that already exists on disk; the kernel validates that file, loads it into memory, resolves its symbols, applies relocations, and finally runs its initialisation entry point:

Chapter 23, Figure 1

The return value travels back through the syscall frame to the caller. A zero means the module is live; any negative value means something failed before or during `module_init`.

What a Module Can Do

A successfully loaded module can call any function in the export table and register itself with any kernel registry that table exposes. In practice that means a module can add new block devices under new names, register character devices, provide a new filesystem implementation, or install an interrupt handler — all without rebuilding the kernel image. The constraint is the export table: if a kernel function is not listed there, the module cannot reach it, and the load fails at symbol resolution rather than at runtime.

Where the Machine Is by the End of Chapter 23

We are no longer limited to code that was linked at build time. A user process can ask us to load a relocatable ELF object from DUFS, parse its section and symbol tables, resolve its external references against the exported-symbol table, apply the two relocation types that 32-bit x86 C code generates, and call its entry point — all inside a single `SYS_MODLOAD` call. The module lands in a single heap allocation, its text sections are immediately executable, and it becomes a permanent part of the running kernel for as long as the machine stays up.

Chapter 24 — Core Dumps

The Moment the Process Can No Longer Continue

Chapter 23 left us with runtime-loadable modules that extend the kernel without rebuilding the kernel image. When a fatal signal arrives and the process has not installed a handler for it, we don't simply discard the process and move on. If the signal is one of the terminating signals — SIGILL (illegal instruction), SIGSEGV (segmentation fault, sent on a bad memory access), or any unhandled SIGTERM or SIGKILL — we have enough information to write a complete snapshot of the crashed process's state to disk before cleaning up. That snapshot is a **core dump**: an ELF file containing the process's entire user-space memory and the CPU register state at the moment of the fault.

The point of the file is post-mortem debugging. Load it into GDB alongside the original binary and you can see exactly where execution was, what the stack looked like, and what values every register held. On Linux this has been standard for decades. We produce core files in the same format, so the same tools work without modification.

When a Core Dump Fires

Signals deliver core dumps through the normal signal-delivery path described in Chapter 19. When that path finds a deliverable signal whose disposition is `SIG_DFL`, it determines whether the signal's default action is termination. If the process has a saved crash frame (set earlier by the exception handler when the fault first occurred), the kernel writes the core file before transitioning the process to the zombie state.

The crash frame is a copy of the full interrupt frame saved at the moment the CPU faulted. The exception handlers detect a ring-3 fault, copy the frame into the process's crash record, and set a valid flag before sending the appropriate signal. The signal is then delivered on the next scheduler pass, which is when the kernel still has both the crash context and the intact user address space available to write out.

The ELF Core File Format

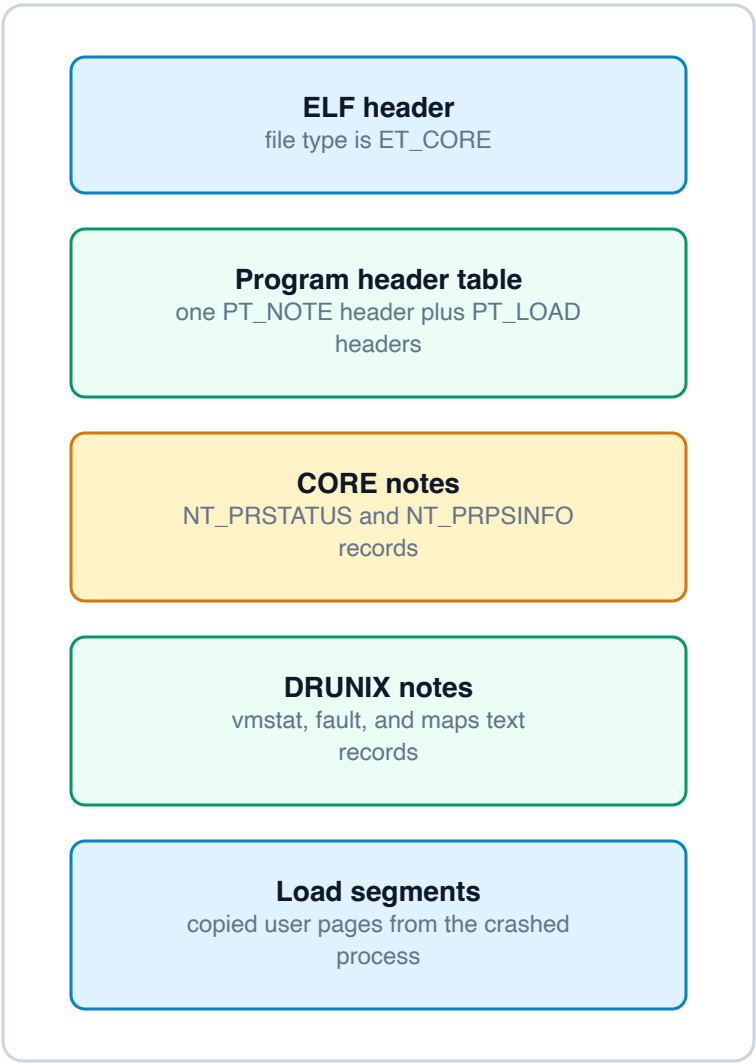
A core file is an ELF — Executable and Linking Format — binary with type `ET_CORE` rather than the usual `ET_EXEC` or `ET_DYN`. ELF is a standard binary format used throughout Linux and most Unix systems; its header describes what the file contains and where each section lives.

An `ET_CORE` file carries two kinds of program headers:

- **PT_NOTE** — a single segment pointing to a block of metadata notes. Each note describes some aspect of the process: the signal that killed it, the register state, the process name, the command-line arguments, and the process group it belonged to.
- **PT_LOAD** — one segment per contiguous region of user-space memory. Each describes a range of virtual addresses, its file offset, and whether the region was readable, writable, or executable.

A debugger reconstructs the process's address space by mapping each `PT_LOAD` segment to its virtual address and reading the process metadata from the `PT_NOTE` segment.

The overall file layout is:



Chapter 24, Figure 1

The notes follow the program headers directly. The memory dumps are pushed past the next page boundary from the end of the notes so that each `PT_LOAD`'s file offset is page-aligned — this makes it possible for a debugger to `mmap` each segment directly if it chooses.

ELF Notes

An ELF note section is a sequence of variable-length records. Each record starts with a fixed header:

```
typedef struct {
    uint32_t n_namesz;  /* length of the name string including NUL */
    uint32_t n_descsz;  /* length of the descriptor data           */
    uint32_t n_type;    /* note type                               */
} Elf32_Nhdr;
```

The header is followed by the name string (zero-padded to a 4-byte boundary), then the descriptor data (also zero-padded to a 4-byte boundary). We write two notes under the name `"CORE"` and three additional notes under the name `"DRUNIX"`.

NT_PRSTATUS (type 1) is the register state note. Its descriptor is a 144-byte structure that matches Linux's `elf_prstatus` exactly:

```
typedef struct __attribute__((packed)) {
    core_siginfo_t pr_info;    /* signal number, code, errno  */
    uint16_t       pr_cursig;  /* current signal              */
    uint16_t       pr_pad0;
    uint32_t       pr_sigpend; /* pending signal bitmask      */
    uint32_t       pr_sighold; /* blocked signal bitmask      */
    int32_t        pr_pid;
    int32_t        pr_ppid;
    int32_t        pr_pgrp;
    int32_t        pr_sid;
    core_timeval_t pr_utime;   /* user time (zeroed)          */
    core_timeval_t pr_stime;   /* system time (zeroed)        */
    core_timeval_t pr_cutime;
    core_timeval_t pr_cstime;
    elf_gregset_t  pr_reg;     /* 17 general-purpose registers */
    int32_t        pr_fpvalid;
} core_prstatus_t;
```

The seventeen general-purpose registers in `pr_reg` are written in a specific order that matches the Linux i386 `user_regs_struct` layout, which is what GDB expects:

Index	Register
0	EBX
1	ECX
2	EDX
3	ESI
4	EDI
5	EBP
6	EAX
7	DS
8	ES

Index	Register
9	FS
10	GS
11	orig_eax (not tracked; set to 0xFFFFFFFF)
12	EIP
13	CS
14	EFLAGS
15	user ESP
16	SS

These values are drawn directly from the saved crash frame (`proc->crash.frame`), which was copied from the CPU's interrupt frame before any cleanup took place.

NT_PRPSINFO (type 3) is the process information note. Its descriptor is a 124-byte structure matching Linux's `elf_prpsinfo` for 32-bit targets:

```
typedef struct __attribute__((packed)) {
    char     pr_state;      /* numeric process state (0=R, 1=S, 3=Z, 4=T) */
    char     pr_sname;      /* state as a letter: R, S, Z, or T            */
    char     pr_zomb;       /* 1 if zombie, 0 otherwise                    */
    char     pr_nice;       /* nice priority value (0)                     */
    uint32_t pr_flag;       /* process flags (0)                           */
    uint16_t pr_uid;
    uint16_t pr_gid;
    int32_t  pr_pid;
    int32_t  pr_ppid;
    int32_t  pr_pgrp;
    int32_t  pr_sid;
    char     pr_fname[16];  /* executable basename                         */
    char     pr_psargs[80]; /* first 79 chars of the command line          */
} core_prpsinfo_t;
```

When GDB opens a core file, it reads `pr_fname` to identify the binary and `pr_psargs` to show the command that was running. Without this note the debugger can still read registers and memory, but it cannot automatically find the matching executable or show you what arguments the program was called with.

DRUNIX Text Notes for Memory Forensics

The core writer also emits three text-backed `DRUNIX` notes that mirror the live `/proc/<pid>/` memory-forensics views:

- `DRUNIX` note type `0x4458564d` (`"DXVM"`) carries the same compact per-process totals shown by `/proc/<pid>/vmstat`.
- `DRUNIX` note type `0x44584654` (`"DXFT"`) carries the same fault snapshot shown by `/proc/<pid>/fault`.
- `DRUNIX` note type `0x44584d50` (`"DXMP"`) carries the same detailed virtual-memory map text shown by `/proc/<pid>/maps`.

These notes are generated from the same internal process-memory model used by `procfs`, then copied directly into the ELF `PT_NOTE` payload during core creation. That alignment gives us a stable live/post-mortem bridge: while a process is running, `/proc/<pid>/vmstat`, `/proc/<pid>/fault`, and `/proc/<pid>/maps` show current state; after a crash, the core file preserves the same views as frozen text notes.

Capturing the Command Line at Launch Time

The process name and command-line string cannot be recovered from memory after a crash reliably — the process may have corrupted its own stack, or `argv` may have been overwritten by the program's own data. We therefore copy them into the process descriptor at creation time, before the process ever gets to run.

During process creation, after the ELF image has been loaded and before control returns to the caller, the kernel does two things. First, it extracts the executable basename from `argv[0]` by scanning backward for the last `/` character and copying the tail into `proc->name`, capped at 15 characters plus a NUL terminator. Second, it assembles the full argument list into `proc->psargs` by concatenating all `argv[]` strings with single spaces between them, stopping at 79 characters. Both fields live in the process descriptor permanently, surviving any subsequent modification of the user-space stack.

The `name` field was introduced earlier for kernel logging; `psargs` was added alongside the `NT_PRPSINFO` note to give debuggers the command line at the moment of the crash.

Walking User-Space Memory

For the `PT_LOAD` segments, we need to find every contiguous region of user-space memory that was mapped at the time of the crash. The page directory and page tables are still intact — the process has not been destroyed yet, only marked for termination. We walk the process's page directory looking for user-accessible, present pages, and group adjacent pages with the same permission flags into a single segment.

The permission flags on each segment reflect the page table entry (PTE) flags: a read-only code region produces a `PF_R` segment; a read-write data or stack region produces `PF_R | PF_W`. This matters for debugging: GDB uses the flags to distinguish code from data and to know which regions it can safely disassemble.

One boundary condition matters here: we enforce that only pages below `USER_STACK_TOP` are included. Pages above that address are kernel-mapped and never written to the core file, both for security and because the debugger has no use for kernel virtual addresses.

Writing the File

The core file is written to the process's current working directory with the name `core.<pid>`. The full path is assembled as `<cwd>/core.<pid>` when the process has a working directory, or just `core.<pid>` at the filesystem root. The file is created through the VFS layer, and all subsequent writes go through the ordinary filesystem write path on the resulting inode.

The write proceeds in four passes:

1. Write the ELF header.

2. Write all program headers — first the `PT_NOTE` header, then one `PT_LOAD` header for each user-space segment identified during the walk. All program headers are written before any data, because the program header table must immediately follow the ELF header and its total size is known once the segment count is known.
3. Write the notes block at the fixed `note_off` offset (immediately after the program header table). The block begins with the two Linux-compatible `CORE` notes — `NT_PRSTATUS`, then `NT_PRPSINFO` — and then appends the three `DRUNIX` text notes for `vmstat`, `fault`, and `maps` in that order.
4. Walk user-space memory a second time and write each segment's page data at the file offset recorded in its `PT_LOAD` header.

After all data is written, the filesystem's flush step ensures the data reaches the on-disk image before the kernel returns.

Where the Machine Is by the End of Chapter 24

When a process dies from an unhandled fatal signal and a crash frame was recorded, we now write a fully-formed ELF core file to disk before discarding the process. The file contains two Linux-compatible `CORE` notes — `NT_PRSTATUS` with register and signal context, and `NT_PRPSINFO` with process identity — plus three `DRUNIX` text notes mirroring `/proc/<pid>/vmstat`, `/proc/<pid>/fault`, and `/proc/<pid>/maps`. These are followed by the raw bytes of every user-space page that was mapped at the moment of the fault.

The `process_t` descriptor carries two new fields — `name` and `psargs` — populated at process-creation time so the information is available regardless of what the process did to its own stack before crashing. The file format is layout-compatible with Linux's 32-bit core format, so standard tools such as GDB can load the file and restore the full crash context without any modification.

Part VIII — Fault-Driven Memory

Part II built the first version of memory management: a bitmap-backed physical allocator, a fixed identity map, and enough paging support to launch a user process with its code, data, heap start, and a four-page stack. That was enough to get ring-3 code running, but it also froze the memory story in its simplest form. A larger heap immediately consumed physical memory even if the program never touched it. `fork()` copied every user page up front even when the child was about to `exec()` something else. A write to a shared page could not be expressed as "copy this page now"; the kernel had no place where such a decision could even be made.

The missing piece is the page-fault path. On x86, the CPU does not only use page faults to report bugs. It also uses them to report that translation state is incomplete and ask the kernel to finish the job. A not-present fault can mean "this heap page has never been committed"; a protection fault on a writable mapping can mean "this page is copy-on-write, give this process a private copy"; and a fault just below the user stack can mean "the stack needs one more page". Once the kernel can distinguish those cases, a fault becomes part of normal execution rather than the end of it.

Chapter 25 builds that decision path: how the CPU reports the fault through `CR2` and the page-fault error code, how the kernel turns a missing heap or stack page into a live mapping, and how the old SIGSEGV path remains the fallback for wild pointers and true protection violations. Chapter 26 then uses the same machinery to make `fork()` cheap in the common case by sharing pages until the first write, while the physical memory manager keeps a per-frame reference count so shared pages can be reclaimed safely.

By the end of Part VIII, the memory subsystem stops being a static mapping phase that happens only at process creation time. It becomes a live conversation between the CPU, the page tables, and the kernel's policy code.

Chapter 25 — Page Faults and Demand Paging

The Moment a Missing Mapping Stops Being a Crash

By the end of Chapter 24 the kernel can dump a crashed process's memory to disk, but it is still treating every ring-3 page fault as a terminal event. That was acceptable when all user pages were allocated eagerly: if the CPU faulted, the mapping was either invalid or forbidden, so the right answer really was SIGSEGV. The moment the heap becomes lazy and the user stack is allowed to grow, that assumption breaks. A fault is no longer always a bug. Sometimes it is the CPU's way of saying, "this virtual address is part of the process, but the page tables are not finished yet."

This chapter turns the page-fault path into a normal part of execution. The process asks for a larger heap with `brk()`, but the kernel does not allocate any physical memory yet. The process reaches into that heap later, the CPU faults because the page is not present, and the kernel allocates exactly one page, installs exactly one mapping, and retries exactly the same instruction. The same thing now happens when a deep call chain reaches the bottom of the four-page stack and needs one more page underneath it.

This is the shape Linux uses too: not every page fault means "kill the process". Some faults mean "finish building the mapping and continue".

What the CPU Reports on a Page Fault

When the x86 CPU raises vector 14, it does two extra things that no other exception provides in quite the same way.

First, it writes the faulting linear address into **CR2** (Control Register 2, a CPU register dedicated to remembering the address that triggered the most recent page fault). The kernel reads `CR2` in `isr_handler` before doing anything else, because any later page fault would overwrite it.

Second, it pushes a page-fault error code whose low bits tell the kernel what kind of translation failed. The three bits that matter are:

Bit	Name	Meaning when set
0	`P`	The page was present, so the fault is a protection failure rather than a missing mapping
1	`W`	The access that faulted was a write
2	`U`	The fault came from ring 3 rather than ring 0

The CPU packages those bits into the saved trap frame, and the exception handler reads them back through `f->error_code`.

The combination is what turns a raw fault into a policy decision. `P=0, U=1` means "user mode touched an address whose translation is missing". `P=1, W=1, U=1` means "user mode tried to write through a present but read-only mapping". The first case may be a lazy heap or stack page. The second case may be a copy-on-write promotion. Anything else is usually a real protection bug.

The New Stop in the Fault Path

The interrupt path itself changes in only one place. Ring-3 page faults now get one chance to recover before the signal path runs. Kernel faults still print state and halt. User faults that are truly invalid still become `SIGSEGV`. The new possibility is that some user faults are now recognised as incomplete-but-valid memory accesses.

The sequence is:

1. The CPU faults on a ring-3 memory access and jumps to vector 14.
2. The kernel saves the register frame and reads the faulting address from `CR2`.
3. The page-fault handler classifies the access and checks whether the address belongs to a valid user mapping.
4. If the fault is recoverable, the kernel installs or repairs the mapping and returns from the exception.
5. The CPU retries the exact instruction that faulted, this time against the updated page tables.
6. If the fault is not recoverable, the old `SIGSEGV` path runs unchanged.

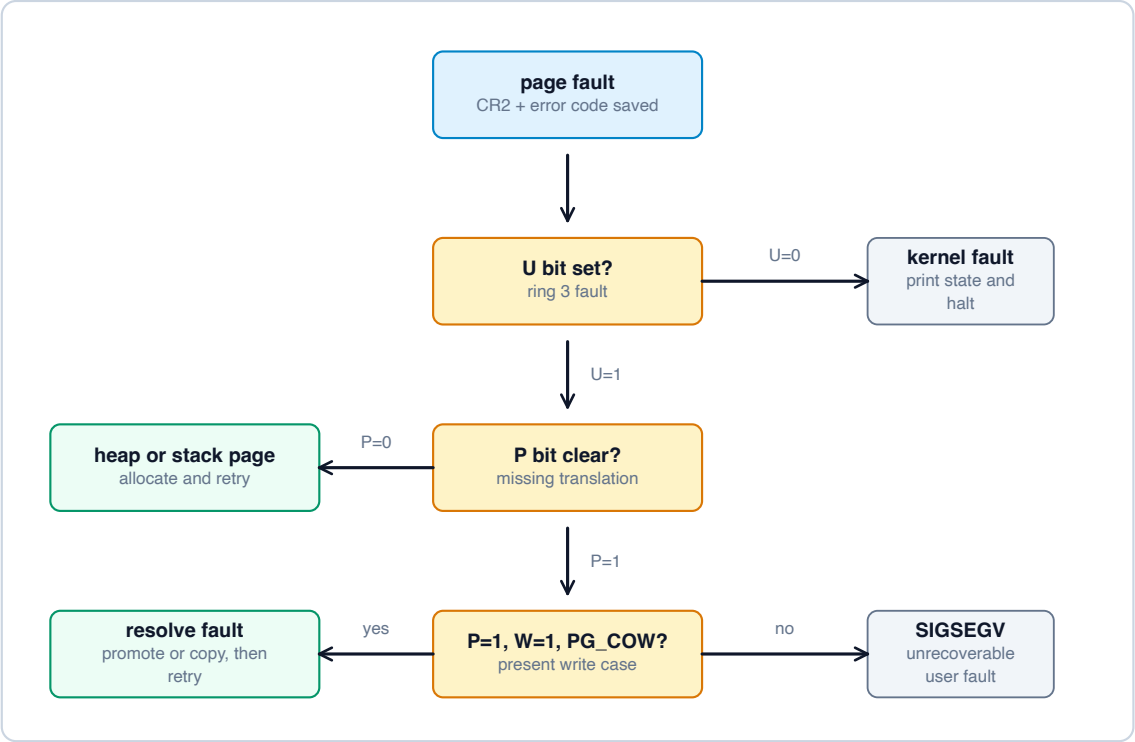
The important point is that the retry is free. There is no second special control transfer for the resumed instruction. `iret` restores the saved `EIP`, `CS`, and `EFLAGS`, and the CPU simply fetches the same instruction again.

The Decision Tree

The page-fault handler starts from one question: was this a missing translation, or was it a write through a present but non-writable mapping?

Page-fault decision tree

Recoverable user faults allocate heap or stack pages, or resolve copy-on-write, before the instruction is retried.



Chapter 25, Figure 1

The first split is the `U` bit. A page fault raised in ring 0 is still fatal here. The kernel has no recovery path for faults taken while it is executing on its own behalf, so the function returns failure immediately and the old exception path halts the CPU after printing the register dump.

The second split is the `P` bit:

- `P = 0` means the CPU could not complete the page walk because some PDE or PTE was not present.
- `P = 1` means the page walk succeeded, but the permissions on the resulting PTE rejected the access.

That distinction is what lets one handler implement both demand paging and copy-on-write without confusing the two.

Heap Pages Are Now Committed on First Touch

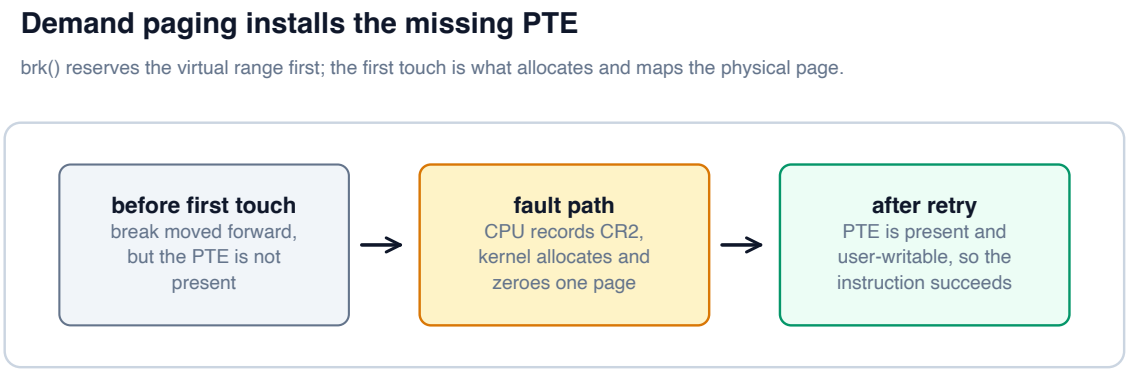
`SYS_BRK` used to do all of its work eagerly. If a process moved its heap break 10 MB forward, the syscall immediately allocated and zeroed 2,560 physical pages and mapped all of them before returning. That made `brk()` expensive and meant an untouched heap reservation consumed physical memory it never used.

The new `SYS_BRK` path now does almost nothing on growth. It checks that the requested break

stays inside the legal heap window, records the larger reservation, and returns. No physical page is allocated. No PTE is installed. The virtual address range between the old break and the new break becomes *valid in principle* but still unmapped in the page tables.

The first real access happens later. Suppose a process calls `sbrk(4096)` and then writes to the first byte of the newly reserved range. The CPU performs the page walk, discovers that the PTE is missing, writes the faulting address into `CR2`, and raises vector 14. The fault handler rounds that address down to a 4 KB page boundary, confirms that it lands inside the process's writable heap reservation, allocates one physical page from the physical memory manager, zeros it, maps it with `PG_PRESENT | PG_WRITABLE | PG_USER`, and returns success.

The mapping step itself invalidates the stale **TLB** (Translation Lookaside Buffer, the CPU's cache of recent virtual-to-physical translations) entry for that virtual page with `invlpg`, so the retried instruction will see the new PTE immediately rather than a cached "not present" result.



Chapter 25, Figure 2

The result is true demand paging for the heap: the address space grows at `brk()` time, but physical memory is committed page by page as the process actually touches it.

Shrinking the Heap Now Frees Only What Exists

Heap shrink takes the opposite path. When the break moves backward, the kernel walks the affected virtual page range. For each present PTE in that range, it decrements the frame's reference count, clears the PTE, and invalidates the TLB entry.

What matters here is that the loop skips any page that was never touched. In the old eager model, every heap page existed, so shrinking had to free a dense run of physical pages. In the new model, a 10 MB heap reservation may contain only two present PTEs because the program only touched two pages. Shrink removes exactly those two mappings and leaves the untouched holes alone because there is nothing to free.

This is why the new demand path and the refcounted physical allocator belong together. Unmapping a heap page is not always "free this frame immediately". If the page is shared for some other reason, the right operation is "drop this mapping's reference". The physical memory manager handles the final transition to free when the count reaches zero.

A Small VMA Table Now Defines What Counts As Valid Memory

The page-fault handler no longer hard-codes "heap versus stack" by checking only raw address ranges. Each process now carries a small sorted VMA table describing the user regions the kernel intentionally created:

- the `brk()` heap VMA,
- the reserved grow-down stack VMA, and
- any later anonymous private mappings created with `mmap()`.

Each VMA stores a half-open address range, a kind (`heap`, `stack`, or generic mapping), and permission flags such as readable, writable, executable, anonymous, private, and grow-down. Process creation seeds the table with the initial heap and stack entries. `SYS_BRK` adjusts the heap VMA's end. `SYS_MMAP`, `SYS_MUNMAP`, and `SYS_MPROTECT` add, split, remove, or retag generic VMAs without touching the ELF image metadata.

This matters in the fault path because the first policy question is now "which VMA contains `CR2`?" If the answer is "none", the fault is still invalid and falls through to `SIGSEGV`. If the answer is a writable anonymous VMA, the handler can allocate and map a zeroed page on demand. There is one extra wrinkle now: because every process still inherits the kernel's low 128 MB identity map, an untouched anonymous address may fault as a present supervisor-only mapping rather than as a clean not-present miss. The handler explicitly detects that shadow-mapping case and treats it like the same lazy anonymous fault, replacing the inherited kernel-only PTE with a real user mapping. If the answer is a present but non-writable VMA and the PTE carries `PG_COW`, the same lookup confirms that the write is allowed in principle before the copy-on-write promotion proceeds.

The User Stack Can Grow Below the Current Bottom

The user stack still starts the same way it did in Chapter 15: the kernel maps four pages just below `USER_STACK_TOP` so that `argv`, `envp`, and the first few nested function calls have somewhere to live before the process executes any user code.

The difference is that the process descriptor now records `stack_low_limit`, the lowest currently mapped stack page, and the address-space contract reserves a larger stack window: up to `USER_STACK_MAX_PAGES`, which is 64 pages or 256 KB. `USER_HEAP_MAX` is moved upward accordingly, so the heap must stop below the entire potential stack window rather than below the four pages that happen to be mapped at process start.

Stack growth stays in the page-fault handler. On a not-present ring-3 fault, after the heap test fails, the kernel computes two bounds:

- the lowest legal page in the reserved stack window, `USER_STACK_TOP - USER_STACK_MAX_PAGES * PAGE_SIZE`
- a 32-byte slack below the saved user `ESP`, or zero if `ESP` is already near the bottom of the address space

The 32-byte slack is deliberate. On x86, instructions such as `push`, `pusha`, and `enter` can touch memory slightly below the architectural value of `ESP` before the stack pointer itself is updated. Linux uses the same idea: a strict `CR2 >= ESP` check rejects valid stack growth patterns.

If the faulting page lies:

- above `stack_floor`,
- below the current `stack_low_limit`,
- below `USER_STACK_TOP`, and
- no more than 32 bytes below the saved user `ESP`,

then the handler allocates and maps zeroed pages from the current `stack_low_limit - PAGE_SIZE` downward until the faulting page itself is covered, updating `cur->stack_low_limit` after each successful step.

The common case is still one page per fault, but the implementation is slightly more forgiving than that wording suggests: if the process jumps several pages downward within the legal stack window, one fault can backfill the whole missing run in a single pass. Either way, the stack keeps growing until it reaches the 256 KB cap, at which point the next fault below the reserved window falls through to SIGSEGV like any other invalid access.

Anonymous `mmap()` Regions Ride the Same Fault Path

The new `SYS_MMAP` interface deliberately reuses the same machinery rather than inventing a second allocator. A successful anonymous private `mmap()` call only records a generic VMA in the process descriptor. The returned virtual range is reserved immediately, but it is still backed by no present PTEs. The first touch to any page in that range faults exactly like a lazy heap touch does: the fault handler finds the generic VMA, sees that the mapping is anonymous and private, allocates one zeroed physical page, and installs it with permissions derived from the VMA flags.

`SYS_MUNMAP` tears the mapping down in the opposite order: it removes the VMA metadata first, then walks the covered PTEs and drops any physical frames that had actually been committed. `SYS_MPROTECT` keeps the same address range but rewrites the VMA flags and any present PTE permissions so future faults and copy-on-write decisions observe the new policy. That means heap, stack, and anonymous mappings are all now governed by one fault-time policy engine rather than by three unrelated code paths.

Present Write Faults Usually Mean Copy-on-Write or a Real Protection Error

The other half of the decision tree is usually `P = 1, W = 1`: the CPU found a present mapping, but the access was a write and the PTE did not permit it.

There is one notable exception before the copy-on-write logic runs. A first touch on an anonymous heap, stack, or `mmap()` page can still arrive as `P = 1` if the process is looking through the inherited supervisor-only identity map rather than through a user PTE. The fault handler checks for exactly that shape — present, not user, not copy-on-write, inside an anonymous private VMA — and routes it into the same lazy-allocation path as a not-present anonymous miss.

We reserve bit 9 in the PTE as `PG_COW`, an operating-system-defined flag the hardware ignores. The invariant is:

`PG_COW`	`PG_WRITABLE`	Meaning
0	1	ordinary writable mapping
1	0	copy-on-write mapping; a write fault should be resolved
0	0	genuinely read-only mapping; a write fault should fail

If the write fault hits a PTE with `PG_COW` clear, the fault is treated as a real protection failure and falls through to the normal `SIGSEGV` path. That is how writes into true read-only mappings still behave.

If `PG_COW` is set, the handler resolves the fault in one of two ways:

- If the physical frame's reference count is 1, no other process points at it. We simply clear `PG_COW`, set `PG_WRITABLE`, invalidate the TLB entry, and resume. No copy is needed because the faulting process already owns the last reference.
- If the reference count is greater than 1, we allocate a new physical frame, copy the old 4 KB page into it, rewrite the faulting PTE to point at the new frame with `PG_WRITABLE` set and `PG_COW` clear, invalidate the TLB entry, and decrement the old frame's reference count.

The important point is the control shape: the write fault is not automatically an error. It is the deferred completion of a copy we deliberately avoided doing earlier.

Wild Pointers Still Become SIGSEGV

Demand paging only works because the handler keeps the old failure path intact for addresses outside the process's contract.

These cases still fail:

- a not-present user fault outside the heap and outside the legal stack window,
- a write through a present but genuinely read-only PTE,
- any ring-0 page fault, and
- any out-of-memory case while trying to allocate a demand page or a private CoW copy.

That last case is worth making explicit. We don't panic when a copy-on-write promotion runs out of memory. We also don't leave the process half-updated. The handler returns failure, `isr_handler` records the user fault, and the process takes the same SIGSEGV path as any other unrecoverable memory access. This matches the design principle the Linux memory manager uses too: out-of-memory during fault resolution is a process-level failure, not necessarily a kernel-level one.

Where the Machine Is by the End of Chapter 25

The CPU's vector-14 path is no longer just a crash chute. Ring-3 page faults now go through a recovery path that reads `CR2` and the page-fault error code, consults the process's VMA table, and decides whether the fault means "allocate a heap page", "grow the stack", "back an anonymous `mmap()` page", "replace a supervisor-only shadow mapping with a user page", "resolve copy-on-write", or "deliver `SIGSEGV`".

`SYS_BRK` now grows the heap lazily by recording a larger reservation without allocating pages, and shrinking the heap frees only the pages that were actually committed. The user stack starts with the same four eager pages as before, but the address-space contract reserves a 64-page window and the fault handler can backfill one or more stack pages down to the faulting address in a single recovery. A small per-process VMA table now gives the kernel a unified description of heap, stack, and anonymous `mmap()` regions. The old signal path still exists, but it is now the fallback when a fault is truly invalid rather than merely incomplete.

Chapter 26 — Copy-on-Write Fork

Fork as a Copy That Usually Does Not Happen

Chapter 25 left us with a demand-paging fault handler that can commit heap and stack pages lazily. In the eager `fork()` model from Chapter 15, we paid the full cost of independence before either process proved it needed it. Every mapped user page in the parent was copied into a fresh physical frame for the child, even though the common Unix pattern is `fork(); exec();` — the child often throws away the copied address space almost immediately and replaces it with a new program image.

Copy-on-write changes that trade-off. Instead of copying pages at fork time, we share them first and delay the expensive part until the first write. Parent and child leave `fork()` with the same virtual addresses pointing at the same physical frames. The pages are made read-only and tagged as copy-on-write. A later write fault is what forces the split.

Linux does exactly this for ordinary anonymous memory, and for the same reason: the cheap case should be the common case. A fork that only reads, or a fork followed quickly by `exec()`, should not allocate and copy megabytes of memory that no process will ever mutate.

Shared Pages Need a Real Ownership Count

The moment two page tables point at the same physical frame, the old bitmap-only physical allocator stops being sufficient. A single bit can answer "is this frame free?", but it cannot answer "how many live mappings still refer to it?" That question matters in three places now:

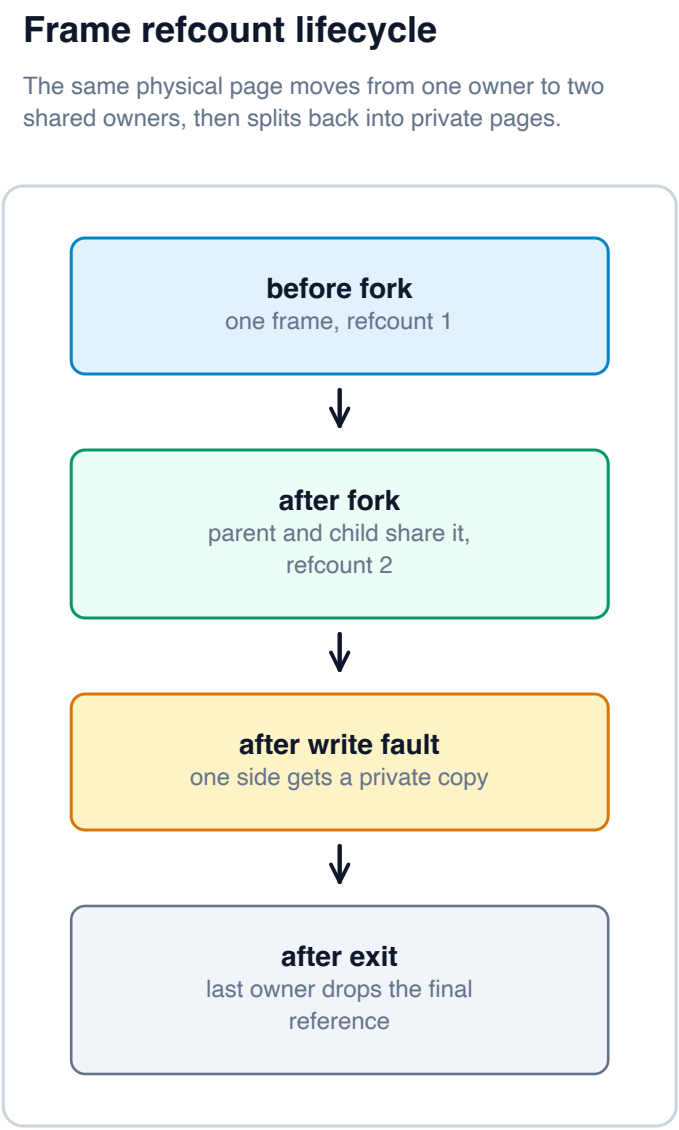
- when `fork()` shares a frame into a child,
- when a copy-on-write write fault stops using the old shared frame, and
- when a process exits and tears down its page tables.

The physical memory manager therefore grows a second array beside the bitmap:

Structure	Granularity	Meaning
`bitmap[]`	1 bit per page	whether the page is currently allocatable
`refcount[]`	1 byte per page	how many live references still point at the page

Allocating a frame now does two things at once: it marks the bitmap entry used and sets the new frame's reference count to 1. Later increments and decrements adjust that count. A frame becomes free again only when the count reaches zero, at which point the bitmap bit is cleared too.

The count saturates at 255. That value is also used to pin pages the kernel must never free through normal teardown, such as the kernel image, the identity-mapped page directory and page tables, the low-memory reserved range, and the kernel heap region. In other words, the refcount array is not only about sharing user pages. It also gives the kernel a way to say, "this page is permanently reserved; do not recycle it through the normal user-memory path."



Chapter 26, Figure 1

The process teardown path from Phase 0 now leans on the same mechanism. Releasing a process's user address space drops one reference to each user frame rather than unconditionally freeing it, so a shared frame is not released prematurely and a private frame becomes free immediately.

`fork()` Now Clones Page Tables, Not User Frames

The shape of the fork-time address-space clone changes completely. It still allocates a fresh page directory for the child, and it still allocates fresh page tables for any PDE that contains user mappings. What it no longer does is allocate a fresh 4 KB frame for every user PTE.

The new algorithm is:

1. Allocate a new child page directory.
2. Walk every present PDE in the parent.
3. If the page table contains no user PTEs, keep sharing that kernel-only page table exactly as before.
4. If the page table does contain user mappings, allocate a fresh child page table and copy the parent's page table entries into it.
5. For each present user PTE:
   - if the PTE is writable, clear `PG_WRITABLE` and set `PG_COW` in the parent,
   - copy the resulting PTE into the child page table,
   - increment the physical frame's reference count.

Kernel entries inside mixed PDEs are preserved verbatim. That point matters because PDE 1 can contain both user text around `0x00400000` and kernel identity mappings in the same page table. The loop therefore checks `PG_USER` on each PTE rather than assuming an entire PDE is user-owned.

The result is that the parent and child get:

- separate page directories,
- separate user page tables,
- the same user *frames* underneath those page tables.

The page tables are private because each process will later update its own PTEs independently. The frames are shared because no physical copy is needed until one of the processes writes.

What the Shared Mappings Look Like

The writable-to-copy-on-write transition is encoded entirely in the PTE flags. Before `fork()`, an anonymous heap page in the parent looks like an ordinary user-writable mapping:

Flag	Value
`PG_PRESENT`	1
`PG_USER`	1
`PG_WRITABLE`	1
`PG_COW`	0

After `fork()`, the parent's PTE is rewritten to:

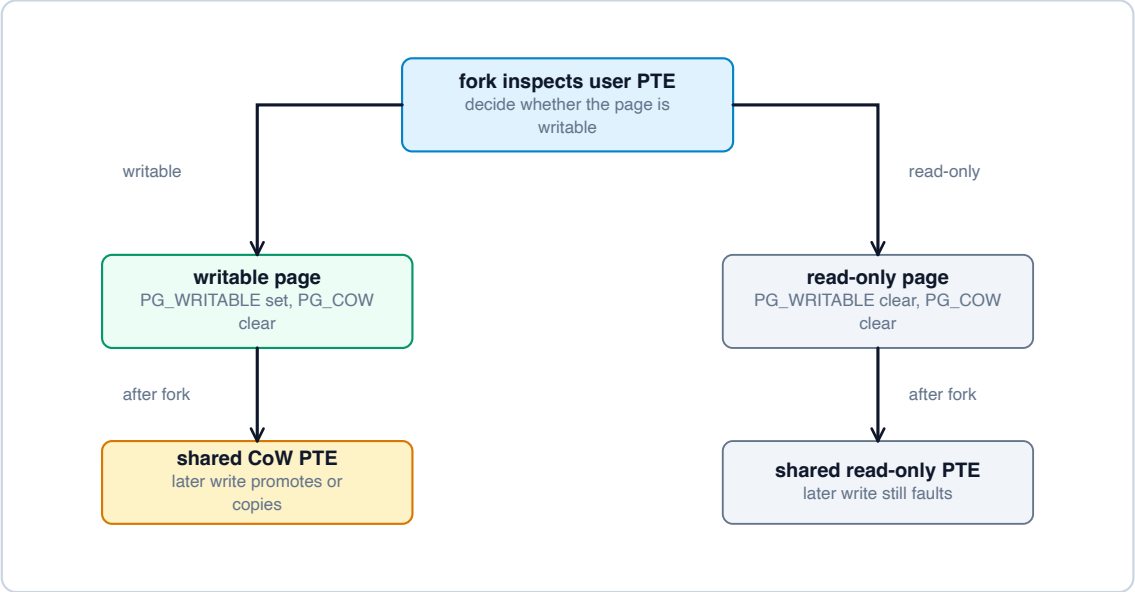
Flag	Value
`PG_PRESENT`	1
`PG_USER`	1
`PG_WRITABLE`	0
`PG_COW`	1

The child receives the same PTE bits. Both page tables now point at the same physical frame, and the physical memory manager's count for that frame is 2.

Read-only pages follow a slightly different path. If a user PTE was already non-writable before `fork()` — for example, a code page or true read-only data — the kernel leaves `PG_COW` clear and simply shares the same frame into the child while incrementing the physical reference count. That means one rule remains intact: writing to a genuinely read-only page still produces SIGSEGV rather than copy-on-write promotion. The presence or absence of `PG_COW` is our explicit promise about whether a write fault is recoverable.

PTE state machine

Writable pages become copy-on-write during fork(), while true read-only pages stay read-only.



Chapter 26, Figure 2

The First Write Splits the Shared Frame

Once parent and child return from `fork()`, reads are cheap. Both page tables translate the same virtual page to the same physical frame, and a plain load works without any involvement from the kernel. The deferred cost appears only when one process writes.

Suppose the parent writes to a shared heap page first:

1. The CPU performs the page walk, finds a present PTE, sees that the write bit is clear, and raises vector 14 with `P=1, W=1, U=1`.
2. The fault handler inspects the live PTE and sees `PG_COW` set.
3. The handler reads the physical frame's reference count.

From there, two cases are possible.

Case 1: the refcount is greater than 1.
This means some other mapping still points at the same frame. The handler allocates a new 4 KB physical page, copies the old frame into it, rewrites the faulting process's PTE to point at the new frame with `PG_WRITABLE` set and `PG_COW` clear, invalidates the TLB entry, and decrements

the old frame's refcount.

The other process keeps pointing at the original frame. The faulting process now has a private writable copy. Both are correct, and neither saw the copy until the write demanded it.

Case 2: the refcount is 1.
This means the faulting process already owns the last reference to the old frame. Perhaps the sibling process exited and tore down its page tables first. In that case, there is nothing left to copy. The handler simply flips the PTE back to writable in place by clearing `PG_COW` and setting `PG_WRITABLE`, invalidates the TLB entry, and resumes.

This is the quiet second half of copy-on-write and the reason the refcount matters so much. A write fault does not always imply an actual data copy. Sometimes it only means, "the page used to be shared, but it is private now, so make the PTE match reality again."

Out-of-Memory and Partial Failure Paths

Two failure paths become important once `fork()` stops being a monolithic memcpy loop.

The first is **partial failure during the clone itself**. The clone uses a two-pass strategy so that an OOM is trivially rolled back. Pass 1 allocates every child page table the clone will need without touching any parent PTEs. If any allocation fails, the kernel frees the child page tables allocated so far plus the child page directory and returns failure — the parent's address space is completely unmodified. Only after every allocation succeeds does Pass 2 apply CoW flags to the parent and child PTEs and bump frame refcounts. Because Pass 2 cannot fail, the clone either completes fully or never begins modifying shared state.

The second is **out-of-memory during a later CoW write fault**. That case is handled in Chapter 25's fault path: if the fault handler cannot allocate a private replacement frame, it returns failure and the process falls through to SIGSEGV. The rest of the system remains consistent because the original shared frame and original PTE are left untouched.

Both failure paths follow the same design rule: either the fork or the write fault completes all the way to a consistent new mapping, or we leave the old state in place and report failure to the process.

Reclaiming Shared Pages on Exit

Copy-on-write only stays safe if every path that drops an address space also drops its references to shared frames.

That is why Phase 0 landed first. A process still becomes `PROC_ZOMBIE` before it is fully reaped, because it is still executing on its own kernel stack at the moment it exits. The real cleanup happens later when a waiting parent observes the zombie state and reaps it. At that point the kernel:

1. reads the encoded exit status,
2. drops the user-space frame references,
3. frees the kernel stack, and
4. returns the process-table slot to `PROC_UNUSED`.

The crucial interaction with copy-on-write is that exit no longer "frees" a shared frame in the old absolute sense. It drops one reference. If the sibling process still points at the frame, the reference count remains non-zero and the frame stays live. If the exiting process was the last owner, the count reaches zero and the physical memory manager puts the frame back on the free list.

How the Common Unix Case Gets Cheaper

The classic shell pattern is:

1. the shell calls `fork()`,
2. the child rearranges file descriptors if needed,
3. the child calls `exec()`.

Under the eager model, step 1 copied the shell's full address space even though step 3 was about to discard it. Under copy-on-write, step 1 mostly allocates page-table pages and increments refcounts. The expensive 4 KB data copy moves to the uncommon case where either side actually writes into a shared page before the child calls `exec()`.

That does not make `fork()` free. We still allocate a new page directory, new user page tables, and a new kernel stack for the child. We still copy the saved syscall frame so that the parent and child can return with different `EAX` values. But the cost now scales with the number of page tables that contain user mappings, not with the number of user frames those tables point at.

For real workloads, that is the right asymmetry. A process can reserve a large heap, read from it, fork, and continue almost immediately. Only the pages that either side later dirties will be copied.

Where the Machine Is by the End of Chapter 26

`fork()` no longer deep-copies every user frame up front. It allocates a new page directory and private user page tables for the child, rewrites writable shared pages as read-only copy-on-write mappings in both address spaces, and increments the physical frame's reference count. Read-only user pages are also shared, but they remain genuinely read-only rather than becoming copy-on-write.

The physical memory manager now tracks frame ownership explicitly with a refcount array, so shared frames survive until the last mapping drops them. A write fault against a copy-on-write mapping either promotes the page in place when the refcount has already fallen to 1 or allocates a private replacement frame and copies 4 KB of data when the page is still shared. Process exit, heap shrink, and fork failure cleanup all use the same reference-counted release path, which means the memory subsystem now has a consistent answer to the question "who still owns this frame?"

Chapter 27 — Debugging

Why Kernel Debugging Feels Different

Chapter 26 completed the copy-on-write fork implementation. Throughout all of that work, debugging the kernel required a different mindset from debugging a user program. A user program is comfortable to debug because the operating system is already alive around it — the debugger can stop the process, inspect registers, read memory, and resume execution while the kernel keeps the rest of the machine stable. Kernel debugging removes that safety net. When the kernel itself is the thing being debugged, there is no deeper layer to catch mistakes. A bad stack pointer, an invalid descriptor-table entry, or an interrupt delivered too early can tear control away from the debugger and leave the CPU executing nonsense.

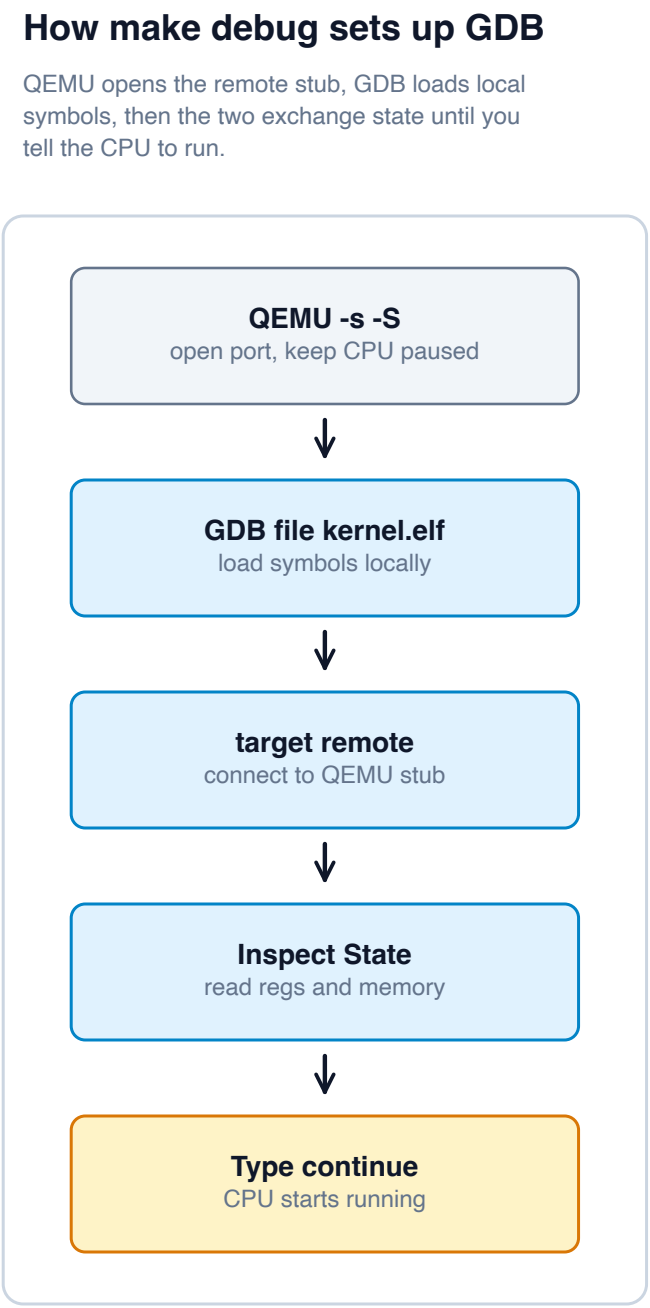
That is why early-boot debugging is mostly about establishing a valid exception path before doing anything ambitious. The debugger does not only need symbols. It needs the target CPU to be able to enter a trap handler and return from it cleanly.

What Happens When GDB Attaches

GDB does not talk to the kernel directly. It talks to QEMU's built-in remote stub. The `make debug` target starts QEMU with `-s -S`. `-s` opens the GDB remote port on `localhost:1234`. `-S` starts the virtual CPU paused, before the guest executes any further instructions.

GDB is then started with a short command script. `file kernel.elf` loads symbols into GDB itself; that step is local and does not send the binary into the guest. It simply tells GDB how to decode addresses as kernel functions, source lines, and variables. Next, `target remote localhost:1234` connects GDB to QEMU's remote stub. From then on, GDB can ask QEMU for register contents, read guest memory, and send control commands such as "resume", "single-step", or "insert breakpoint".

At this stage the guest is still paused. Attaching does not automatically run the kernel. That is why the Makefile ends with an explicit `continue`. Up to that moment, the kernel does not need to do anything special. No exception has to be delivered inside the guest, no IDT entry has to run, and no interrupt handler has to exist yet. The CPU is simply being held still from the outside while GDB learns how to interpret the machine state.



Chapter 27, Figure 1

What Happens When a Breakpoint Hits

When execution reaches a breakpoint, the machine stops and control goes back to GDB. A hardware breakpoint can do this very early in boot because the stop is arranged by the debugger and the CPU's debug hardware, not by an ordinary device IRQ such as the timer or keyboard. That is why `hbreak start_kernel` can stop at the entry to the function even when almost nothing is

initialized yet.

That first stop is useful, but it does not mean the machine is ready for ordinary source-level stepping. Hitting one breakpoint and taking the *next* stop are two different problems. The first stop only proves that the debugger could halt the CPU. The next stop depends on whether the guest itself can take an exception and get back out of it cleanly.

The boot path now separates interrupt setup into two distinct phases:

1. `gdt_init()` installs the kernel's final **GDT** (Global Descriptor Table) and **TSS** (Task State Segment).
2. `idt_init_early()` builds the **IDT** (Interrupt Descriptor Table) and loads it with `lidt`.
3. The kernel registers IRQ handlers and initializes early devices.
4. `interrupts_enable()` remaps the PIC, programs the PIT, and executes `sti`.

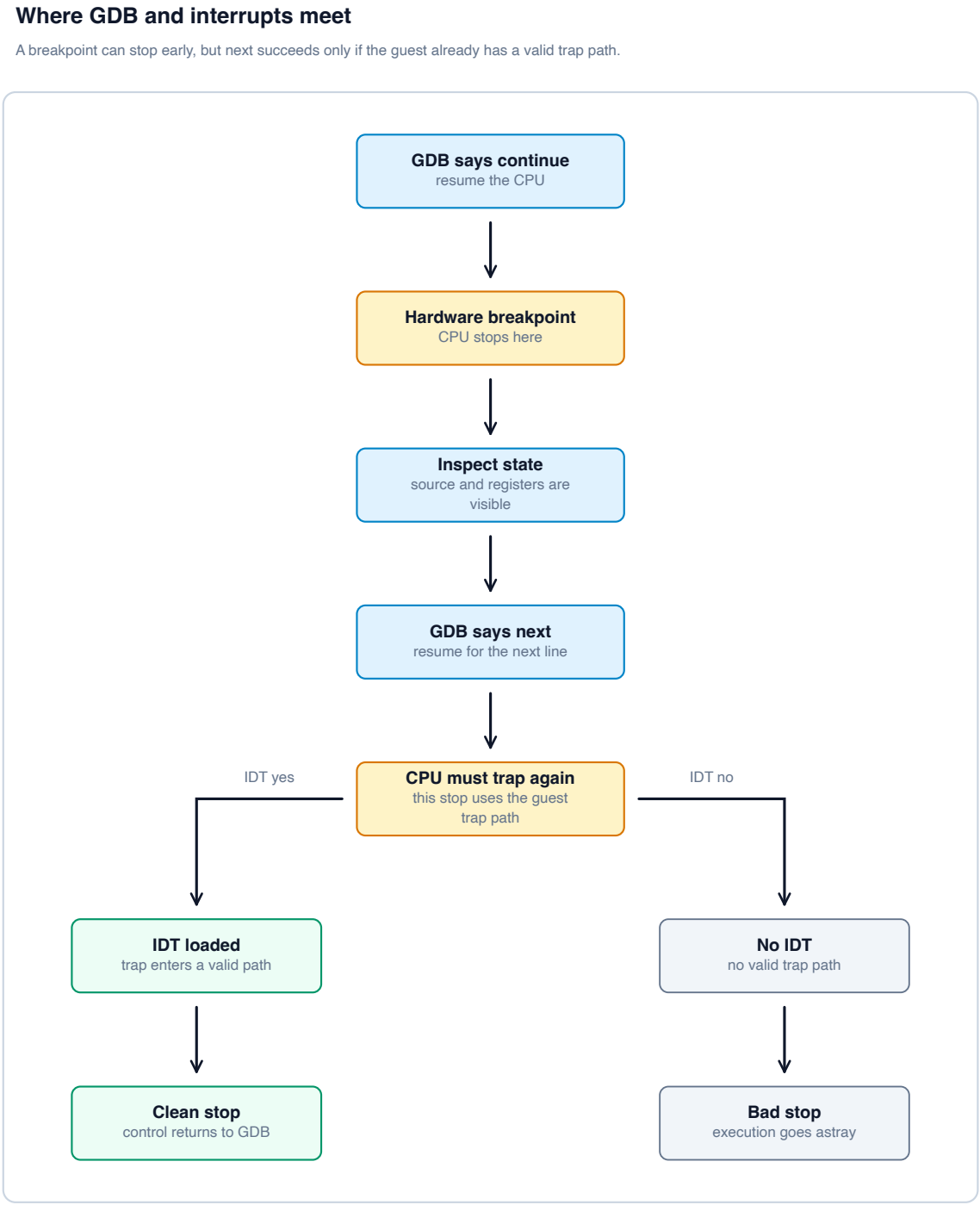
That split exists for debugging as much as for correctness. `lidt` is enough to tell the CPU where exception and trap handlers live. `sti` is the later step that allows real hardware devices to interrupt the kernel asynchronously. In debugger terms: GDB needs a valid exception path early, but it does not need timer ticks or keyboard IRQs to be enabled.

What Happens When You Type `next`

When GDB says `next` or `step`, it is not performing magic from outside the machine. It resumes the target and arranges for execution to stop again after a small amount of code has run. On x86 that usually means a trap or temporary breakpoint exception of some kind will be delivered back through the CPU's normal exception machinery.

This is the moment where interrupts and debugging meet. GDB asks the machine to run. The CPU runs guest code. Then the CPU needs to stop again by taking a trap. That trap is not a special debugger-only side channel inside the guest; it still enters through the same kind of exception-routing machinery the kernel uses for faults and software traps. If the kernel has not loaded its own IDT yet, there is no valid kernel-owned destination for that exception. The debugger may show a clean stop at the current instruction because QEMU's remote stub can seize the CPU externally, but the *next* stop still depends on the target being able to take a trap correctly. If it cannot, execution may appear to jump into low memory, firmware code, or an unknown address instead of returning to the expected source line.

The practical consequence is simple: stopping at the opening brace of `start_kernel` is too early for reliable source-level stepping. Stopping at `idt_init_early`, or after it has returned, is usually early enough.



Chapter 27, Figure 2

A Practical Early-Boot GDB Workflow

Our Makefile starts QEMU paused under the GDB remote stub and sets a hardware breakpoint on `idt_init_early`. That choice is deliberate: it lands at the earliest point where source-level

stepping is expected to work consistently.

The safe early workflow is:

1. Break at `idt_init_early`.
2. Continue until the breakpoint hits.
3. Step through `idt_init_early` or use `finish` to return to `start_kernel`.
4. Continue stepping normally once the IDT is live.

If you truly need to inspect the very first instructions of `start_kernel`, prefer hardware breakpoints plus `continue` over source-level `next` at the function entry. Before the IDT is loaded, GDB may be able to stop *at* a hardware breakpoint, but it cannot rely on the guest to deliver the trap needed for the *next* source-level stop.

What `sti` Changes

Loading the IDT does not by itself make the machine fully interrupt-driven. Until `sti` executes, the CPU's interrupt flag is still clear, so external hardware IRQs remain blocked. That delay is important. The timer, keyboard, and disk should not be able to interrupt the kernel until their handlers, dispatch tables, and controller configuration are all ready. Exception delivery and debugger traps can be usable before this point; ordinary hardware interrupts should not be.

This is also why the kernel has two different debugging phases:

- Before `idt_init_early()`: safe stopping is limited; source-level stepping is fragile.
- After `idt_init_early()` but before `sti`: exception and debugger-trap handling are available, but device interrupts are still quiescent.
- After `interrupts_enable()`: the machine is in its normal asynchronous state; timer ticks, keyboard interrupts, and ATA IRQs may all arrive.

The middle phase is especially pleasant for debugging because the exception path is valid while the machine is still mostly deterministic.

Logs Still Matter

A debugger is not the only source of truth. We deliberately keep several output paths alive because the most interesting failures are often the ones that make interactive stepping awkward.

- The VGA console shows the ordinary boot narrative.
- COM1 serial logging survives many failures that leave the screen stale.
- QEMU's debug console on port `0xE9` is even simpler and often remains usable when other output paths are compromised.

For fatal kernel faults, the panic path writes directly to serial and debugcon rather than depending on the normal VGA code path. That is why a crash can still leave readable diagnostics behind even when the visible screen looks frozen.

Live Debugging and Post-Mortem Debugging

Not every bug is easiest to catch live. Early-boot ordering bugs, descriptor-table mistakes, and interrupt-path failures are best investigated with QEMU and GDB attached to a running kernel.

User-process crashes later in the booted system are often easier to inspect after the fact from a core dump.

The two techniques complement each other:

- Live debugging answers "what is the kernel doing right now?"
- Core dumps answer "what state did this process die in?"

That distinction is why we have both a live GDB workflow and the ELF core-dump writer from Chapter 24. The debugger attached to QEMU is the tool for bringing the machine up. The core file is the tool for understanding why a process came down.

The process-memory forensics path now gives an explicit end-to-end check:

1. Inspect `/proc/<pid>/vmstat` for compact totals and `/proc/<pid>/fault` for the current fault snapshot (`State: none` before a crash).
2. Trigger a deliberate user crash (for example, `crash badptr`) to generate `core.<pid>`.
3. Extract the core file and inspect its notes.
4. Confirm the three `DRUNIX` text notes (`vmstat`, `fault`, and `maps`) match the same model-backed views from live procfs.

In other words, `/proc/<pid>/maps` remains the detailed live layout view, while `vmstat` and `fault` provide condensed status; after a crash, all three survive as text notes in the core file for post-mortem inspection.

Where the Machine Is by the End of Chapter 27

Debugging the kernel is no longer just "attach GDB and hope". The boot sequence now exposes an explicit early-debugging boundary: once `idt_init_early()` has loaded the IDT, traps and exceptions have a valid entry path even though hardware interrupts are still disabled. The later `interrupts_enable()` step turns the machine from a mostly linear boot path into a fully asynchronous system. Alongside serial logs, debugcon output, and ELF core dumps with `DRUNIX` memory-forensics notes aligned to `/proc/<pid>/vmstat`, `/proc/<pid>/fault`, and `/proc/<pid>/maps`, that split gives the kernel both live and post-mortem debugging tools that are predictable enough to rely on.